

Java Interview Master Guide

Interview-ready answers for Java Developer / Software Engineer freshers

Fundamentals · OOP · Strings · Collections · HashMap Internals · Exceptions
Multithreading · Java 8 Streams · JVM & GC · JDBC · Design Basics

50+ Trick Questions · 50 Coding Problems · Top 100 Questions · Revision Sheets

Prepared for Abhinav Danthuri

Contents

How to Use This Guide	3
1 Java Fundamentals	4
2 Variables, Data Types & Operators	9
3 Control Statements	14
4 OOPs Concepts	16
5 Important Java Keywords	25
6 Strings	29
7 Arrays	32
8 Collection Framework	34
9 HashMap Internal Working	41
10 Exception Handling	45
11 Multithreading	49
12 Java 8+ Features	54
13 Java Memory & JVM	59
14 Garbage Collection	61
15 Object Class Methods	63
16 Packages & Access Control	65
17 Generics	66
18 Enums & Inner Classes	68
19 JDBC & Database Basics	70
20 Common Design & Concept Questions	74
21 Java Interview Trick Questions	77
22 50 Most Repeated Java Fresher Coding Questions	81
23 Top 100 Java Interview Questions for Freshers	101
24 Rapid Revision	105
25 If the Interviewer Asks Me Anything About Java	108

How to Use This Guide

Every topic is tagged so you know where to spend your time:

- **MUST-KNOW** — expect it in almost every Java fresher interview. Know the spoken answer cold.
- **IMPORTANT** — commonly asked; know it well enough to explain with an example.
- **OPTIONAL** — good to know; asked in some interviews or as a follow-up.

Each concept follows the same pattern:

Interview answer: the 2-4 sentence definition you actually say out loud. It is written the way a confident fresher speaks — not copied from a textbook.

Then **Simple explanation**, **Example**, **Follow-ups** (what the interviewer asks next) and **Common mistakes**.

Tip for speaking the answers: Start with the definition, add *one* line on why it matters, then offer an example. That three-step shape (What → Why → Example) works for almost every Java question.

1 Java Fundamentals

1.1 What is Java? MUST-KNOW

Interview answer: “Java is a high-level, object-oriented, platform-independent programming language. We write code once, compile it into bytecode, and that bytecode runs on any machine that has a JVM. That’s why Java is known for ‘Write Once, Run Anywhere’. It’s widely used for backend systems, Android apps and enterprise applications.”

Simple explanation: Java was built so that the same program runs on Windows, Linux or Mac without changes. It does this by not compiling directly to machine code — it compiles to an in-between format (bytecode) that a JVM on each machine understands.

Real-world: Banking backends, e-commerce systems (order services, payment services), Android apps, big-data tools like Kafka and Hadoop are all written in or run on Java.

Follow-ups: Who developed Java? (James Gosling at Sun Microsystems, released 1995; now owned by Oracle.) Is Java 100% object-oriented? (No — see below.)

Is Java 100% object-oriented?

Interview answer: “No. Java is not purely object-oriented because it has primitive data types like int, char and boolean which are not objects, and it also supports static members that can be used without creating an object.”

1.2 Features of Java MUST-KNOW

Interview answer: “The main features of Java are: it’s simple, object-oriented, platform independent, secure, robust, multithreaded, and it has automatic memory management through garbage collection. Platform independence and robustness are the biggest reasons it’s used so heavily in enterprise systems.”

Feature	What it means (one line)
Simple	No pointers, no operator overloading, no manual memory freeing
Object-oriented	Everything is organised around classes and objects
Platform independent	Bytecode runs on any OS that has a JVM
Secure	No direct pointer access; bytecode verifier; runs inside JVM sandbox
Robust	Strong type checking, exception handling, garbage collection
Multithreaded	Built-in support for running tasks in parallel
Portable	Fixed data-type sizes (int is always 32-bit) on every platform
High performance	JIT compiler converts hot bytecode to native code at runtime
Distributed	Built-in networking APIs (RMI, sockets, HTTP)
Dynamic	Classes are loaded at runtime when needed

Common mistake: Saying “Java is platform independent, so the JVM is too.” The JVM is **platform dependent** — each OS has its own JVM. Only the *bytecode* is platform independent.

1.3 Why is Java Platform Independent? MUST-KNOW

Interview answer: “Java is platform independent because the Java compiler doesn’t produce machine code — it produces bytecode, a .class file. That bytecode isn’t tied to any OS. Every operating system has its own JVM, and the JVM converts that bytecode into the machine’s native code. So the same .class file runs anywhere a JVM exists.”

Real-world analogy: Bytecode is like an English document. Each country (OS) has its own translator (JVM). You write the document once; each translator converts it for their audience.

Follow-up: *Is the JVM platform independent?* — “No. The JVM is platform dependent; there’s a separate JVM for Windows, Linux and Mac. That’s exactly what makes the bytecode platform independent.”

1.4 JVM, JRE and JDK MUST-KNOW

JVM (Java Virtual Machine)

Interview answer: “The JVM is the engine that runs Java bytecode. It loads the .class files, verifies the bytecode, executes it using an interpreter and JIT compiler, and manages memory through garbage collection. It’s the part that actually makes Java platform independent.”

What the JVM does internally (simple):

1. **Class Loader** — loads .class files into memory.
2. **Bytecode Verifier** — checks the bytecode is valid and safe.
3. **Execution Engine** — runs the code using:
 - **Interpreter** — executes bytecode line by line.
 - **JIT (Just-In-Time) compiler** — compiles frequently-used (“hot”) code to native machine code for speed.
4. **Garbage Collector** — frees memory of unused objects.
5. **Runtime data areas** — Heap, Stack, Method Area/Metaspace, PC register, Native method stack.

JRE (Java Runtime Environment)

Interview answer: “The JRE is what you need to *run* a Java program. It contains the JVM plus the core Java libraries like java.lang and java.util. It doesn’t have development tools like the compiler.”

JDK (Java Development Kit)

Interview answer: “The JDK is what you need to *develop* Java programs. It contains the JRE plus development tools like javac for compiling, java for running, javadoc, jar and the debugger. As a developer, we install the JDK.”

JVM vs JRE vs JDK

	JVM	JRE	JDK
Full form	Java Virtual Machine	Java Runtime Environment	Java Development Kit
Purpose	Executes bytecode	Runs Java programs	Develops + runs Java programs
Contains	Class loader, execution engine, GC	JVM + libraries	JRE + dev tools (javac, jar, javadoc, jdb)
Who needs it	— (part of JRE)	End users	Developers
Platform dependent?	Yes	Yes	Yes

Easy formula to say: **JDK = JRE + Development tools**, and **JRE = JVM + Libraries**.

Follow-up: *Since Java 11, is there a separate JRE download?* — Oracle stopped shipping a standalone JRE from Java 11; you install the JDK (or build a custom runtime with `jlink`). Good to mention, but the concept of JRE is still asked.

1.5 Compilation and Execution Process MUST-KNOW

Interview answer: “First we write source code in a .java file. The javac compiler compiles it into bytecode and stores it in a .class file. When we run the program with the java command, the JVM’s class loader loads the .class file, the bytecode verifier checks it, and the execution engine runs it — interpreting it and using the JIT compiler to convert frequently used code into native machine code.”

```

Hello.java  --javac-->  Hello.class (bytecode)  --java (JVM)-->  Machine code  -->  Output
|              |              |              |              |
Source code    Platform independent          Class Loader → Verifier →
                                              Interpreter + JIT → CPU

```

```

// Hello.java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java");
    }
}

```

```

javac Hello.java    # creates Hello.class
java Hello          # JVM runs the bytecode

```

Bytecode

Interview answer: “Bytecode is the intermediate, platform-independent code that the Java compiler generates and stores in .class files. It’s not machine code — it’s a set of instructions that only the JVM understands and executes.”

javac

Interview answer: “javac is the Java compiler that comes with the JDK. It checks the source code for syntax and type errors and converts a .java file into a .class file containing bytecode.”

Why public static void main(String[] args)? MUST-KNOW

Interview answer: “main is the entry point the JVM calls to start a program. It’s **public** so the JVM can access it from outside the class, **static** so the JVM can call it without creating an object, **void** because it doesn’t return anything to the JVM, and **String[] args** holds command-line arguments.”

Follow-ups: Can we overload main? (Yes, but JVM only calls `main(String[])`.) What if main is not static? (Compiles, but the classic launcher refuses to run it. Java 25 finalised “instance main methods” that relax this for simple programs, but the classic answer is what’s expected.)

1.6 Java Architecture IMPORTANT

Interview answer: “Java architecture has three layers: the JDK for development, the JRE for runtime, and the JVM at the core for execution. Inside the JVM we have the class loader subsystem, runtime memory areas like heap and stack, and the execution engine with the interpreter, JIT compiler and garbage collector.”

```

+----- JDK -----+
| javac, jar, javadoc, jdb ... |
+----- JRE -----+
| Core libraries (java.lang, java.util, java.io ...) |
+----- JVM -----+
| | Class Loader → Bytecode Verifier | | | | | |
| | Memory: Method Area | Heap | Stack | PC | Native | |
| | Execution Engine: Interpreter + JIT + GC | |
+-----+

```

Class loader types (follow-up): Bootstrap (core classes like `java.lang`), Platform/Extension, Application/System (your classes). They use the **parent delegation model** — a loader first asks its parent to load a class.

1.7 Java Editions OPTIONAL

Edition	Used for
Java SE (Standard Edition)	Core Java — language, collections, threads, I/O
Java EE / Jakarta EE (Enterprise)	Web & enterprise — Servlets, JPA, JMS
Java ME (Micro Edition)	Embedded / small devices
JavaFX	Rich desktop UI

1.8 Java Versions and LTS IMPORTANT

Interview answer: “Java releases a new version every six months, and every few releases one is marked LTS — Long-Term Support — which gets updates and security patches for years. Companies usually run on LTS versions like Java 8, 11, 17 and 21. Java 8 was the big one because it introduced lambdas and the Stream API.”

Version	Key features (the ones interviewers mention)
Java 8 (LTS, 2014)	Lambdas, Stream API, functional interfaces, Optional, default/static interface methods, new Date-Time API
Java 11 (LTS, 2018)	<code>var</code> in lambdas, new String methods (<code>isBlank</code> , <code>strip</code> , <code>lines</code> , <code>repeat</code>), HTTP Client
Java 17 (LTS, 2021)	Sealed classes, records (final in 16), pattern matching for <code>instanceof</code> , text blocks
Java 21 (LTS, 2023)	Virtual threads, pattern matching for <code>switch</code> , record patterns, sequenced collections
Java 25 (LTS, 2025)	Latest LTS at the time of writing

Tip: If asked “which version do you use?”, say “I’ve mainly worked with Java 17 / 21, and I’m comfortable with Java 8 features like streams and lambdas since they’re used everywhere.”

1.9 Java vs C/C++ MUST-KNOW

Point	C++	Java
Platform	Platform dependent (compiled to machine code)	Platform independent (bytecode + JVM)
Pointers	Supported	Not exposed to programmer
Memory management	Manual (<code>new</code> / <code>delete</code>)	Automatic (garbage collection)
Multiple inheritance	Supported through classes	Only through interfaces
Operator overloading	Supported	Not supported (except <code>+</code> for String)
Destructors	Yes	No (GC handles it)
Compilation	Compiler only	Compiler + JVM (interpreter + JIT)
Structures/unions	Yes	No
Call by reference	Supported	Only pass-by-value

Interview answer: “The main differences are: Java is platform independent while C++ is platform dependent, Java has no explicit pointers, Java uses automatic garbage collection instead of manual memory management, and Java doesn’t support multiple inheritance through classes or operator overloading. That makes Java simpler and safer, while C++ gives more low-level control and speed.”

Is Java pass-by-value or pass-by-reference? MUST-KNOW

Interview answer: “Java is always pass-by-value. For primitives, a copy of the value is passed. For objects, a copy of the reference is passed — so the method can change the object’s data, but if it reassigns the reference to a new object, the caller’s reference doesn’t change.”

```
static void change(StringBuilder sb) {  
    sb.append(" World"); // modifies the same object -> visible to caller  
    sb = new StringBuilder("X"); // only the local copy of the reference changes  
}  
StringBuilder s = new StringBuilder("Hello");  
change(s);  
System.out.println(s); // Hello World
```

1.10 Advantages and Limitations of Java IMPORTANT

Advantages	Limitations
Platform independent	Slower startup than C/C++ (JVM warm-up)
Automatic memory management	Higher memory usage
Huge ecosystem (Spring, Hibernate, Maven)	More verbose code than Python/Kotlin
Strong typing → fewer runtime bugs	GC pauses can affect latency-sensitive apps
Excellent multithreading support	No low-level hardware control
Backward compatible, very stable	

2 Variables, Data Types & Operators

2.1 Variables MUST-KNOW

Interview answer: “A variable is a named memory location that stores a value. In Java every variable has a data type. There are three kinds: local variables inside methods, instance variables that belong to each object, and static variables that are shared across all objects of a class.”

Type	Declared	Stored	Default value?	Lifetime
Local	Inside method/block	Stack	No — must be initialised	Until method ends
Instance	Inside class, no <code>static</code>	Heap (inside object)	Yes (0, null, false)	As long as object lives
Static	Inside class with <code>static</code>	Method area/Metaspace (class-level)	Yes	As long as class is loaded

```
class Student {
    static String college = "ACE"; // static: shared by all students
    String name;                  // instance: one per object
    void study() {
        int hours = 3;           // local: must initialise before use
    }
}
```

Common mistake: Using a local variable without initialising it → compile error “variable might not have been initialized”.

2.2 Constants IMPORTANT

Interview answer: “A constant is a variable whose value can’t change once assigned. In Java we create constants using the `final` keyword, and usually make them `static final` so there’s only one copy — for example, `public static final double PI = 3.14`.”

```
public static final int MAX_USERS = 100; // naming convention: UPPER_SNAKE_CASE
```

2.3 Primitive Data Types MUST-KNOW

Interview answer: “Java has eight primitive data types: `byte`, `short`, `int`, `long` for whole numbers; `float` and `double` for decimals; `char` for a single character; and `boolean` for true or false. They store actual values directly and are not objects.”

Type	Size	Range / values	Default	Example
byte	1 byte	-128 to 127	0	<code>byte b = 10;</code>
short	2 bytes	-32,768 to 32,767	0	<code>short s = 1000;</code>
int	4 bytes	about ±2.1 billion (-2^{31} to $2^{31}-1$)	0	<code>int i = 50000;</code>
long	8 bytes	-2^{63} to $2^{63}-1$	0L	<code>long l = 9999999999L;</code>
float	4 bytes	~6-7 decimal digits	0.0f	<code>float f = 3.14f;</code>
double	8 bytes	~15-16 decimal digits	0.0d	<code>double d = 3.14159;</code>
char	2 bytes	0 to 65,535 (Unicode)	'\u0000'	<code>char c = 'A';</code>
boolean	JVM-dependent (1 bit logically)	true / false	false	<code>boolean ok = true;</code>

Follow-ups:

- *Why is char 2 bytes in Java but 1 byte in C?* — “Because Java uses Unicode (UTF-16), so it can represent characters from all languages, not just ASCII.”
- *Why do we write L and f?* — Integer literals are `int` by default and decimal literals are `double` by default. `L` tells the compiler it's a `long`; `f` makes it a `float`.

float vs double IMPORTANT

	float	double
Size	4 bytes	8 bytes
Precision	~7 digits	~15 digits
Default for decimal literals?	No (needs <code>f</code>)	Yes
Use	Memory-sensitive (graphics, large arrays)	General calculations (default choice)

Interview answer: “float is a 32-bit single-precision decimal type and double is a 64-bit double-precision type. double is more precise and is the default for decimal numbers in Java. For money, we shouldn't use either — we use `BigDecimal` because floating-point numbers can't represent values like 0.1 exactly.”

```
System.out.println(0.1 + 0.2);           // 0.30000000000000004
BigDecimal a = new BigDecimal("0.1");
System.out.println(a.add(new BigDecimal("0.2"))); // 0.3
```

2.4 Non-Primitive (Reference) Data Types MUST-KNOW

Interview answer: “Non-primitive or reference types are types that refer to objects — like `String`, arrays, classes and interfaces. The variable doesn't hold the object itself; it holds a reference to the object stored in the heap. Their default value is `null`, and they come with methods we can call.”

Primitive	Non-primitive
Predefined by Java (8 types)	Created by programmer (except <code>String</code>)
Stores actual value	Stores reference (address) to an object
Cannot be null	Can be null
Has no methods	Has methods
Starts lowercase (<code>int</code>)	Starts uppercase by convention (<code>String</code>)
Size is fixed	Size depends on the object

2.5 Type Casting MUST-KNOW

Interview answer: “Type casting is converting a value from one data type to another. Widening casting — small type to large type, like `int` to `long` — happens automatically. Narrowing casting — large type to small type, like `double` to `int` — must be done manually and can lose data.”

```
// Widening (implicit) : byte -> short -> int -> long -> float -> double
int i = 100;
double d = i;           // 100.0, automatic

// Narrowing (explicit)
double price = 99.99;
int p = (int) price;    // 99 -> decimal part lost

int big = 130;
byte b = (byte) big;    // -126 -> overflow wraps around
```

Common trap:

```
byte a = 10, b = 20;
byte c = a + b;      // COMPILER ERROR: a + b becomes int
byte c = (byte)(a + b); // OK
```

“In arithmetic, byte, short and char are promoted to int first.”

2.6 Wrapper Classes, Autoboxing and Unboxing MUST-KNOW

Interview answer: “Wrapper classes are object versions of primitive types — Integer for int, Double for double, Character for char and so on. We need them because collections like ArrayList can only store objects, not primitives. Autoboxing is Java automatically converting a primitive into its wrapper, and unboxing is converting the wrapper back to a primitive.”

Primitive	Wrapper
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

```
List<Integer> list = new ArrayList<>();
list.add(5);        // autoboxing: int → Integer
int x = list.get(0); // unboxing: Integer → int

int n = Integer.parseInt("123"); // String → int
String s = String.valueOf(123);  // int → String
```

Classic trick question — Integer cache:

```
Integer a = 127, b = 127;
System.out.println(a == b); // true (cached between -128 and 127)
Integer c = 128, d = 128;
System.out.println(c == d); // false (different objects)
System.out.println(c.equals(d)); // true
```

Interview answer: “Java caches Integer objects from -128 to 127. So autoboxed values in that range point to the same object and == returns true. Outside that range, new objects are created, so == compares references and returns false. That’s why we should always compare wrappers with equals().”

Other pitfall: Unboxing a null wrapper throws NullPointerException: Integer x = null; int y = x;

2.7 Operators MUST-KNOW

Interview answer: “Operators are symbols that perform operations on variables and values. Java has arithmetic, relational, logical, bitwise, assignment, unary, ternary and instanceof operators.”

Category	Operators	Example
Arithmetic	+ - * / %	7 / 2 = 3, 7 % 2 = 1
Relational	== != > < >= <=	a > b → boolean
Logical	&& \ \ !	a > 0 && b > 0
Bitwise	& \ ^ ~ << >> >>>	5 & 3 = 1
Assignment	= += -= *= /= %=	x += 5
Unary	++ -- + - !	i++
Ternary	cond ? a : b	max = a > b ? a : b
Type check	instanceof	obj instanceof String

Short-circuit operators

&& and \|\| are **short-circuit**: if the result is known from the left side, the right side isn't evaluated.

```
String s = null;
if (s != null && s.length() > 0) { } // safe: second part never runs
if (s != null & s.length() > 0) { } // NPE: & evaluates both sides
```

Bitwise quick reference

```
5 & 3 // 0101 & 0011 = 0001 = 1
5 | 3 // 0111 = 7
5 ^ 3 // 0110 = 6 (XOR: same bits → 0)
~5 // -6
8 << 1 // 16 (left shift = multiply by 2)
8 >> 1 // 4 (right shift = divide by 2, keeps sign)
-8 >>> 1 // large positive (unsigned shift, fills 0)
```

Interview use: n & 1 checks odd/even; n & (n-1) == 0 checks power of 2; XOR swaps without temp.

Increment / Decrement **MUST-KNOW**

Interview answer: "Pre-increment ++i increases the value first and then uses it. Post-increment i++ uses the current value first and then increases it."

```
int i = 5;
int a = i++; // a = 5, i = 6
int b = ++i; // i = 7, b = 7

int x = 5;
x = x++; // x is still 5! (old value assigned back)
System.out.println(x++ + ++x); // with x=5: 5 + 7 = 12
```

Ternary operator

```
String result = (marks >= 40) ? "Pass" : "Fail";
```

Operator precedence (the ones that trip people up)

Unary (++ -- !) → * / % → + - → shifts → relational → == != → & → ^ → | → && → \|\| → ?: → assignment.

```
System.out.println(1 + 2 + "3"); // "33" (left to right: 3 then "3")
System.out.println("1" + 2 + 3); // "123"
System.out.println(10 / 3); // 3 (int division)
System.out.println(10 / 3.0); // 3.3333333333333335
System.out.println(-7 % 3); // -1 (sign follows dividend)
```

2.8 == VS .equals() **MUST-KNOW**

Interview answer: “== compares references for objects — it checks whether two variables point to the same object in memory. For primitives it compares values. equals() compares the actual content of objects. In the Object class, equals() also compares references, but classes like String and Integer override it to compare content.”

```
String a = new String("java");
String b = new String("java");
System.out.println(a == b);    // false: different objects
System.out.println(a.equals(b)); // true: same content
```

	<code>==</code>	<code>equals()</code>
Type	Operator	Method
Primitives	Compares values	Not applicable
Objects	Compares references (addresses)	Compares content (if overridden)
Can be overridden?	No	Yes
null safety	<code>null == null</code> is fine	<code>null.equals(x)</code> → NPE

Tip: Use `"java".equals(input)` or `Objects.equals(a, b)` to avoid NPE.

3 Control Statements

Interview answer: “Control statements decide the flow of execution in a program. They’re of three types: decision-making statements like if, if-else and switch; looping statements like for, while and do-while; and jump statements like break, continue and return.”

3.1 Decision-Making: if, if-else, else-if, nested if MUST-KNOW

```
int marks = 72;
if (marks >= 90) {
    System.out.println("A");
} else if (marks >= 75) {
    System.out.println("B");
} else if (marks >= 40) {
    System.out.println("C");    // prints C
} else {
    System.out.println("Fail");
}

// Nested if
if (age >= 18) {
    if (hasLicense) System.out.println("Can drive");
}
```

Common mistake: `if (x = 5)` — compile error in Java (needs boolean), unlike C.

3.2 switch MUST-KNOW

Interview answer: “switch selects one block to execute from many options based on a value. It works with byte, short, char, int, their wrappers, enums and String (from Java 7). Without break, execution falls through to the next case. From Java 14, we also have switch expressions with the arrow syntax that don’t fall through and can return a value.”

```
// Classic switch
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Other");
}

// Modern switch expression (Java 14+)
String type = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> "Invalid";
};
```

Follow-ups:

- Can switch use long, float, double, boolean? — No.
- What happens without break? — Fall-through: all following cases run until a break.
- Can we have duplicate case labels? — No, compile error.

3.3 Loops: for, while, do-while MUST-KNOW

```
for (int i = 0; i < 5; i++) { }    // known number of iterations

for (String name : names) { }    // enhanced for (for-each)

while (n > 0) { n /= 10; }        // condition checked first

do { choice = read(); } while (choice != 0); // runs at least once
```

	for	while	do-while
Condition checked	Before	Before	After
Minimum runs	0	0	1
Best for	Known count	Unknown count	Menu-driven / run-at-least-once

Interview answer (while vs do-while): “while checks the condition first, so the body may not execute at all. do-while checks the condition after executing the body, so it always runs at least once — useful for menu-driven programs.”

3.4 Jump Statements: break, continue, return MUST-KNOW

Statement	What it does
break	Exits the loop or switch immediately
continue	Skips the rest of the current iteration, goes to next
return	Exits the method (optionally with a value)

```
for (int i = 1; i <= 10; i++) {
    if (i == 3) continue; // skip 3
    if (i == 6) break;    // stop at 6
    System.out.print(i + " "); // 1 2 4 5
}

// Labeled break - exits the outer loop
outer:
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        if (i * j == 2) break outer;
```

3.5 Common Control-Flow Coding Questions

These appear in fresher online tests and the first round. Full solutions for most are in the **50 Coding Problems** section.

1. Check even or odd — `n % 2 == 0`
2. Largest of three numbers — nested if or `Math.max(a, Math.max(b, c))`
3. Leap year — `(y % 4 == 0 && y % 100 != 0) || y % 400 == 0`
4. Grade calculator using else-if
5. Simple calculator using switch
6. Multiplication table using for
7. Sum of first N natural numbers — loop or `n*(n+1)/2`
8. Pattern printing (see below)

Pattern printing (very common in service-company tests)

```
// Right triangle      // Pyramid
// *                   // *
// * *                 // * *
// * * *               // * * *
// * * * *             // * * * *
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) System.out.print(" ");
    System.out.println();
}

for (int i = 1; i <= n; i++) {
    System.out.print(" ".repeat(n - i));
    System.out.println("***.repeat(2 * i - 1));
}
```

Say while coding: “The outer loop controls rows, the inner loop controls what’s printed in each row. For row i, I print i stars.”

4 OOPs Concepts

Interview answer — “What is OOP?”: “Object-Oriented Programming is a way of designing programs around objects, where each object combines data and the methods that work on that data. It has four main pillars — encapsulation, inheritance, polymorphism and abstraction. OOP makes code reusable, easier to maintain and closer to how we model real-world things.”

Tip: When asked “explain OOPs”, name the four pillars with one line each and give *one* running example (e.g., a `BankAccount` or `Vehicle`). Interviewers love a consistent example.

4.1 Class MUST-KNOW

Interview answer: “A class is a blueprint or template for creating objects. It defines the properties, which are variables, and behaviours, which are methods, that its objects will have. A class itself doesn’t occupy memory for data — memory is allocated when we create objects from it.”

```
class Car {  
    String brand;    // property (state)  
    int speed;  
  
    void accelerate() { // behaviour  
        speed += 10;  
    }  
}
```

Real-world: “Car” as a design drawing — it describes what every car has, but it isn’t a car you can drive.

Follow-up: *What can a class contain?* — Fields, methods, constructors, blocks (static and instance), nested classes.

4.2 Object MUST-KNOW

Interview answer: “An object is an instance of a class — a real entity created from the blueprint. It has state, which is the values of its fields, behaviour, which is its methods, and identity, which is its unique memory address. Objects are created with the `new` keyword and stored in heap memory.”

```
Car c1 = new Car();    // c1 is a reference on the stack; the object is on the heap  
c1.brand = "Honda";  
c1.accelerate();
```

Class	Object
Blueprint / template	Instance of the class
Logical entity	Physical entity
Declared once	Can create many
No memory for fields at declaration	Memory allocated in heap on creation
Declared with <code>class</code> keyword	Created with <code>new</code> (mostly)

Follow-up: *Ways to create an object?* — `new`, reflection (`Class.forName(...).getDeclaredConstructor().newInstance()`), `clone()`, deserialization, factory methods.

4.3 Constructor MUST-KNOW

Interview answer: “A constructor is a special block of code that runs automatically when an object is created, and it’s used to initialise the object. It has the same name as the class and no return type — not even `void`.”

Rules

- Same name as class, no return type.
- Called automatically by `new`.
- Can be overloaded; **cannot** be `static`, `final`, `abstract` or `synchronized`.
- Not inherited, so cannot be overridden.
- If you write no constructor, the compiler adds a **default no-arg constructor**.

Default constructor

Interview answer: “If we don’t write any constructor, the Java compiler automatically adds a no-argument default constructor that initialises fields to their default values. But once we write any constructor ourselves, the compiler no longer adds it.”

Parameterized constructor

```
class Student {
    String name;
    int age;

    Student() {                // no-arg
        this("Unknown", 0);    // constructor chaining
    }
    Student(String name, int age) { // parameterized
        this.name = name;
        this.age = age;
    }
}
```

Constructor overloading

Interview answer: “Constructor overloading means having multiple constructors in the same class with different parameter lists. It lets us create objects in different ways — for example, with default values or with specific values.”

Classic trap:

```
class A { A(int x) { } }
A a = new A(); // COMPILER ERROR – no default constructor, because we wrote one
```

Constructor vs Method

Constructor	Method
Initialises an object	Performs an operation
Same name as class	Any name
No return type	Must have a return type (or void)
Called automatically by <code>new</code>	Called explicitly
Not inherited	Inherited
Compiler provides default	No default method

Follow-ups: Can a constructor be private? (Yes — used in Singleton.) Can a constructor return a value? (No, but it can have `return;` to exit early.) What is constructor chaining? (Calling one constructor from another using `this()` in the same class or `super()` in the parent — must be the first statement.)

4.4 `this` keyword **MUST-KNOW**

Interview answer: “`this` is a reference to the current object. We use it mainly to differentiate instance variables from parameters with the same name, to call another constructor of the same class using `this()`, and to pass the current object as an argument.”

Uses:

1. `this.name = name;` — resolve name clash.
2. `this(...)` — call another constructor (first line only).
3. `return this;` — method chaining (builder style).
4. Pass current object: `service.register(this);`

Follow-up: Can we use `this` in a static method? — No, static methods belong to the class, and there is no current object.

4.5 `super` keyword MUST-KNOW

Interview answer: “`super` is a reference to the immediate parent class object. We use it to call the parent’s constructor with `super()`, to call a parent method that has been overridden, and to access a parent field hidden by a child field.”

```
class Animal {
    String name = "Animal";
    Animal() { System.out.println("Animal created"); }
    void sound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    String name = "Dog";
    Dog() {
        super(); // implicit even if not written
        System.out.println("Dog created");
    }
    void sound() {
        super.sound(); // call parent version
        System.out.println("Bark " + super.name);
    }
}
```

Key fact: Every constructor’s first line is implicitly `super()` unless you write `this(...)` or `super(...)` yourself. That’s why parent constructors always run first.

Follow-up: Can we use both `this()` and `super()` in the same constructor? — No, both must be the first statement.

4.6 Encapsulation MUST-KNOW

Interview answer: “Encapsulation is bundling data and the methods that operate on that data into a single class, and restricting direct access to the data. In Java we achieve it by making fields private and providing public getters and setters. This protects the data — we can add validation in setters, so the object can never go into an invalid state.”

```
public class BankAccount {
    private double balance; // hidden

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("Invalid amount");
        balance += amount; // controlled access
    }
}
```

Simple explanation: Nobody outside the class can do `account.balance = -5000`. They must go through `deposit()`, which checks the rules.

Real-world: An ATM — you can’t open the cash box; you interact through buttons (methods) that enforce rules.

Follow-ups:

- *Benefits?* — Data hiding, validation, flexibility to change internals without breaking callers, easier testing.
- *Encapsulation vs abstraction?* — Encapsulation *hides data* (how it’s protected); abstraction *hides complexity* (what it does vs how).
- *Is a class with private fields and public getters/setters for everything fully encapsulated?* — Technically yes, but good encapsulation only exposes what’s needed.

4.7 Inheritance MUST-KNOW

Interview answer: “Inheritance is the mechanism where one class acquires the properties and methods of another class using the extends keyword. The class being inherited from is the parent or superclass, and the one inheriting is the child or subclass. It promotes code reusability and represents an IS-A relationship — for example, a Dog IS-A Animal.”

```
class Vehicle {  
    void start() { System.out.println("Starting"); }  
}  
class Car extends Vehicle {  
    void openSunroof() { System.out.println("Sunroof open"); }  
}  
Car c = new Car();  
c.start();           // inherited  
c.openSunroof();     // own method
```

Types of inheritance

Type	Example	Supported in Java?
Single	B extends A	Yes
Multilevel	C extends B, B extends A	Yes
Hierarchical	B and C both extend A	Yes
Multiple	C extends A, B	No (with classes); Yes via interfaces
Hybrid	Mix of the above	Only via interfaces

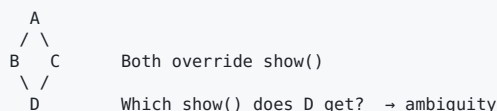
What is NOT inherited: constructors, private members (they exist in the object but aren't accessible), static members aren't *overridden* (they're hidden).

Follow-ups: What is the parent of all classes? (`java.lang.Object` .) Can a class extend itself? (No — cyclic inheritance error.) How to prevent inheritance? (Make the class `final`.)

4.8 Multiple Inheritance & the Diamond Problem MUST-KNOW

Interview answer: “Java doesn't support multiple inheritance through classes to avoid ambiguity — the diamond problem. If class C extended both A and B, and both had a method `show()`, the compiler wouldn't know which `show()` C should inherit. So Java allows a class to extend only one class, but it can implement multiple interfaces. If two interfaces have the same default method, Java forces the class to override it and resolve the conflict explicitly.”

```
interface A { default void show() { System.out.println("A"); } }  
interface B { default void show() { System.out.println("B"); } }  
  
class C implements A, B {  
    @Override  
    public void show() {  
        A.super.show(); // explicitly choose  
    }  
}
```



4.9 Polymorphism MUST-KNOW

Interview answer: “Polymorphism means ‘one name, many forms’ — the same method or reference behaves differently depending on the object. Java has two types: compile-time polymorphism, achieved through method overloading, and runtime polymorphism, achieved through method overriding, where the method call is resolved at runtime based on the actual object.”

Real-world: A person is a son at home, an employee at work, a customer at a shop — same person, different behaviour by context.

	Compile-time (static)	Runtime (dynamic)
Achieved by	Method overloading	Method overriding
Resolved	At compile time	At runtime by JVM
Binding	Static/early binding	Dynamic/late binding
Inheritance needed?	No	Yes
Speed	Slightly faster	Slightly slower

4.10 Method Overloading MUST-KNOW

Interview answer: “Method overloading means having multiple methods with the same name in the same class but with different parameter lists — different number, type or order of parameters. The compiler decides which method to call at compile time based on the arguments, so it’s compile-time polymorphism.”

```
class Calculator {  
    int add(int a, int b) { return a + b; }  
    int add(int a, int b, int c) { return a + b + c; }  
    double add(double a, double b) { return a + b; }  
}
```

Rules: Must change parameter list. Return type alone is **not** enough. Access modifier and exceptions can be anything.

Trick question:

```
int add(int a, int b) { ... }  
long add(int a, int b) { ... } // COMPILER ERROR – only return type differs
```

“We can’t overload by return type alone because when calling `add(2, 3)`, the compiler can’t tell which one we mean — the return value might not even be used.”

Type promotion in overloading:

```
void show(long x) { System.out.println("long"); }  
void show(Integer x) { System.out.println("Integer"); }  
show(5); // prints "long" – widening is preferred over autoboxing
```

Order the compiler tries: exact match → widening → autoboxing → varargs.

4.11 Method Overriding MUST-KNOW

Interview answer: “Method overriding is when a subclass provides its own implementation of a method that’s already defined in its parent class, with the same name, same parameters and a compatible return type. The method that runs is decided at runtime based on the actual object, so it’s runtime polymorphism.”

```
class Payment {  
    void pay(double amt) { System.out.println("Generic payment"); }  
}  
class UpiPayment extends Payment {  
    @Override  
    void pay(double amt) { System.out.println("Paid " + amt + " via UPI"); }  
}
```

Overriding rules (very frequently asked)

1. Same method name and same parameter list.
2. Return type: same, or a **subtype** (covariant return type).
3. Access modifier: **same or wider** (protected → public OK; public → private **X**).
4. Checked exceptions: **same, narrower or none** — cannot throw new/broader checked exceptions.
5. Cannot override `final`, `static` or `private` methods.
6. Constructors cannot be overridden.
7. Use `@Override` — the compiler then catches mistakes like a typo in the method name.

Overloading vs Overriding

	Overloading	Overriding
Where	Same class (or subclass)	Parent-child classes
Parameters	Must differ	Must be same
Return type	Can differ	Same or covariant
Access modifier	Any	Same or less restrictive
Exceptions	Any	Same, narrower or none (checked)
Polymorphism	Compile-time	Runtime
static/private/final methods	Can be overloaded	Cannot be overridden

4.12 Dynamic Method Dispatch (Runtime Polymorphism) MUST-KNOW

Interview answer: “Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at runtime rather than compile time. When a parent class reference points to a child class object, the JVM looks at the actual object type and calls the child’s version of the method.”

```
Payment p = new UpiPayment(); // parent reference, child object (upcasting)
p.pay(500); // "Paid 500.0 via UPI" - decided at runtime
```

Key rule to say: “Methods are resolved by **object type** at runtime; variables and static methods are resolved by **reference type** at compile time.”

```
class A { int x = 10; static void s() { System.out.println("A static"); } void m() { System.out.println("A"); } }
class B extends A { int x = 20; static void s() { System.out.println("B static"); } void m() { System.out.println("B"); } }

A obj = new B();
obj.m(); // B (overridden → runtime)
System.out.println(obj.x); // 10 (fields are not polymorphic)
obj.s(); // A static (static → method hiding, reference type)
```

Follow-up: *Upcasting vs downcasting?* — Upcasting (child → parent reference) is automatic and safe. Downcasting (parent → child) needs an explicit cast and can throw `ClassCastException`; check with `instanceof` first.

4.13 Abstraction MUST-KNOW

Interview answer: “Abstraction means hiding the implementation details and showing only the essential features to the user. The user knows *what* an object does, not *how* it does it. In Java we achieve abstraction using abstract classes and interfaces.”

Real-world: When you drive a car you use the steering, brake and accelerator. You don’t need to know how the engine or ABS works internally.

Code-level example: You call `list.add(x)` without knowing how `ArrayList` resizes its internal array.

4.14 Abstract Class MUST-KNOW

Interview answer: “An abstract class is a class declared with the abstract keyword that cannot be instantiated. It can have both abstract methods — without a body — and concrete methods with a body. It’s used when related classes share some common code but each must also provide its own implementation for certain methods.”

```
abstract class Shape {
    String color; // can have state
    Shape(String color) { this.color = color; } // can have constructor
    abstract double area(); // must be implemented by child
    void describe() { // concrete method
        System.out.println(color + " shape with area " + area());
    }
}

class Circle extends Shape {
    double r;
    Circle(String c, double r) { super(c); this.r = r; }
    double area() { return Math.PI * r * r; }
}
```

Follow-ups:

- *Can an abstract class have a constructor?* — Yes. It runs when a subclass object is created (via `super()`), to initialise the common fields.
- *Can it have zero abstract methods?* — Yes. It still can’t be instantiated.
- *Can an abstract method be private/static/final?* — No. It must be overridden, and those modifiers prevent overriding.

4.15 Interface MUST-KNOW

Interview answer: “An interface is a contract that defines what a class must do, but not how. It contains abstract methods, and from Java 8 it can also have default and static methods, and from Java 9 private methods. Variables in an interface are public static final by default. A class implements an interface with the implements keyword, and a class can implement multiple interfaces — that’s how Java supports multiple inheritance.”

```
interface PaymentGateway {
    int MAX_RETRIES = 3; // public static final
    void pay(double amount); // public abstract
    default void log(String m) { // Java 8
        System.out.println("[LOG] " + m);
    }
    static boolean isValid(double a) { // Java 8
        return a > 0;
    }
}

class RazorpayGateway implements PaymentGateway {
    public void pay(double amount) { log("Paid " + amount); }
}
```

Real-world: A power socket is an interface. Any device with the right plug (implementation) works with it.

Follow-ups:

- *Why are default methods added?* — “To add new methods to existing interfaces without breaking all the classes that already implement them. For example, `forEach` was added to `Iterable` in Java 8.”
- *Can an interface have a constructor?* — No, it has no instance state to initialise.
- *Marker interface?* — An interface with no methods, like `Serializable` or `Cloneable`, that signals something to the JVM/framework.
- *Functional interface?* — An interface with exactly one abstract method (see Java 8 section).

4.16 Abstract Class vs Interface MUST-KNOW

	Abstract class	Interface
Keyword	<code>abstract class</code> , <code>extends</code>	<code>interface</code> , <code>implements</code>
Methods	Abstract + concrete	Abstract + default + static (+ private, Java 9)
Variables	Any type (instance, static, final)	Only <code>public static final</code> constants
Constructor	Yes	No
Multiple inheritance	A class can extend only one	A class can implement many
Access modifiers on methods	Any	Public by default (private allowed for helpers)
State	Can hold object state	No instance state
When to use	Related classes sharing code ("is-a" with common base)	Capability/contract across unrelated classes ("can-do")

Interview answer: "An abstract class can have state, constructors and both abstract and concrete methods, but a class can extend only one. An interface is a pure contract — it has no instance state or constructor, only constants, abstract methods and default or static methods — and a class can implement many interfaces. I use an abstract class when related classes share common code, like a base Employee class, and an interface when I want to define a capability that unrelated classes can have, like Comparable or Runnable."

4.17 IS-A vs HAS-A MUST-KNOW

Interview answer: "IS-A is inheritance — a Car IS-A Vehicle, implemented with extends or implements. HAS-A is when one class contains a reference to another — a Car HAS-A Engine. HAS-A is called association, and it has two forms: aggregation and composition."

```
class Engine { }
class Vehicle { }
class Car extends Vehicle { // IS-A
    private Engine engine = new Engine(); // HAS-A
}
```

4.18 Association, Aggregation, Composition IMPORTANT

	Association	Aggregation	Composition
Meaning	Any relationship between two objects	Weak HAS-A ("whole-part", part can exist alone)	Strong HAS-A (part cannot exist without whole)
Lifecycle	Independent	Independent	Part dies with whole
Example	Teacher ↔ Student	Department has Teachers	House has Rooms; Human has Heart
Code hint	Reference passed around	Part passed in from outside	Part created inside the whole

```
// Aggregation: Department doesn't create Teachers; they exist independently
class Department {
    private List<Teacher> teachers;
    Department(List<Teacher> teachers) { this.teachers = teachers; }
}

// Composition: Room objects are created and owned by House
class House {
    private final List<Room> rooms = new ArrayList<>();
    House() { rooms.add(new Room("Hall")); }
}
```

Interview answer: "Association is a general relationship between two classes. Aggregation is a weak form of association where the child can exist independently of the parent — like a Department and its Teachers; if the department closes, teachers still exist. Composition is a strong form where the child can't exist without the parent — like a House and its Rooms; destroy the house and the rooms are gone."

4.19 Access Modifiers MUST-KNOW

Interview answer: “Access modifiers control the visibility of classes, methods and variables. Java has four: private — only inside the same class; default, with no keyword — only inside the same package; protected — same package plus subclasses in other packages; and public — accessible from everywhere.”

Modifier	Same class	Same package	Subclass (other package)	Other package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓ (through inheritance)	✗
public	✓	✓	✓	✓

Follow-ups:

- *Can a top-level class be private or protected?* — No. Only `public` or default. Nested classes can be private.
- *Most restrictive to least:* private → default → protected → public.

5 Important Java Keywords

5.1 static **MUST-KNOW**

Interview answer: “static means the member belongs to the class rather than to any object. A static variable has only one copy shared by all objects, a static method can be called using the class name without creating an object, and a static block runs once when the class is loaded. We use static for things that are common to all objects, like a counter or utility methods.”

Static variable

```
class Employee {
    static int count = 0;    // one copy for all employees
    String name;
    Employee(String n) { name = n; count++; }
}
new Employee("A"); new Employee("B");
System.out.println(Employee.count); // 2
```

Memory: Static variables live with the class (in the Metaspace/class area, the value itself in heap since Java 8), created once when the class loads.

Static method

```
class MathUtil {
    static int square(int x) { return x * x; }
}
MathUtil.square(5);    // no object needed
```

Rules: A static method can directly access only static members. It cannot use `this` or `super`. It can't be overridden (only hidden).

Static block

Interview answer: “A static block is used to initialise static variables. It runs exactly once, when the class is loaded into memory, even before the main method.”

```
class Config {
    static Map<String, String> props;
    static {
        props = new HashMap<>();
        props.put("env", "prod");
        System.out.println("Static block runs first");
    }
}
```

Execution order question:

```
class Demo {
    static { System.out.println("1 static block"); }
    { System.out.println("3 instance block"); }
    Demo() { System.out.println("4 constructor"); }
    public static void main(String[] a) {
        System.out.println("2 main");
        new Demo();
    }
}
// Output: 1 static block, 2 main, 3 instance block, 4 constructor
```

Static vs Instance

	Static	Instance
Belongs to	Class	Object
Copies	One per class	One per object
Accessed via	ClassName.member	object.member
Memory allocated	When class loads	When object is created
Can access instance members directly?	No	Yes
Can use this ?	No	Yes

Follow-ups: Can we have a static class? (Only a static *nested* class.) Can static methods be overloaded? (Yes.) Overridden? (No — hidden.)

5.2 `final` **MUST-KNOW**

Interview answer: “final is used to restrict changes. A final variable’s value can’t be changed once assigned — it becomes a constant. A final method can’t be overridden by subclasses. A final class can’t be extended — String is a good example of a final class.”

```
final int MAX = 10;           // MAX = 20; → error

class Parent {
    final void rules() { } // child cannot override
}

final class Utility { }      // class X extends Utility → error
```

Important subtlety:

```
final List<String> list = new ArrayList<>();
list.add("A");           // ALLOWED – object content can change
list = new ArrayList<>(); // ERROR – reference cannot change
```

“final makes the reference constant, not the object immutable.”

Blank final variable: declared without a value, must be assigned exactly once — in the constructor (instance) or static block (static).

5.3 `finally` and `finalize` — see the comparison below

5.4 `final` vs `finally` vs `finalize` **MUST-KNOW**

	final	finally	finalize()
What	Keyword	Block	Method of Object class
Used with	Variable, method, class	try-catch	Garbage collection
Purpose	Restrict change / override / inheritance	Always execute cleanup code	Cleanup before GC reclaims object
Status	Common	Common	Deprecated since Java 9 , marked for removal

Interview answer: “They’re completely unrelated despite similar names. final is a keyword that makes a variable constant, a method non-overridable, or a class non-inheritable. finally is a block used with try-catch that always executes, whether or not an exception occurs — we use it to close resources. finalize is a method the garbage collector used to call before destroying an object, but it’s deprecated now because it’s unpredictable — we use try-with-resources instead.”

5.5 `new` **IMPORTANT**

Interview answer: “new is used to create an object. It allocates memory for the object in the heap, calls the constructor to initialise it, and returns a reference to that object.”

5.6 instanceof **IMPORTANT**

Interview answer: “instanceof checks whether an object is an instance of a particular class or interface, and returns true or false. We use it before downcasting to avoid ClassCastException. From Java 16, we can use pattern matching to check and cast in one step.”

```
Object o = "hello";
if (o instanceof String s) {           // Java 16+ pattern matching
    System.out.println(s.length());
}
null instanceof String                 // false (never throws)
```

5.7 abstract **IMPORTANT**

“abstract on a class means it can’t be instantiated; on a method it means no body — subclasses must implement it. abstract can’t be combined with final, static or private.”

5.8 synchronized **MUST-KNOW**

Interview answer: “synchronized ensures that only one thread at a time can execute a block or method on the same object’s lock. It’s used to prevent race conditions when multiple threads access shared data.”

(Details in the Multithreading section.)

5.9 volatile **IMPORTANT**

Interview answer: “volatile tells the JVM that a variable may be changed by multiple threads, so every read goes to main memory rather than a thread’s cached copy. It guarantees visibility of changes across threads, but not atomicity — so count++ on a volatile variable is still not thread-safe.”

5.10 transient **IMPORTANT**

Interview answer: “transient marks a field that should not be serialized. When the object is saved or sent over the network, transient fields are skipped and get their default values on deserialization. We use it for sensitive data like passwords or for fields that can be recalculated.”

```
class User implements Serializable {
    String username;
    transient String password; // not saved
}
```

5.11 native **OPTIONAL**

“native marks a method whose implementation is written in another language like C/C++ and called through JNI. Example: `Object.hashCode()` and `Thread.start0()` are native.”

5.12 throw VS throws **MUST-KNOW**

	throw	throws
Purpose	Actually throws an exception	Declares that a method might throw
Used	Inside method body	In method signature
Followed by	An exception object	Exception class names
Count	One exception at a time	Multiple, comma separated
Example	<code>throw new IllegalArgumentException("bad");</code>	<code>void read() throws IOException, SQLException</code>

Interview answer: “throw is used inside a method to actually throw an exception object. throws is used in the method signature to declare that the method may throw certain checked exceptions, so the caller must handle them.”

6 Strings

6.1 What is a String? MUST-KNOW

Interview answer: “A String in Java is an object that represents a sequence of characters. It belongs to the `java.lang` package, it’s a final class, and it’s immutable — once a String object is created, its value can’t be changed. Any operation that seems to modify it actually creates a new String.”

Internal storage: Before Java 9, a String was backed by a `char[]`. From Java 9, it uses a `byte[]` plus a coder flag (**Compact Strings**) — Latin-1 characters take 1 byte instead of 2, saving memory.

6.2 String Pool MUST-KNOW

Interview answer: “The String pool, or String constant pool, is a special area inside the heap where Java stores string literals. When we create a string literal, the JVM first checks the pool — if the same value already exists, it returns the existing reference instead of creating a new object. This saves memory because identical strings are shared.”

```
String s1 = "java";           // goes into pool
String s2 = "java";           // reuses the pooled object
String s3 = new String("java"); // new object in heap (outside pool)
String s4 = s3.intern();       // returns pooled reference

s1 == s2 // true
s1 == s3 // false
s1 == s4 // true
```

Heap

- String Pool: "java" ← s1, s2, s4
- Normal heap: String("java") ← s3

6.3 Literal vs `new String()` MUST-KNOW

Interview answer: “A string literal is stored in the String pool and reused if the same value exists. `new String()` always creates a new object in the heap, even if the same value is already in the pool. So `String s = new String("abc")` can create up to two objects — one in the pool for the literal “abc” if it isn’t already there, and one in the heap.”

Classic question: *How many objects does `new String("hello")` create?* — “Up to two: one in the pool for the literal (if not already present) and one in the normal heap. If “hello” is already in the pool, only one.”

6.4 Why is String Immutable? MUST-KNOW

Interview answer: “String is immutable for four main reasons. First, the String pool — since many references share the same literal, if one could change it, it would affect all others. Second, security — strings hold things like database URLs, usernames and file paths, and they can’t be altered after validation. Third, thread safety — immutable objects can be shared across threads without synchronization. And fourth, hashCode caching — since the value never changes, its hash code is computed once and cached, which makes String a fast and reliable HashMap key.”

```
String s = "Hello";
s.concat(" World"); // creates a new string, result discarded
System.out.println(s); // Hello
s = s.concat(" World"); // reassign reference to the new object
System.out.println(s); // Hello World
```

Follow-up: *How is String made immutable?* — The class is `final` (can't be subclassed), the internal array is `private final`, there are no setters, and methods that “modify” return new objects.

6.5 String vs StringBuilder vs StringBuffer MUST-KNOW

	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread safe	Yes (immutable)	No	Yes (synchronized methods)
Performance	Slow for repeated changes	Fastest	Slower than StringBuilder
Introduced	Java 1.0	Java 1.5	Java 1.0
Storage	String pool / heap	Heap	Heap
Use when	Value doesn't change	Many changes, single thread	Many changes, multi-thread

Interview answer: “String is immutable, so every modification creates a new object. StringBuilder and StringBuffer are mutable — they modify the same object. The difference between those two is that StringBuffer is synchronized and thread-safe but slower, while StringBuilder isn't synchronized, so it's faster. In practice, I use StringBuilder for building strings in loops, since most string building happens within a single thread.”

```
// Bad: creates many intermediate String objects
String s = "";
for (int i = 0; i < 1000; i++) s += i;

// Good
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i);
String result = sb.toString();
```

StringBuilder useful methods: `append()`, `insert(i, x)`, `delete(start, end)`, `deleteCharAt(i)`, `reverse()`, `setCharAt(i, c)`, `length()`, `capacity()` (default 16, grows as `(old*2)+2`).

6.6 String Methods (know these by heart) MUST-KNOW

Method	Example	Result
<code>length()</code>	<code>"java".length()</code>	4
<code>charAt(i)</code>	<code>"java".charAt(1)</code>	'a'
<code>substring(b, e)</code>	<code>"hello".substring(1, 3)</code>	"el" (end exclusive)
<code>indexOf(x)</code> / <code>lastIndexOf(x)</code>	<code>"hello".indexOf('l')</code>	2
<code>contains(x)</code>	<code>"hello".contains("ell")</code>	true
<code>equals</code> / <code>equalsIgnoreCase</code>	<code>"A".equalsIgnoreCase("a")</code>	true
<code>compareTo(x)</code>	<code>"apple".compareTo("banana")</code>	negative
<code>toUpperCase()</code> / <code>toLowerCase()</code>	<code>"Hi".toUpperCase()</code>	"HI"
<code>trim()</code> / <code>strip()</code>	<code>" hi ".trim()</code>	"hi"
<code>replace(a, b)</code>	<code>"java".replace('a', 'o')</code>	"jovo"
<code>split(regex)</code>	<code>"a,b,c".split(",")</code>	["a","b","c"]
<code>toArray()</code>	<code>"abc".toArray()</code>	['a','b','c']
<code>startsWith</code> / <code>endsWith</code>	<code>"file.txt".endsWith(".txt")</code>	true
<code>isEmpty()</code> / <code>isBlank()</code> (11)	<code>" ".isBlank()</code>	true
<code>String.join(d, ...)</code>	<code>String.join("-", "a", "b")</code>	"a-b"
<code>valueOf(x)</code>	<code>String.valueOf(10)</code>	"10"
<code>repeat(n)</code> (11)	<code>"ab".repeat(3)</code>	"ababab"
<code>chars()</code>	<code>"abc".chars()</code>	IntStream

6.7 String Concatenation IMPORTANT

Interview answer: “We can concatenate strings using the + operator, concat(), StringBuilder.append() or String.join(). Compile-time constants joined with + are combined by the compiler into a single literal. At runtime, + creates new String objects, so inside loops StringBuilder is the right choice.”

```
String a = "Hel" + "lo";    // compile-time constant → pooled
String b = "Hello";
a == b                      // true

String h = "Hel";
String c = h + "lo";        // runtime → new object
c == b                      // false

final String f = "Hel";
String d = f + "lo";        // f is a compile-time constant
d == b                      // true
```

6.8 == VS equals() VS compareTo() for Strings

```
String a = "abc", b = new String("abc");
a == b                // false (reference)
a.equals(b)           // true  (content)
a.compareTo(b)        // 0     (lexicographic: 0 equal, <0 before, >0 after)
```

6.9 Frequently Asked String Coding Questions

Full solutions are in the **50 Coding Problems** section (Problems 11–20). Quick patterns:

Problem	One-line approach
Reverse a string	Two pointers on char array, or <code>StringBuilder.reverse()</code>
Palindrome	Compare <code>i</code> and <code>n-1-i</code> until they meet
Count vowels/consonants	Loop, check <code>"aeiou".indexOf(ch)</code>
Character frequency	<code>int[26]</code> or <code>HashMap<Character, Integer></code>
First non-repeating char	Count frequency, then scan again in order
Anagram	Sort both / count array of 26
Remove duplicates	<code>LinkedHashSet</code> or <code>boolean[256]</code> seen array
Reverse words	<code>split("\\s+")</code> , iterate backwards
String compression	Count consecutive run lengths (<code>"aaabb" → "a3b2"</code>)
Check rotation	<code>(s1 + s1).contains(s2)</code> with equal lengths

7 Arrays

7.1 What is an Array? MUST-KNOW

Interview answer: “An array is a fixed-size container that stores multiple values of the same data type in contiguous memory locations. Each element is accessed using an index starting from 0. In Java, arrays are objects, so they’re stored in the heap and have a length property.”

7.2 One-Dimensional Arrays

```
int[] a;           // declaration (preferred style)
int b[];           // also valid
a = new int[5];    // allocation → {0,0,0,0,0}
int[] c = {10, 20, 30}; // declaration + initialization
int[] d = new int[]{1, 2, 3};

for (int i = 0; i < c.length; i++) System.out.println(c[i]); // index loop
for (int x : c) System.out.println(x);                       // for-each
System.out.println(Arrays.toString(c));                      // [10, 20, 30]
```

Default values: 0 for numeric, false for boolean, '\u0000' for char, null for objects.

7.3 Two-Dimensional Arrays

```
int[][] m = new int[3][4]; // 3 rows, 4 columns
int[][] grid = { {1, 2}, {3, 4}, {5, 6} };

for (int i = 0; i < grid.length; i++) {
    for (int j = 0; j < grid[i].length; j++) {
        System.out.print(grid[i][j] + " ");
        System.out.println();
    }
}

int[][] jagged = new int[3][]; // jagged array: rows of different length
jagged[0] = new int[2];
jagged[1] = new int[5];
```

Follow-up: Is a 2D array really a matrix in memory? — “No. In Java, a 2D array is an array of arrays — each row is a separate array object, which is why jagged arrays are possible.”

7.4 Array Limitations IMPORTANT

1. **Fixed size** — can’t grow or shrink after creation.
2. Only **same type** elements.
3. Insertion/deletion in the middle is costly (shift elements) — O(n).
4. No built-in methods like add/remove/contains (use `Arrays` utility class).
5. Accessing invalid index → `ArrayIndexOutOfBoundsException`.

7.5 Arrays vs ArrayList MUST-KNOW

	Array	ArrayList
Size	Fixed	Dynamic (grows automatically)
Types	Primitives and objects	Objects only (uses wrappers)
Length	<code>arr.length</code> (field)	<code>list.size()</code> (method)
Performance	Slightly faster, less memory	Small overhead (boxing, resizing)
Generics	No	Yes (type safe)
Methods	None (use <code>Arrays</code> class)	add, remove, contains, sort...
Multi-dimensional	Yes	No (list of lists)

Interview answer: “An array has a fixed size and can store primitives directly, which makes it slightly faster and more memory-efficient. An ArrayList is dynamic — it grows automatically — but can only store objects, so primitives get autoboxed. I use arrays when the size is known and performance matters, and ArrayList when I need a flexible, resizable collection with built-in methods.”

7.6 Useful `Arrays` class methods

```
Arrays.sort(arr);           // dual-pivot quicksort for primitives
Arrays.binarySearch(arr, key); // array must be sorted
Arrays.fill(arr, -1);
Arrays.copyOf(arr, newLength);
Arrays.copyOfRange(arr, 1, 4);
Arrays.equals(a, b);        // compare content
Arrays.asList(1, 2, 3);     // fixed-size list backed by array
Arrays.stream(arr).sum();
```

Trap: `Arrays.asList()` returns a fixed-size list — `add()` throws `UnsupportedOperationException`. `List.of()` is fully immutable.

Trap: `arr1 == arr2` and `arr1.equals(arr2)` both compare references. Use `Arrays.equals()`.

7.7 Common Array Interview Problems

Full solutions in the **50 Coding Problems** section (Problems 21–35):

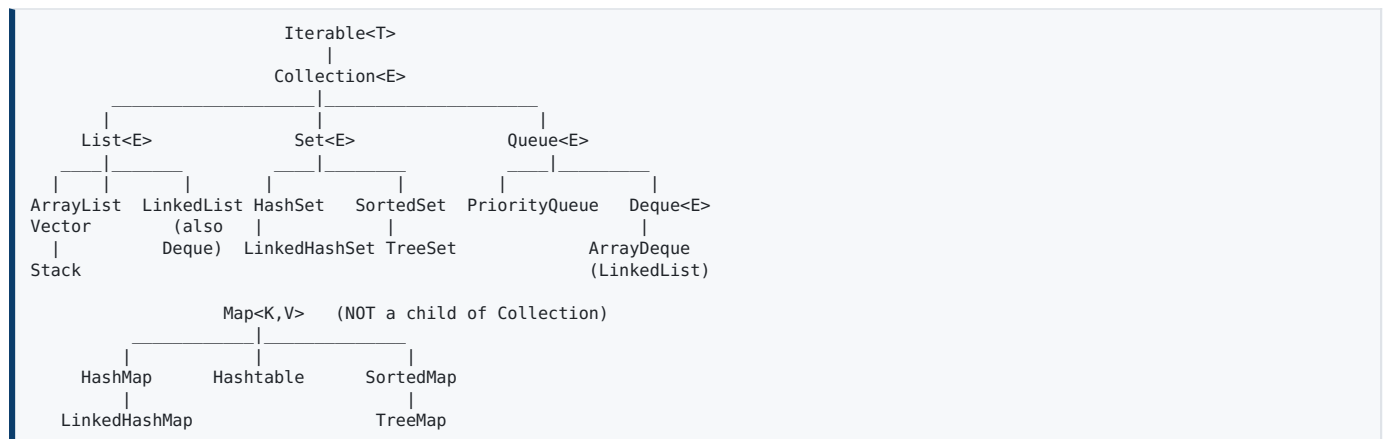
max/min, second largest, reverse, remove duplicates from sorted array, find duplicate, missing number, move zeros, Two Sum, merge sorted arrays, intersection, union, rotate by k, element frequency, Kadane’s maximum subarray.

8 Collection Framework

8.1 What is the Collection Framework? **MUST-KNOW**

Interview answer: “The Collection Framework is a set of interfaces and classes in java.util used to store and manipulate groups of objects. It gives ready-made data structures like lists, sets, queues and maps, along with algorithms for sorting and searching. Before it, we had to manage arrays manually; now we get dynamic size, type safety with generics, and consistent APIs.”

8.2 Collection Hierarchy **MUST-KNOW**



Interface	One-line meaning
Iterable	Root; anything that can be looped with for-each (iterator())
Collection	Group of individual elements; common methods add , remove , size , contains
List	Ordered, index-based, allows duplicates
Set	No duplicates
Queue	Elements processed in a specific order (usually FIFO)
Deque	Double-ended queue — add/remove at both ends
Map	Key-value pairs; keys unique

Interview answer: “At the top is Iterable, then Collection, which has three main sub-interfaces: List for ordered collections that allow duplicates, Set for unique elements, and Queue for processing elements in order. Map is separate — it doesn’t extend Collection because it stores key-value pairs instead of single elements.”

Follow-up: *Why doesn’t Map extend Collection?* — “Collection methods like add(E) work with single elements, but a Map works with key-value pairs, so the contracts don’t fit.”

8.3 List Implementations

ArrayList **MUST-KNOW**

Interview answer: “ArrayList is a resizable array implementation of the List interface. It maintains insertion order, allows duplicates and nulls, and gives fast random access using index. Internally it uses an Object array; when it fills up, it creates a new array about 1.5 times larger and copies the elements over.”

Internal working:

- Backed by `Object[] elementData`.
- `new ArrayList<>()` starts with an empty array; the first `add` allocates capacity **10**.
- When full: `newCapacity = oldCapacity + (oldCapacity >> 1) → 1.5x`, then `Arrays.copyOf`.
- `remove(index)` shifts all later elements left.

Operation	Time
<code>get(i)</code> / <code>set(i)</code>	$O(1)$
<code>add(e)</code> at end	$O(1)$ amortised ($O(n)$ when resizing)
<code>add(i, e)</code> / <code>remove(i)</code> in middle	$O(n)$ — shifting
<code>contains</code> / <code>indexOf</code>	$O(n)$

Use when: Mostly reading/iterating, adding at the end. This is the default List choice.

Disadvantages: Slow insert/delete in the middle; not synchronized.

LinkedList MUST-KNOW

Interview answer: “LinkedList is a doubly linked list implementation of both the List and Deque interfaces. Each element is a node holding the data plus references to the previous and next nodes. Inserting or removing at the ends is $O(1)$, but accessing an element by index is $O(n)$ because it has to walk through the nodes.”

```
private static class Node<E> {
    E item;
    Node<E> next;
    Node<E> prev;
}
```

Operation	Time
<code>addFirst</code> / <code>addLast</code> / <code>removeFirst</code> / <code>removeLast</code>	$O(1)$
<code>get(i)</code>	$O(n)$ (walks from nearer end)
Insert/delete in middle	$O(n)$ to find + $O(1)$ to relink

ArrayList vs LinkedList MUST-KNOW

	ArrayList	LinkedList
Internal structure	Dynamic array	Doubly linked list
Random access <code>get(i)</code>	$O(1)$ — fast	$O(n)$ — slow
Insert/delete at ends	$O(1)$ end, $O(n)$ start	$O(1)$ both ends
Insert/delete middle	$O(n)$ shifting	$O(n)$ to reach, $O(1)$ relink
Memory	Less (just data)	More (data + 2 pointers per node)
Cache friendly	Yes (contiguous)	No
Implements	List, RandomAccess	List, Deque
Best for	Frequent reads	Frequent add/remove at ends, queue use

Interview answer: “ArrayList uses a dynamic array, so accessing by index is $O(1)$, but inserting or removing in the middle requires shifting elements. LinkedList uses a doubly linked list, so adding or removing at the ends is $O(1)$, but getting an element by index is $O(n)$. In practice, ArrayList is the default choice because most apps read more than they insert, and it’s more cache-friendly. I’d use LinkedList mainly when I need queue or deque behaviour — though ArrayDeque is usually even better for that.”

Vector and Stack IMPORTANT

- **Vector:** Legacy, synchronized dynamic array. Grows by **2x**. Slower due to locking.
- **Stack:** Extends Vector; LIFO with `push`, `pop`, `peek`. Legacy — prefer `ArrayDeque` as a stack.

```
Deque<Integer> stack = new ArrayDeque<>();
stack.push(1); stack.push(2);
stack.pop(); // 2
```

ArrayList vs Vector

	ArrayList	Vector
Synchronized	No	Yes (every method)
Performance	Faster	Slower
Growth	1.5×	2×
Legacy	No (Java 1.2)	Yes (Java 1.0)
Iteration	Iterator	Iterator + Enumeration
Thread-safe alternative	Collections.synchronizedList or CopyOnWriteArrayList	—

8.4 Set Implementations

HashSet MUST-KNOW

Interview answer: “HashSet stores unique elements and doesn’t maintain any order. Internally, it’s backed by a HashMap — every element is stored as a key in that map with a constant dummy value. Since HashMap keys are unique, the set automatically rejects duplicates. add, remove and contains are O(1) on average.”

```
// Inside HashSet
private transient HashMap<E, Object> map;
private static final Object PRESENT = new Object();
public boolean add(E e) { return map.put(e, PRESENT) == null; }
```

Key follow-up: *How does HashSet check duplicates?* — “Using hashCode() and equals() of the element, because it delegates to HashMap. If you store custom objects and don’t override both, duplicates won’t be detected.”

LinkedHashSet IMPORTANT

“A HashSet that **maintains insertion order**, using a LinkedHashMap internally (hash table + doubly linked list across entries). Slightly more memory, same O(1) operations.”

TreeSet IMPORTANT

Interview answer: “TreeSet stores unique elements in sorted order. It’s backed by a TreeMap, which is a Red-Black tree, so add, remove and contains are O(log n). Elements must be Comparable or we pass a Comparator. It doesn’t allow null, because it needs to compare elements.”

Extra methods: first(), last(), floor(x), ceiling(x), higher, lower, headSet, tailSet, descendingSet().

HashSet vs LinkedHashSet vs TreeSet MUST-KNOW

	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion order	Sorted (natural / comparator)
Internal	HashMap	LinkedHashMap	TreeMap (Red-Black tree)
add/remove/contains	O(1)	O(1)	O(log n)
Null	One null	One null	No null (NPE)
Uses for equality	hashCode + equals	hashCode + equals	compareTo / compare
Use when	Fast uniqueness check	Unique + keep order	Unique + sorted / range queries

8.5 Queue and Deque

PriorityQueue **IMPORTANT**

Interview answer: “PriorityQueue is a queue where elements are ordered by priority rather than insertion order. By default it’s a min-heap, so poll() always returns the smallest element. Internally it uses a binary heap stored in an array. offer and poll are O(log n), and peek is O(1).”

```
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
minHeap.addAll(List.of(5, 1, 3));
minHeap.poll(); // 1
```

Trap: Iterating or printing a PriorityQueue does **not** give sorted order — only poll() does. No nulls allowed.

Real-world: Task scheduling by priority, top-K problems, Dijkstra.

Deque / ArrayDeque **IMPORTANT**

“Deque (double-ended queue) allows insertion and removal at both ends. ArrayDeque is a resizable circular array — faster than Stack for LIFO and faster than LinkedList for FIFO. No nulls.”

Queue method (throws)	Queue method (returns special value)
add(e)	offer(e) → false
remove()	poll() → null
element()	peek() → null

8.6 Map Implementations

HashMap **MUST-KNOW**

Interview answer: “HashMap stores data as key-value pairs. Keys are unique, it allows one null key and multiple null values, it doesn’t maintain order, and it isn’t synchronized. It works on hashing, so get and put are O(1) on average.”

(Full internal working in the next section.)

LinkedHashMap **IMPORTANT**

“HashMap + doubly linked list across entries → **maintains insertion order**. With accessOrder = true it maintains access order, which makes it perfect for an **LRU cache** by overriding removeEldestEntry().”

```
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    LRUCache(int cap) { super(16, 0.75f, true); this.cap = cap; }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > cap; }
}
```

TreeMap **IMPORTANT**

“A sorted map based on a **Red-Black tree**. Keys are sorted by natural order or a Comparator. O(log n) operations. No null keys (null values allowed). Offers firstKey, lastKey, floorKey, ceilingKey, headMap, tailMap.”

Hashtable **IMPORTANT**

“Legacy synchronized map. **No null key or null value**. Slower because every method locks the whole table. Use ConcurrentHashMap instead in modern code.”

HashMap vs Hashtable **MUST-KNOW**

	HashMap	Hashtable
Synchronized	No	Yes
Thread safe	No	Yes (but coarse locking)
Null key / values	1 null key, many null values	None allowed
Performance	Faster	Slower
Legacy	No (Java 1.2)	Yes (Java 1.0)
Iteration	Iterator (fail-fast)	Iterator + Enumeration
Parent	AbstractMap	Dictionary
Modern thread-safe alternative	ConcurrentHashMap	—

HashMap vs LinkedHashMap vs TreeMap **MUST-KNOW**

	HashMap	LinkedHashMap	TreeMap
Order	None	Insertion (or access) order	Sorted by key
Internal	Array of buckets	Buckets + doubly linked list	Red-Black tree
get/put	O(1) avg	O(1) avg	O(log n)
Null key	1 allowed	1 allowed	Not allowed
Use when	Fast lookup	Order matters, LRU cache	Sorted keys, range queries

ConcurrentHashMap (common follow-up) **IMPORTANT**

Interview answer: “ConcurrentHashMap is a thread-safe HashMap. Instead of locking the whole map like Hashtable, Java 8’s version uses CAS operations and locks only the individual bucket being written, so many threads can read and write in parallel. Reads are mostly lock-free. It doesn’t allow null keys or values, because null would be ambiguous — you couldn’t tell ‘not present’ from ‘mapped to null’ in a concurrent setting.”

8.7 Important Comparisons

List vs Set **MUST-KNOW**

	List	Set
Duplicates	Allowed	Not allowed
Order	Insertion order maintained	Depends (Hash: none, Linked: insertion, Tree: sorted)
Index access	Yes (get(i))	No
Nulls	Multiple	At most one (none in TreeSet)
Implementations	ArrayList, LinkedList, Vector	HashSet, LinkedHashSet, TreeSet

Collection vs Collections **MUST-KNOW**

Interview answer: “Collection is the root interface of the collection hierarchy — List, Set and Queue extend it. Collections is a utility class with static helper methods like sort, reverse, shuffle, max, min, unmodifiableList and synchronizedList.”

```

Collections.sort(list);
Collections.reverse(list);
Collections.max(list);
Collections.frequency(list, x);
Collections.unmodifiableList(list);
Collections.synchronizedList(list);

```

Comparable vs Comparator **MUST-KNOW**

	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T a, T b)
Where defined	Inside the class being sorted	Separate class / lambda
Sort sequences	One — the “natural order”	Many — any number of custom orders
Modifies class?	Yes	No
Usage	Collections.sort(list)	list.sort(comparator)

Interview answer: “Comparable defines the natural ordering of a class — the class itself implements compareTo, so there’s only one way to sort it. Comparator defines a custom ordering outside the class using compare, so we can have many sorting strategies without modifying the class. For example, Employee can implement Comparable to sort by ID, and I can pass Comparators to sort by name or salary.”

```
class Employee implements Comparable<Employee> {
    int id; String name; double salary;
    public int compareTo(Employee o) { return Integer.compare(this.id, o.id); }
}

Collections.sort(emps); // by id
emps.sort(Comparator.comparing(e -> e.name)); // by name
emps.sort(Comparator.comparingDouble((Employee e) -> e.salary).reversed()
    .thenComparing(e -> e.name)); // salary desc, then name
```

Trap: Don’t write `return this.salary - o.salary;` for ints — it can overflow. Use `Integer.compare()`.

Iterator vs ListIterator IMPORTANT

	Iterator	ListIterator
Works on	Any Collection	Only List
Direction	Forward only	Forward and backward
Methods	hasNext, next, remove	+ hasPrevious, previous, add, set, nextIndex
Can add/replace	No	Yes

Fail-fast vs Fail-safe MUST-KNOW

Interview answer: “A fail-fast iterator throws ConcurrentModificationException if the collection is structurally modified while iterating, except through the iterator’s own remove. ArrayList and HashMap iterators are fail-fast — they track a modCount. Fail-safe iterators, like those of CopyOnWriteArrayList and ConcurrentHashMap, work on a copy or a weakly consistent view, so they don’t throw.”

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4));
for (Integer x : list)
    if (x == 2) list.remove(x); // ConcurrentModificationException

Iterator<Integer> it = list.iterator();
while (it.hasNext())
    if (it.next() == 2) it.remove(); // safe

list.removeIf(x -> x == 2); // cleanest (Java 8)
```

8.8 Time Complexity Cheat Table

Collection	add	remove	get / contains	Ordered?	Duplicates?	Null?
ArrayList	$O(1)^*$	$O(n)$	$O(1)$ get / $O(n)$ contains	Insertion	Yes	Yes
LinkedList	$O(1)$ ends	$O(1)$ ends	$O(n)$	Insertion	Yes	Yes
HashSet	$O(1)$	$O(1)$	$O(1)$	No	No	One
LinkedHashSet	$O(1)$	$O(1)$	$O(1)$	Insertion	No	One
TreeSet	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted	No	No
PriorityQueue	$O(\log n)$	$O(\log n)$ poll	$O(1)$ peek	Heap	Yes	No
ArrayDeque	$O(1)$	$O(1)$ ends	$O(n)$ contains	Insertion	Yes	No
HashMap	$O(1)$	$O(1)$	$O(1)$	No	Keys unique	1 key
LinkedHashMap	$O(1)$	$O(1)$	$O(1)$	Insertion	Keys unique	1 key
TreeMap	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sorted	Keys unique	No key

* amortised

9 HashMap Internal Working

This is one of the most asked Java questions. Learn the 2-minute answer at the end of this section word for word, then understand each part so you can handle follow-ups.

9.1 What is HashMap? MUST-KNOW

Interview answer: “HashMap is a class in java.util that implements the Map interface and stores data as key-value pairs. It uses hashing to store and retrieve entries in O(1) average time. It allows one null key and many null values, doesn't guarantee order, and isn't thread-safe.”

9.2 How HashMap Stores Data

Internally, a HashMap is an **array of buckets**. Each bucket holds a linked list (or a tree) of `Node` objects.

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;    // next node in same bucket
}
transient Node<K,V>[] table;    // the bucket array
```

```
table (capacity 16)
index 0: null
index 1: [hash|"apple"|10|next] -> [hash|"grape"|40|null] ← collision chain
index 2: null
index 3: [hash|"mango"|20|null]
...
index 15: [hash|"kiwi"|30|null]
```

9.3 Hashing, hashCode() and Bucket Index

Interview answer: “Hashing is converting a key into an integer, the hash code, which is then used to find the bucket index. HashMap calls the key's hashCode(), spreads the bits by XOR-ing the high 16 bits into the low 16, and then computes the index as hash AND (n - 1), where n is the table size. Since n is always a power of two, this is a fast replacement for hash modulo n.”

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16);
}
int index = (n - 1) & hash;    // n = table length (power of 2)
```

Why XOR with `h >> 16`? Because with a small table only the low bits are used for the index. Mixing in the high bits reduces collisions for keys whose hash codes differ only in high bits.

Why is capacity a power of 2? So `(n-1) & hash` equals `hash % n` but faster, and elements distribute evenly.

9.4 Role of equals()

`hashCode()` finds the **bucket**. `equals()` finds the **exact key inside the bucket**. Two different keys can land in the same bucket; `equals()` tells them apart.

9.5 Collision and Collision Handling MUST-KNOW

Interview answer: “A collision happens when two different keys map to the same bucket index. HashMap handles collisions with chaining — it stores the entries in the same bucket as a linked list. From Java 8, if a bucket’s list grows beyond 8 nodes and the table has at least 64 buckets, that list is converted into a Red-Black tree, which brings the worst-case lookup in that bucket from $O(n)$ down to $O(\log n)$.”

9.6 Treeification IMPORTANT

Constant	Value	Meaning
TREEIFY_THRESHOLD	8	Chain length at which a bucket becomes a tree
MIN_TREEIFY_CAPACITY	64	Table must be at least this big; otherwise resize instead
UNTREEIFY_THRESHOLD	6	Tree shrinks back to list at this size (during resize)

Why a tree? Protects against many keys with the same hash (bad `hashCode()` or deliberate hash-flooding attacks).

9.7 Initial Capacity, Load Factor and Rehashing MUST-KNOW

Interview answer: “The default initial capacity is 16 buckets and the default load factor is 0.75. The threshold is capacity times load factor — 12 for the default. When the number of entries exceeds the threshold, HashMap resizes: it doubles the capacity and redistributes all entries into the new table. This is called rehashing. 0.75 is a balance between memory usage and the number of collisions.”

- Default capacity **16**, load factor **0.75** → threshold **12**. The 13th insert triggers resize to 32.
- Table is **lazily created** on the first `put`.
- On resize in Java 8, each node either stays at index `i` or moves to `i + oldCapacity` — decided by one extra bit of the hash, so no full rehash computation is needed.
- **Tip:** If you know the size up front, set initial capacity to avoid repeated resizes: `new HashMap<>(expected / 0.75f + 1)`.

9.8 put() — Step by Step MUST-KNOW

```
map.put("apple", 10);
```

1. If table is null → create it (capacity 16).
2. Compute `hash("apple")` → spread bits.
3. `Index = (n-1) & hash`.
4. If bucket empty → place new Node there.
5. If bucket not empty → walk the chain/tree:
 - If a node has same hash **and** `key.equals(existing)` → **replace value**, return old value.
 - Otherwise append new node at the tail (Java 8; Java 7 inserted at head).
6. If chain length ≥ 8 → treeify (or resize if table < 64).
7. `++size`; if `size > threshold` → resize (double + redistribute).

9.9 get() — Step by Step MUST-KNOW

1. Compute hash of key → find index.
2. Check the first node: same hash and `equals()` → return value.
3. Otherwise walk the list (or search the tree) comparing with `equals()`.
4. Not found → return `null`.

9.10 Why HashMap Allows One Null Key MUST-KNOW

Interview answer: “HashMap handles null specially — the hash of a null key is taken as 0, so it always goes to bucket 0, and it doesn’t call hashCode() on it. Since keys are unique, only one null key can exist; putting null again just replaces its value. Hashtable and ConcurrentHashMap don’t allow null because they call hashCode() directly on keys, and in concurrent maps null would be ambiguous.”

9.11 Why HashMap is Not Synchronized IMPORTANT

Interview answer: “HashMap isn’t synchronized by design, for performance — most maps are used by a single thread, and locking would slow them down. If multiple threads modify a HashMap at the same time, we can lose updates or corrupt its structure. For thread safety we use ConcurrentHashMap, or Collections.synchronizedMap for simple cases.”

9.12 Time Complexity

Case	get / put / remove
Average	O(1)
Worst (Java 7, all in one bucket as list)	O(n)
Worst (Java 8+, treeified bucket)	O(log n)

9.13 What if we use a mutable object as key? IMPORTANT

“If a key’s fields change after insertion, its hashCode changes, so get() looks in the wrong bucket and the entry is effectively lost. That’s why immutable keys like String and Integer are preferred.”

9.14 What if we override equals() but not hashCode()? IMPORTANT

```
class Emp {
    int id;
    Emp(int id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return o instanceof Emp e && e.id == id;
    }
    // hashCode NOT overridden
}
Map<Emp, String> m = new HashMap<>();
m.put(new Emp(1), "A");
m.get(new Emp(1)); // null! different hashCodes → different buckets
```

“Equal objects must have equal hash codes. Without overriding hashCode, two equal keys usually land in different buckets, so HashMap can’t find them.”

9.15 The 2-Minute Answer: “How does HashMap work internally?” MUST-KNOW

“HashMap stores key-value pairs in an array of buckets, where each bucket is a linked list of nodes. Each node holds the hash, the key, the value and a pointer to the next node.

When we call **put(key, value)**, HashMap first calls hashCode() on the key and spreads the bits to reduce collisions. Then it finds the bucket index using hash AND (capacity – 1). If that bucket is empty, the entry goes straight in. If it’s not empty, that’s a collision — HashMap walks through the nodes in that bucket and uses equals() to check if the key already exists. If it does, it replaces the value; if not, it adds a new node at the end of the list.

For **get(key)**, it does the same thing — computes the hash, goes to the bucket, and uses equals() to find the matching key. So hashCode() finds the bucket and equals() finds the exact entry. That’s why both must be overridden together for custom keys.

The default capacity is 16 and the load factor is 0.75, so when the map has more than 12 entries it doubles its size and redistributes the entries — that’s rehashing.

From Java 8, if one bucket gets more than 8 entries and the table has at least 64 buckets, the linked list is converted into a Red-Black tree, so the worst case becomes $O(\log n)$ instead of $O(n)$. On average, get and put are $O(1)$.

HashMap allows one null key, which is always stored in bucket 0, and it’s not thread-safe — for concurrent access we use ConcurrentHashMap.”

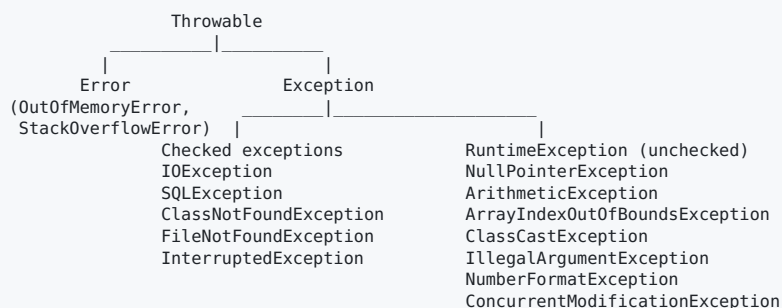
Likely follow-ups after this answer: Why load factor 0.75? · Why power of 2? · What happens during resize? · Difference with ConcurrentHashMap? · What if two keys have the same hashCode? · Can HashMap key be a custom class? (Yes, override equals + hashCode, prefer immutable.)

10 Exception Handling

10.1 What is an Exception? MUST-KNOW

Interview answer: “An exception is an unwanted event that occurs during program execution and disrupts the normal flow — like dividing by zero, accessing a null reference or a missing file. Exception handling is the mechanism to handle these events gracefully using try, catch, finally, throw and throws, so the program doesn’t crash abruptly and we can recover or show a meaningful message.”

10.2 Exception Hierarchy MUST-KNOW



10.3 Exception vs Error MUST-KNOW

	Exception	Error
Cause	Problems in application code/conditions	Serious problems in the JVM/system
Recoverable	Usually yes	Usually no
Should we catch?	Yes	Generally no
Types	Checked and unchecked	All unchecked
Examples	IOException, NullPointerException	OutOfMemoryError, StackOverflowError

Interview answer: “Both extend Throwable. An Exception represents a condition the application can reasonably catch and recover from, like a missing file or invalid input. An Error represents a serious problem in the JVM, like running out of memory or a stack overflow, which the application normally shouldn’t try to catch.”

10.4 Checked vs Unchecked Exceptions MUST-KNOW

	Checked	Unchecked
Checked at	Compile time	Runtime
Must handle/declare?	Yes (try-catch or throws)	No
Extends	Exception (not RuntimeException)	RuntimeException (or Error)
Usually caused by	External factors (file, network, DB)	Programming mistakes
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException

Interview answer: “Checked exceptions are checked by the compiler — if a method can throw one, we must either catch it or declare it with throws, otherwise the code won’t compile. They usually represent external problems like file or database errors. Unchecked exceptions extend RuntimeException, aren’t checked at compile time, and usually indicate bugs in the code, like a null pointer or a bad array index.”

10.5 try, catch, finally MUST-KNOW

```
try {  
    int result = 10 / 0;           // risky code  
} catch (ArithmeticException e) {  
    System.out.println("Cannot divide by zero: " + e.getMessage());  
} finally {  
    System.out.println("Always runs – cleanup here");  
}
```

- `try` — code that may throw.
- `catch` — handles a specific exception type.
- `finally` — always runs (success, exception, or even `return` in `try`).

Rules: `try` must be followed by at least one `catch` or a `finally`. Catch blocks go from **specific to general** — putting `catch (Exception e)` before `catch (IOException e)` is a compile error (“already caught”).

When does finally NOT run? `System.exit()`, JVM crash, or the thread being killed.

Trick:

```
static int test() {  
    try { return 1; }  
    finally { return 2; } // overrides! returns 2 (bad practice)  
}
```

10.6 Multiple catch and Multi-catch

```
try {  
    // ...  
} catch (NumberFormatException | NullPointerException e) { // Java 7 multi-catch  
    System.out.println("Bad input");  
} catch (Exception e) {  
    System.out.println("Something else");  
}
```

In multi-catch, the types can't be parent-child of each other, and `e` is effectively final.

10.7 Nested try

A try block inside another try. If the inner catch doesn't handle the exception, it propagates to the outer catch.

10.8 throw and throws MUST-KNOW

```
void withdraw(double amt) throws InsufficientFundsException { // declares  
    if (amt > balance)  
        throw new InsufficientFundsException("Balance too low"); // throws  
    balance -= amt;  
}
```

(Comparison table is in the Keywords section.)

10.9 Custom Exceptions MUST-KNOW

Interview answer: “A custom exception is our own exception class, created by extending `Exception` for a checked exception or `RuntimeException` for an unchecked one. We use them to represent business-specific errors, like `InsufficientFundsException` or `UserNotFoundException`, which makes the code more readable and the errors more meaningful.”

```

public class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(Long id) {
        super("User not found: " + id);
    }
}

```

Follow-up: *Checked or unchecked for custom exceptions?* — “In modern apps, like Spring Boot, I’d usually go unchecked (RuntimeException), and handle it centrally with a global exception handler. Checked is suitable when the caller can realistically recover.”

10.10 try-with-resources MUST-KNOW

Interview answer: “try-with-resources, introduced in Java 7, automatically closes resources like files, streams and database connections at the end of the try block. Any resource that implements AutoCloseable can be used. It removes the need for a finally block just for closing, and it’s safer because resources get closed even when an exception occurs.”

```

try (BufferedReader br = new BufferedReader(new FileReader("data.txt"));
     Connection con = DriverManager.getConnection(url, user, pass)) {
    System.out.println(br.readLine());
} catch (IOException | SQLException e) {
    e.printStackTrace();
} // br and con closed automatically, in reverse order

```

Follow-up: *Suppressed exceptions?* — If both the body and `close()` throw, the body’s exception is thrown and the close exception is attached via `getSuppressed()`.

10.11 Exception Propagation IMPORTANT

“If a method doesn’t catch an exception, it’s passed up the call stack to the caller, then its caller, until someone catches it. If nobody does, the JVM’s default handler prints the stack trace and the thread terminates.”

10.12 Common Exceptions Asked

Exception	Cause
NullPointerException	Calling a method/field on <code>null</code>
ArithmeticException	Integer divide by zero
ArrayIndexOutOfBoundsException	Invalid array index
StringIndexOutOfBoundsException	Invalid string index
NumberFormatException	<code>Integer.parseInt("abc")</code>
ClassCastException	Invalid downcast
ConcurrentModificationException	Modifying collection during iteration
IllegalArgumentException	Bad argument passed
IllegalStateException	Method called at wrong time
UnsupportedOperationException	<code>Arrays.asList(...).add()</code> / <code>List.of().add()</code>
FileNotFoundException / IOException	File/IO problems (checked)
ClassNotFoundException	Class not found at runtime via <code>Class.forName</code> (checked)
InterruptedException	Thread interrupted while waiting/sleeping (checked)

Note: `10 / 0` throws `ArithmeticException`, but `10.0 / 0` gives `Infinity` — no exception.

10.13 Best Practices (good to mention)

- Catch specific exceptions, not `Exception/Throwable`.

- Never swallow exceptions with an empty catch.
- Use try-with-resources for closing resources.
- Throw early, catch late; include meaningful messages.
- Don't use exceptions for normal control flow.

11 Multithreading

11.1 Process vs Thread MUST-KNOW

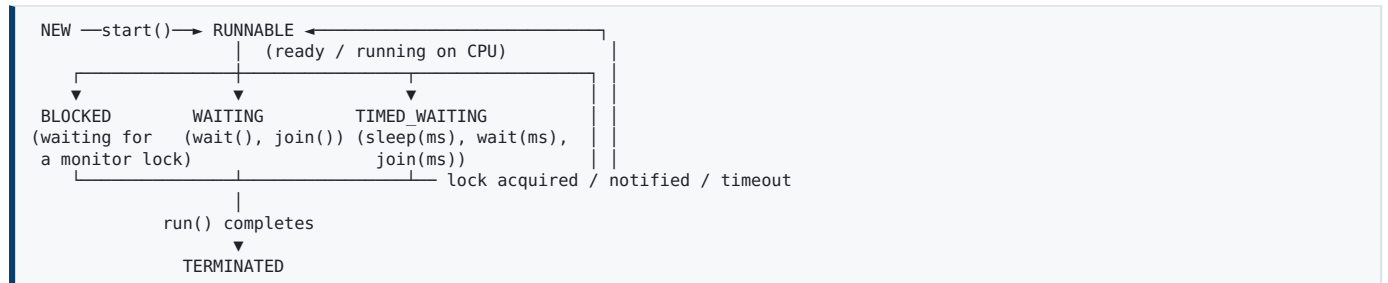
Interview answer: “A process is an independent program in execution with its own memory space. A thread is the smallest unit of execution inside a process — a lightweight sub-process. Multiple threads in the same process share the same heap memory but each has its own stack, so they’re cheaper to create and communicate faster than processes.”

	Process	Thread
Definition	Program in execution	Unit of execution within a process
Memory	Own separate memory	Shares heap with other threads; own stack
Creation cost	Heavy	Light
Communication	Slow (IPC)	Fast (shared memory)
Failure impact	Isolated	One bad thread can affect the whole process
Example	Chrome browser, IntelliJ	Each tab’s rendering, auto-save in background

11.2 Multithreading MUST-KNOW

Interview answer: “Multithreading is running multiple threads concurrently within a single program so tasks can progress at the same time. It improves CPU utilisation and responsiveness — for example, a web server handles each request on a separate thread, so one slow request doesn’t block the others.”

11.3 Thread Lifecycle MUST-KNOW



Interview answer: “A thread goes through these states: **NEW** when it’s created but `start()` hasn’t been called; **RUNNABLE** when it’s ready or running; **BLOCKED** when it’s waiting to acquire a lock; **WAITING** when it waits indefinitely for another thread, like with `wait()` or `join()`; **TIMED_WAITING** when it waits for a fixed time, like `sleep(1000)`; and **TERMINATED** when `run()` finishes.”

11.4 Creating Threads MUST-KNOW

1. Extending Thread

```
class MyThread extends Thread {
    public void run() { System.out.println("Running: " + getName()); }
}
new MyThread().start();
```

2. Implementing Runnable (preferred)

```
Runnable task = () -> System.out.println("Running in " + Thread.currentThread().getName());
new Thread(task).start();
```

3. Implementing Callable (returns a result)

```
Callable<Integer> job = () -> 42;
ExecutorService ex = Executors.newSingleThreadExecutor();
Future<Integer> f = ex.submit(job);
System.out.println(f.get()); // 42 (blocks until ready)
ex.shutdown();
```

Thread vs Runnable **MUST-KNOW**

	Extending Thread	Implementing Runnable
Inheritance	Uses up your one <code>extends</code>	Class can still extend another class
Design	Couples task with thread	Separates task from thread
Reuse	Each object is a thread	Same task can be given to many threads / executors
Recommended	No	Yes

Interview answer: “Implementing Runnable is preferred because Java allows only single inheritance — if we extend Thread, we can’t extend any other class. Runnable also separates the task from the thread that runs it, so the same task can be submitted to a thread pool.”

Runnable vs Callable

	Runnable	Callable
Method	<code>void run()</code>	<code>V call()</code> throws Exception
Returns value	No	Yes (via Future)
Throws checked exception	No	Yes
Since	Java 1.0	Java 5

11.5 start() vs run() **MUST-KNOW**

Interview answer: “start() creates a new thread and then the JVM calls run() on that new thread. If we call run() directly, no new thread is created — it just executes like a normal method on the current thread. Also, start() can be called only once per thread; calling it twice throws `IllegalThreadStateException`.”

11.6 sleep(), wait(), notify(), notifyAll(), join()

Method	Belongs to	What it does
<code>sleep(ms)</code>	Thread (static)	Pauses current thread; keeps locks
<code>wait()</code>	Object	Releases lock and waits until notified; must be inside <code>synchronized</code>
<code>notify()</code>	Object	Wakes one waiting thread on that object
<code>notifyAll()</code>	Object	Wakes all waiting threads on that object
<code>join()</code>	Thread	Current thread waits until the other thread finishes
<code>yield()</code>	Thread (static)	Hint to scheduler to let other threads run

sleep() vs wait() **MUST-KNOW**

	sleep()	wait()
Class	Thread	Object
Lock	Does not release	Releases the lock
Called from	Anywhere	Only inside synchronized block/method
Wakes up	After time elapses	notify/notifyAll or timeout
Purpose	Pause execution	Inter-thread communication
Static	Yes	No

Interview answer: “sleep() is a static method of Thread that pauses the current thread for a given time but keeps any locks it holds. wait() is a method of Object used for inter-thread communication — it must be called inside a synchronized block, and it releases the lock until another thread calls notify() or notifyAll().”

Follow-up: Why are wait/notify in Object, not Thread? — “Because they work on an object’s lock (monitor), and every object in Java can act as a lock.”

11.7 Race Condition and Synchronization MUST-KNOW

Interview answer: “A race condition happens when two or more threads access and modify shared data at the same time and the final result depends on the timing of their execution. Synchronization fixes this by allowing only one thread at a time to execute the critical section, using the synchronized keyword or locks.”

```
class Counter {
    private int count = 0;
    public void increment() { count++; } // NOT atomic: read → add → write
}
// Two threads × 1000 increments → often less than 2000

class SafeCounter {
    private int count = 0;
    public synchronized void increment() { count++; } // one thread at a time
}

class AtomicCounter {
    private final AtomicInteger count = new AtomicInteger();
    public void increment() { count.incrementAndGet(); } // lock-free, CAS-based
}
```

synchronized method vs synchronized block MUST-KNOW

```
public synchronized void update() { ... } // locks 'this' for whole method

public void update() {
    // non-critical work here runs in parallel
    synchronized (this) { balance += x; } // lock only the critical part
}

public static synchronized void log() { ... } // locks the Class object
```

	synchronized method	synchronized block
Scope	Entire method	Only specified lines
Lock	this (or Class for static)	Any object you choose
Performance	Lower (bigger locked area)	Better (smaller critical section)
Flexibility	Less	More

Interview answer: “A synchronized method locks the whole method on the current object, or on the class object if it’s static. A synchronized block locks only a specific section of code, and we can choose which object to lock on. Blocks are usually better because they keep the locked area small, which improves concurrency.”

11.8 Thread Safety IMPORTANT

Interview answer: “A class is thread-safe if it behaves correctly when accessed by multiple threads at the same time, without any extra synchronization by the caller. We achieve it through immutability, synchronization, atomic classes like AtomicInteger, thread-safe collections like ConcurrentHashMap, or by not sharing state at all, for example with local variables.”

11.9 volatile IMPORTANT

```
class Worker implements Runnable {
    private volatile boolean running = true;
    public void run() { while (running) { /* work */ } }
    public void stop() { running = false; } // visible to worker immediately
}
```

“Without `volatile`, the worker thread might keep reading a cached `true` forever. `volatile` guarantees **visibility**, not **atomicity** — for `count++` you still need `synchronized` or `AtomicInteger`.”

	volatile	synchronized
Guarantees	Visibility	Visibility + atomicity (mutual exclusion)
Applies to	Variables	Methods / blocks
Blocking	Never blocks	Can block

11.10 Deadlock MUST-KNOW

Interview answer: “A deadlock is a situation where two or more threads are blocked forever, each waiting for a lock held by the other. For example, thread 1 holds lock A and waits for lock B, while thread 2 holds lock B and waits for lock A. We avoid it by always acquiring locks in the same order, using timeouts with `tryLock`, and keeping synchronized sections small.”

```
Object A = new Object(), B = new Object();
new Thread(() -> { synchronized (A) { sleep(); synchronized (B) { } } }).start();
new Thread(() -> { synchronized (B) { sleep(); synchronized (A) { } } }).start();
// Fix: both threads lock A first, then B
```

Four conditions for deadlock: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent it.

11.11 ExecutorService Basics IMPORTANT

Interview answer: “ExecutorService is a framework for managing a pool of threads. Instead of creating a new thread for each task, we submit tasks to the executor and it reuses a fixed set of worker threads. This avoids the cost of creating threads repeatedly and gives us control over how many run at once. We must call `shutdown()` when done.”

```
ExecutorService pool = Executors.newFixedThreadPool(4);
for (int i = 1; i <= 10; i++) {
    int id = i;
    pool.submit(() -> System.out.println("Task " + id + " on " + Thread.currentThread().getName()));
}
pool.shutdown(); // stop accepting, finish queued
pool.awaitTermination(5, TimeUnit.SECONDS);
```

Factory	Behaviour
<code>newFixedThreadPool(n)</code>	n threads, tasks queue up
<code>newCachedThreadPool()</code>	Creates threads as needed, reuses idle ones
<code>newSingleThreadExecutor()</code>	One thread, tasks run in order
<code>newScheduledThreadPool(n)</code>	Run after a delay or periodically
<code>newVirtualThreadPerTaskExecutor()</code> (Java 21)	One lightweight virtual thread per task

`submit()` **vs** `execute()` : `execute(Runnable)` returns nothing; `submit()` accepts `Runnable` or `Callable` and returns a `Future`.

Virtual threads (Java 21) — a good bonus line: “They’re lightweight threads managed by the JVM, not the OS, so we can have millions of them — great for I/O-heavy apps like web servers.”

12 Java 8+ Features

12.1 Lambda Expressions MUST-KNOW

Interview answer: “A lambda expression is a short way to write an anonymous function — a block of code with parameters and a body but no name. It’s used to provide the implementation of a functional interface, and it makes code shorter and more readable, especially with collections and streams.”

```
// Syntax: (parameters) -> expression or (parameters) -> { statements }

// Before Java 8
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Hi"); }
};

// With lambda
Runnable r2 = () -> System.out.println("Hi");

Comparator<String> byLen = (a, b) -> a.length() - b.length();
list.forEach(x -> System.out.println(x));
```

Rule: A lambda can use local variables from the enclosing scope only if they are **effectively final**.

12.2 Functional Interfaces MUST-KNOW

Interview answer: “A functional interface is an interface with exactly one abstract method. It can still have default and static methods. Lambdas and method references are used to implement them. We can mark them with `@FunctionalInterface` so the compiler checks there’s only one abstract method. Examples are `Runnable`, `Comparator`, `Callable`, and the `java.util.function` interfaces like `Predicate` and `Function`.”

```
@FunctionalInterface
interface Calculator { int operate(int a, int b); }

Calculator add = (a, b) -> a + b;
Calculator mul = (a, b) -> a * b;
add.operate(2, 3); // 5
```

The four core functional interfaces

Interface	Method	Input → Output	Example
<code>Predicate<T></code>	<code>boolean test(T t)</code>	<code>T → boolean</code>	<code>Predicate<Integer> isEven = n -> n % 2 == 0;</code>
<code>Function<T,R></code>	<code>R apply(T t)</code>	<code>T → R</code>	<code>Function<String,Integer> len = String::length;</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>	<code>T → nothing</code>	<code>Consumer<String> print = System.out::println;</code>
<code>Supplier<T></code>	<code>T get()</code>	<code>nothing → T</code>	<code>Supplier<Double> rnd = Math::random;</code>

Also: `BiFunction<T,U,R>`, `UnaryOperator<T>` (`T→T`), `BinaryOperator<T>` (`(T,T)→T`), `BiConsumer`, `BiPredicate`.

Chaining: `isEven.and(isPositive)`, `.or()`, `.negate()`; `f.andThen(g)`, `f.compose(g)`.

Interview answer: “`Predicate` takes a value and returns a boolean — used for filtering. `Function` takes a value and returns another value — used for transforming. `Consumer` takes a value and returns nothing — used for actions like printing. `Supplier` takes nothing and returns a value — used for generating or lazily supplying values.”

12.3 Method References IMPORTANT

Interview answer: “A method reference is a shorter form of a lambda that just calls an existing method. We write it with the double-colon operator, like `String::toUpperCase` or `System.out::println`.”

Type	Syntax	Lambda equivalent
Static method	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
Instance method of a particular object	<code>System.out::println</code>	<code>x -> System.out.println(x)</code>
Instance method of an arbitrary object	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>
Constructor	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>

12.4 Stream API MUST-KNOW

Interview answer: “The Stream API, introduced in Java 8, lets us process collections of data in a functional, declarative way — we say what we want, not how to loop. A stream pipeline has a source, zero or more intermediate operations like `filter`, `map` and `sorted`, and one terminal operation like `collect`, `forEach` or `reduce`. Streams don’t store data or modify the source, and they’re lazy — nothing runs until the terminal operation is called.”

Key properties:

1. **Not a data structure** — it’s a pipeline over a source.
2. **Doesn’t modify the source.**
3. **Lazy** — intermediate operations run only when a terminal operation is invoked.
4. **Single use** — a stream can’t be reused after a terminal operation (`IllegalStateException`).
5. Can be **parallel** with `parallelStream()`.

Intermediate vs Terminal Operations

Intermediate (return a Stream, lazy)	Terminal (produce a result, trigger execution)
<code>filter</code> , <code>map</code> , <code>flatMap</code> , <code>distinct</code> , <code>sorted</code> , <code>limit</code> , <code>skip</code> , <code>peek</code>	<code>collect</code> , <code>forEach</code> , <code>reduce</code> , <code>count</code> , <code>min</code> , <code>max</code> , <code>anyMatch</code> , <code>allMatch</code> , <code>noneMatch</code> , <code>findFirst</code> , <code>findAny</code> , <code>toList()</code> (16)

Practical examples (use this sample data)

```
record Employee(String name, String dept, double salary, int age) {}

List<Employee> emps = List.of(
    new Employee("Asha", "IT", 75000, 28),
    new Employee("Ravi", "HR", 45000, 35),
    new Employee("Kiran", "IT", 92000, 31),
    new Employee("Meena", "Sales", 50000, 26),
    new Employee("Arjun", "IT", 60000, 24));
```

filter() — keep elements that match a condition

```
List<Employee> it = emps.stream()
    .filter(e -> e.dept().equals("IT"))
    .toList();
```

map() — transform each element

```
List<String> names = emps.stream().map(Employee::name).toList();
List<Integer> squares = List.of(1, 2, 3).stream().map(n -> n * n).toList(); // [1, 4, 9]
```

flatMap() — flatten nested structures (one-to-many)

```
List<List<Integer>> nested = List.of(List.of(1, 2), List.of(3, 4));
List<Integer> flat = nested.stream().flatMap(List::stream).toList(); // [1, 2, 3, 4]

List<String> words = List.of("hello world", "java streams");
List<String> all = words.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .toList(); // [hello, world, java, streams]
```

map vs flatMap — interview answer: “map transforms each element into exactly one element, so a stream of lists stays a stream of lists. flatMap transforms each element into a stream and then flattens all those streams into one.”

reduce() — combine elements into one value

```
int sum = List.of(1, 2, 3, 4).stream().reduce(0, Integer::sum); // 10
double total = emps.stream().map(Employee::salary).reduce(0.0, Double::sum);
Optional<Integer> max = List.of(4, 9, 2).stream().reduce(Integer::max); // 9
```

sorted(), distinct(), limit(), skip()

```
List.of(5, 3, 5, 1, 3).stream().distinct().sorted().toList(); // [1, 3, 5]

// Top 2 highest paid
emps.stream()
    .sorted(Comparator.comparingDouble(Employee::salary).reversed())
    .limit(2)
    .map(Employee::name)
    .toList(); // [Kiran, Asha]
```

forEach()

```
emps.stream().filter(e -> e.age() < 30).forEach(e -> System.out.println(e.name()));
```

collect() with Collectors — the most asked

```
// Group by department
Map<String, List<Employee>> byDept =
    emps.stream().collect(Collectors.groupingBy(Employee::dept));

// Count per department
Map<String, Long> countByDept =
    emps.stream().collect(Collectors.groupingBy(Employee::dept, Collectors.counting()));

// Average salary per department
Map<String, Double> avgSal =
    emps.stream().collect(Collectors.groupingBy(Employee::dept,
        Collectors.averagingDouble(Employee::salary)));

// Partition by condition (true/false)
Map<Boolean, List<Employee>> highEarnings =
    emps.stream().collect(Collectors.partitioningBy(e -> e.salary() > 60000));

// Join names
String csv = emps.stream().map(Employee::name).collect(Collectors.joining(", "));

// To map: name -> salary
Map<String, Double> m = emps.stream()
    .collect(Collectors.toMap(Employee::name, Employee::salary));

// Highest paid employee
Optional<Employee> top = emps.stream().max(Comparator.comparingDouble(Employee::salary));
```

Most-asked stream coding questions

```

List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 2, 4);

// 1. Sum of even numbers
nums.stream().filter(n -> n % 2 == 0).mapToInt(Integer::intValue).sum();

// 2. Find duplicates
Set<Integer> seen = new HashSet<>();
nums.stream().filter(n -> !seen.add(n)).collect(Collectors.toSet()); // [2, 4]

// 3. Frequency of each character in a string
Map<Character, Long> freq = "banana".chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()));
// {b=1, a=3, n=2}

// 4. First non-repeating character
"swiss".chars().mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream().filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey).findFirst(); // Optional[w]

// 5. Second highest number
nums.stream().distinct().sorted(Comparator.reverseOrder()).skip(1).findFirst(); // 5

// 6. Names starting with 'A', uppercase, sorted
emps.stream().map(Employee::name).filter(n -> n.startsWith("A"))
    .map(String::toUpperCase).sorted().toList(); // [ARJUN, ASHA]

// 7. Count strings longer than 3 chars
List.of("a", "java", "go", "spring").stream().filter(s -> s.length() > 3).count(); // 2

```

Stream vs Collection

	Collection	Stream
Purpose	Stores data	Processes data
Evaluation	Eager	Lazy
Reusable	Yes	No (single use)
Iteration	External (you write the loop)	Internal (stream handles it)
Modifies source	Can	Never

12.5 Optional IMPORTANT

Interview answer: “Optional is a container object that may or may not hold a value. It was introduced in Java 8 to avoid NullPointerException and to make it explicit in the method signature that a result might be absent. Instead of returning null, a method returns Optional, and the caller uses methods like `orElse`, `ifPresent` or `map` to handle the missing case safely.”

```

Optional<String> name = Optional.ofNullable(findUser(id)); // may be empty

name.isPresent(); // true/false
name.ifPresent(System.out::println);
String n = name.orElse("Guest"); // default value
String m = name.orElseGet(() -> loadDefault()); // lazy default
String x = name.orElseThrow(() -> new UserNotFoundException(id));
name.map(String::toUpperCase).filter(s -> s.startsWith("A"));

```

Follow-ups:

- `Optional.of(null)` → NPE; use `ofNullable`.
- `orElse` vs `orElseGet` — `orElse` always evaluates its argument; `orElseGet` only when empty.
- Don't use Optional for fields or method parameters; it's meant for return types.

12.6 Default and Static Methods in Interfaces IMPORTANT

Interview answer: “Java 8 allowed interfaces to have default methods, which have a body and are inherited by implementing classes, and static methods, which belong to the interface itself and are called using the interface name. Default methods were added so that new methods could be added to existing interfaces — like `forEach` in `Iterable` — without breaking every class that implemented them.”

```
interface Vehicle {  
    default void honk() { System.out.println("Beep"); } // can be overridden  
    static int wheels() { return 4; } // Vehicle.wheels()  
}
```

Follow-up: Static interface methods are **not** inherited — you can't call them via the implementing class or an object.

12.7 Other Java 8+ features worth naming

- **New Date-Time API** (`java.time`): `LocalDate`, `LocalDateTime`, `Duration`, `Period` — immutable and thread-safe, replacing `Date`/`Calendar`.
- **var** (Java 10): local variable type inference — `var list = new ArrayList<String>();`
- **Records** (Java 16): immutable data carriers — `record Point(int x, int y) {}` auto-generates constructor, getters, `equals`, `hashCode`, `toString`.
- **Text blocks** (Java 15): multi-line strings with `"""`.
- **Switch expressions** (Java 14), **pattern matching for instanceof** (Java 16), **sealed classes** (Java 17).

13 Java Memory & JVM

13.1 JVM Memory Areas MUST-KNOW

Interview answer: “The JVM divides memory into several areas. The heap stores all objects and is shared by all threads. Each thread has its own stack, which stores method call frames, local variables and references. The method area — implemented as Metaspace since Java 8 — stores class-level data like class structure, method code and the runtime constant pool. Each thread also has a program counter register pointing to the current instruction, and a native method stack for native code.”

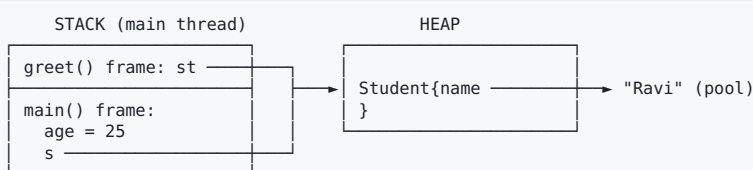
Area	Stores	Shared?	Error when full
Heap	Objects, arrays, instance variables, String pool	All threads	OutOfMemoryError: Java heap space
Stack	Method frames, local variables, references, partial results	Per thread	StackOverflowError
Method Area / Metaspace	Class metadata, method bytecode, runtime constant pool, static info	All threads	OutOfMemoryError: Metaspace
PC Register	Address of current instruction	Per thread	—
Native Method Stack	Native (C/C++) method calls	Per thread	—

Metaspace vs PermGen IMPORTANT

“Before Java 8, class metadata was stored in PermGen, which had a fixed maximum size and often caused OutOfMemoryError: PermGen space. Java 8 replaced it with **Metaspace**, which lives in native memory and grows automatically (can be capped with -XX:MaxMetaspaceSize). The String pool moved to the heap in Java 7.”

13.2 Stack vs Heap MUST-KNOW

```
public static void main(String[] args) {  
    int age = 25;           // stack: primitive value  
    Student s = new Student("Ravi"); // stack: reference s; heap: Student object  
    greet(s);              // new stack frame for greet  
}
```



	Stack	Heap
Stores	Local variables, method calls, references	Objects and their instance variables
Access	Per-thread (thread-safe by nature)	Shared by all threads
Size	Small	Large
Speed	Faster (LIFO)	Slower
Memory management	Automatic when method returns	Garbage collector
Lifetime	Until method ends	Until no references remain
Error	StackOverflowError	OutOfMemoryError

Interview answer: “Stack memory is used for method execution — each thread has its own stack, and every method call creates a frame holding its local variables and object references. It’s fast and freed automatically when the method returns. Heap memory stores the actual objects, is shared by all threads, and is cleaned up by the garbage collector. So in `Student s = new Student()`, the reference `s` lives on the stack and the `Student` object lives on the heap.”

13.3 Object Creation — what happens on new **IMPORTANT**

1. **Class loading** (if not loaded yet) — load, link (verify, prepare, resolve), initialise (static blocks run).
2. **Memory allocation** in the heap (young generation / Eden).
3. Fields set to **default values** (0, null, false).
4. **Instance initialisers and constructor** run (parent constructor first).
5. **Reference** returned and stored in the variable.

13.4 StackOverflowError vs OutOfMemoryError **MUST-KNOW**

	StackOverflowError	OutOfMemoryError
Memory area	Stack	Heap (or Metaspace)
Common cause	Infinite/too-deep recursion	Too many objects kept alive, memory leak, huge collections
Example	<code>void f() { f(); }</code>	Adding objects to a static list forever

```
static int depth(int n) { return depth(n + 1); } // StackOverflowError

List<byte[]> leak = new ArrayList<>();
while (true) leak.add(new byte[1_000_000]); // OutOfMemoryError
```

13.5 Memory Leaks in Java **IMPORTANT**

Interview answer: “Even with garbage collection, Java can have memory leaks when objects are no longer needed but are still referenced, so the GC can’t remove them. Common causes are static collections that keep growing, caches without eviction, listeners that are never removed, unclosed resources, and `ThreadLocal` variables not cleared in thread pools.”

14 Garbage Collection

14.1 What is Garbage Collection? MUST-KNOW

Interview answer: “Garbage collection is the process by which the JVM automatically finds objects in the heap that are no longer reachable from the program and frees their memory. This means developers don’t have to manually free memory like in C++, which prevents dangling pointers and most memory leaks.”

Why Java needs GC: No `free()` / `delete` in Java. Without GC, memory would fill up with unused objects.

14.2 When is an Object Eligible for GC? MUST-KNOW

“An object becomes eligible when it’s **unreachable** — no live thread can reach it through any chain of references from GC roots (local variables on stacks, static fields, active threads, JNI references).”

```
// 1. Nullifying the reference
Student s = new Student(); s = null;

// 2. Reassigning the reference
Student a = new Student(); a = new Student(); // first object eligible

// 3. Object created inside a method – eligible after method returns
void m() { Student t = new Student(); }

// 4. Island of isolation – objects referencing only each other
a.friend = b; b.friend = a; a = null; b = null; // both eligible
```

14.3 System.gc() IMPORTANT

Interview answer: “System.gc() only requests the JVM to run garbage collection — it’s a hint, not a guarantee. The JVM may ignore it. In production code we generally shouldn’t call it.”

14.4 Generational Heap (high level) IMPORTANT

```
Heap
├── Young Generation
│   ├── Eden        ← new objects created here
│   ├── Survivor S0
│   └── Survivor S1  ← objects that survive minor GCs move between these
└── Old (Tenured) Generation ← long-lived objects promoted here
```

- **Minor GC** — cleans the young generation; fast and frequent. Most objects die young.
- **Major / Full GC** — cleans the old generation (or whole heap); slower.
- Objects surviving several minor GCs get **promoted** to old generation.

Basic algorithm — Mark and Sweep (and Compact): Mark all reachable objects starting from GC roots → sweep (free) the unmarked ones → compact to remove fragmentation.

14.5 Stop-the-World IMPORTANT

Interview answer: “Stop-the-world means the JVM pauses all application threads while certain phases of garbage collection run, so that object references don’t change during collection. Modern collectors like G1 and ZGC are designed to keep these pauses very short.”

14.6 Garbage Collectors (names only needed) OPTIONAL

Collector	One-liner
Serial GC	Single thread; small apps
Parallel GC	Multiple threads; focuses on throughput
G1 GC	Default since Java 9; splits heap into regions, predictable pauses
ZGC / Shenandoah	Very low pause times (milliseconds), large heaps

14.7 finalize() IMPORTANT

“`finalize()` was called by the GC before reclaiming an object, but it’s unpredictable (may never run), slows GC, and is **deprecated since Java 9**. Use `try-with-resources` / `close()` instead.”

15 Object Class Methods

15.1 Object Class MUST-KNOW

Interview answer: “Object is the root class of every class in Java — every class directly or indirectly extends it. It provides common methods that every object has, like `toString()`, `equals()`, `hashCode()`, `getClass()`, `clone()`, and the thread methods `wait()`, `notify()` and `notifyAll()`.”

Method	Default behaviour	Usually override?
<code>toString()</code>	<code>ClassName@hexHashCode</code>	Yes — readable output
<code>equals(Object)</code>	Reference comparison (<code>==</code>)	Yes — for content equality
<code>hashCode()</code>	Identity-based int	Yes — whenever <code>equals</code> is overridden
<code>clone()</code>	Shallow copy (needs <code>Cloneable</code>)	Sometimes
<code>getClass()</code>	Runtime class	No (final)
<code>wait/notify/notifyAll</code>	Thread coordination	No (final)
<code>finalize()</code>	Deprecated	No

15.2 `toString()`

```
class Student {
    String name; int age;
    @Override
    public String toString() { return "Student{name=" + name + ", age=" + age + "}"; }
}
System.out.println(new Student()); // calls toString() automatically
```

15.3 `equals()` and `hashCode()` Contract MUST-KNOW

Interview answer: “The contract says: if two objects are equal according to `equals()`, they must have the same `hashCode()`. But if two objects have the same `hashCode()`, they don’t have to be equal — that’s just a collision. So whenever we override `equals()`, we must also override `hashCode()`; otherwise hash-based collections like `HashMap` and `HashSet` will break — they’ll treat equal objects as different because they look in different buckets.”

`equals()` rules: reflexive (`a.equals(a)`), symmetric, transitive, consistent, and `a.equals(null)` is false.

```
public class Employee {
    private final int id;
    private final String email;

    public Employee(int id, String email) { this.id = id; this.email = email; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true; // same reference
        if (o == null || getClass() != o.getClass()) return false;
        Employee e = (Employee) o;
        return id == e.id && Objects.equals(email, e.email);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, email); // use the same fields
    }
}

Set<Employee> set = new HashSet<>();
set.add(new Employee(1, "a@x.com"));
set.add(new Employee(1, "a@x.com"));
System.out.println(set.size()); // 1 (duplicates detected correctly)
```

Scenario	Result in HashSet/HashMap
Override both correctly	Works
Override only equals	Equal objects in different buckets → duplicates allowed, get fails
Override only hashCode	Same bucket, but equals uses == → still duplicates
Same hashCode, not equal	Fine — collision, handled by chaining

15.4 clone() IMPORTANT

Interview answer: “clone() creates a copy of an object. The class must implement the Cloneable marker interface, otherwise it throws CloneNotSupportedException. By default, clone() does a shallow copy — primitive fields are copied, but for object fields only the references are copied, so both objects share the same inner objects. For a deep copy, we have to clone the inner objects too — or better, use a copy constructor.”

Shallow copy	Deep copy
Copies field values; references point to same inner objects	Copies inner objects too; fully independent
Changes to nested objects affect both	Changes are isolated
Default clone()	Custom logic / copy constructor / serialization

15.5 getClass()

```
Object o = "hi";
System.out.println(o.getClass().getName()); // java.lang.String
```

“Returns the runtime class of the object. Used in equals() for exact type checks and in reflection.”

16 Packages & Access Control

16.1 Package IMPORTANT

Interview answer: “A package is a namespace that groups related classes and interfaces together, like a folder. Packages help avoid name conflicts, organise code, and control access through package-level visibility. Java has built-in packages like `java.lang` and `java.util`, and we create our own with the package keyword.”

```
package com.abhinav.bank.service;    // must be first statement

import java.util.List;               // single class
import java.util.*;                 // all classes in java.util (not sub-packages)
import static java.lang.Math.PI;     // static import
import static java.lang.Math.*;

public class AccountService { double area = PI * 2; }
```

Facts:

- `java.lang` is imported automatically (String, Math, Object, System).
- `import java.util.*` does **not** import `java.util.function.*` — sub-packages need their own import.
- Imports don't affect performance or class size — they're only a compile-time convenience.

16.2 Static Import OPTIONAL

“Static import lets us use static members without the class name — `sqrt(x)` instead of `Math.sqrt(x)`. Use sparingly, since it can hurt readability.”

16.3 Default (Package-Private) Access IMPORTANT

“When no access modifier is written, the member is visible only within the same package. It's useful for helper classes that shouldn't be part of the public API.”

(Full access-modifier table is in the OOP section.)

17 Generics

17.1 What are Generics? MUST-KNOW

Interview answer: “Generics let us write classes, interfaces and methods with type parameters, so the same code works with different types while keeping type safety. For example, `List<String>` can only hold Strings, and the compiler catches type errors at compile time instead of runtime. They also remove the need for explicit casting.”

```
// Without generics (pre-Java 5)
List list = new ArrayList();
list.add("hello");
list.add(10); // allowed – no type safety
String s = (String) list.get(1); // ClassCastException at runtime

// With generics
List<String> names = new ArrayList<>();
names.add("hello");
// names.add(10); // compile error
String t = names.get(0); // no cast needed
```

Why generics: compile-time type safety · no casting · reusable code · better readability.

17.2 Generic Class

```
class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}
Box<Integer> b = new Box<>();
b.set(10);
int x = b.get();

class Pair<K, V> { // multiple type parameters
    K key; V value;
    Pair(K k, V v) { key = k; value = v; }
}
```

Naming convention: T Type, E Element, K Key, V Value, N Number, R Result.

17.3 Generic Method

```
public static <T> void printAll(List<T> items) {
    for (T item : items) System.out.println(item);
}

public static <T extends Comparable<T>> T max(T a, T b) { // bounded type
    return a.compareTo(b) > 0 ? a : b;
}
```

17.4 Wildcards IMPORTANT

Wildcard	Meaning	Can read as	Can add
<code><?></code>	Any type (unbounded)	Object	Nothing (except null)
<code><? extends T></code>	T or any subclass (upper bound)	T	Nothing
<code><? super T></code>	T or any superclass (lower bound)	Object	T and its subclasses

```
double sum(List<? extends Number> nums) { // accepts List<Integer>, List<Double>
    double s = 0;
    for (Number n : nums) s += n.doubleValue(); // reading is safe
    return s;
}

void addNumbers(List<? super Integer> list) { // accepts List<Integer>, List<Number>, List<Object>
    list.add(1); list.add(2); // writing Integers is safe
}
```

Interview answer: “Wildcards make generic methods more flexible. `<? extends T>` accepts `T` or its subclasses — we use it when we only read from the collection. `<? super T>` accepts `T` or its superclasses — we use it when we add to the collection. The easy rule is PECS: Producer Extends, Consumer Super.”

Follow-ups:

- *Is `List<Integer>` a subtype of `List<Number>`?* — No. Generics are invariant. That’s why wildcards exist.
- *Type erasure?* — “Generic type information is removed by the compiler after type checking, so at runtime `List<String>` and `List<Integer>` are both just `List`. That’s why we can’t do `new T()` or `instanceof List<String>`.”
- *Can generics use primitives?* — No, `List<int>` is invalid; use `List<Integer>`.

18 Enums & Inner Classes

18.1 Enum IMPORTANT

Interview answer: “An enum is a special class that represents a fixed set of constants, like days of the week or order statuses. Enums are type-safe — you can’t assign an invalid value. Every enum implicitly extends `java.lang.Enum`, so it can’t extend another class, but it can have fields, constructors, methods and implement interfaces. Enum constructors are always private.”

```
enum OrderStatus {
    PLACED("Order placed"), SHIPPED("On the way"), DELIVERED("Delivered");

    private final String label;
    OrderStatus(String label) { this.label = label; } // implicitly private
    public String getLabel() { return label; }
}

OrderStatus s = OrderStatus.SHIPPED;
switch (s) {
    case PLACED -> System.out.println("Wait");
    case SHIPPED -> System.out.println(s.getLabel());
    case DELIVERED -> System.out.println("Done");
}
```

Enum method	Purpose
<code>values()</code>	Array of all constants
<code>valueOf("SHIPPED")</code>	Constant from name (throws <code>IllegalArgumentException</code> if invalid)
<code>name()</code>	Constant's name as String
<code>ordinal()</code>	Position (0-based)
<code>compareTo()</code>	Compare by ordinal

Follow-ups: Enums are the best way to implement a **Singleton** (thread-safe, serialization-safe). Enums can be compared with `==`. `EnumMap` / `EnumSet` are efficient collections for enum keys.

18.2 Inner Classes IMPORTANT

Interview answer: “A nested class is a class defined inside another class. There are four types: a member inner class, which is non-static and tied to an outer object; a static nested class, which doesn’t need an outer object; a local inner class, defined inside a method; and an anonymous inner class, which has no name and is used for one-time implementations.”

```

class Outer {
    private int x = 10;

    class Inner { // 1. Member inner class
        void show() { System.out.println(x); } // can access outer's private members
    }

    static class Nested { // 2. Static nested class
        void show() { System.out.println("static nested"); }
    }

    void method() {
        class Local { // 3. Local inner class
            void show() { System.out.println("local"); }
        }
        new Local().show();
    }
}

Outer o = new Outer();
Outer.Inner in = o.new Inner(); // needs outer object
Outer.Nested n = new Outer.Nested(); // no outer object needed

Runnable r = new Runnable() { // 4. Anonymous inner class
    public void run() { System.out.println("anonymous"); }
};

```

Type	Needs outer object?	Access outer instance members?	Typical use
Member inner	Yes	Yes	Helper tightly bound to outer object
Static nested	No	Only static ones	Builder pattern, Map.Entry, Node in LinkedList
Local inner	Yes (in method)	Yes + effectively final locals	Rare; logic used in one method
Anonymous	—	Yes	One-off implementations (mostly replaced by lambdas)

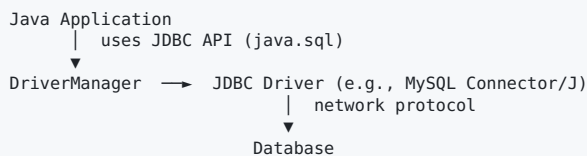
Lambda vs anonymous class: A lambda works only for functional interfaces; `this` in a lambda refers to the enclosing class, while in an anonymous class it refers to the anonymous object itself.

19 JDBC & Database Basics

19.1 What is JDBC? MUST-KNOW

Interview answer: “JDBC — Java Database Connectivity — is a Java API that lets Java applications connect to relational databases and execute SQL queries. It provides interfaces like Connection, Statement, PreparedStatement and ResultSet, and each database vendor provides a driver that implements them. So the same Java code can work with MySQL, PostgreSQL or Oracle just by changing the driver and URL.”

19.2 JDBC Architecture



Driver types (asked occasionally): Type 1 JDBC-ODBC bridge (removed), Type 2 native API, Type 3 network protocol, **Type 4 thin/pure Java driver** (used today — e.g., MySQL Connector/J).

19.3 Core Interfaces

Component	Role
DriverManager	Manages drivers, creates connections
Connection	A session with the database; manages transactions
Statement	Executes static SQL
PreparedStatement	Executes precompiled, parameterised SQL
CallableStatement	Calls stored procedures
ResultSet	Table of results from a SELECT, read row by row with a cursor

19.4 Steps to Connect MUST-KNOW

1. Load the driver (automatic since JDBC 4.0 if the jar is on the classpath; earlier `Class.forName("com.mysql.cj.jdbc.Driver")`).
2. Get a connection: `DriverManager.getConnection(url, user, password)`.
3. Create a Statement / PreparedStatement.
4. Execute the query.
5. Process the ResultSet.
6. Close resources (use try-with-resources).

19.5 executeQuery() vs executeUpdate() vs execute()

Method	Used for	Returns
<code>executeQuery()</code>	SELECT	ResultSet
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE, DDL	int (rows affected)
<code>execute()</code>	Any SQL	boolean (true if a ResultSet is returned)

19.6 Statement vs PreparedStatement MUST-KNOW

	Statement	PreparedStatement
SQL	Static, built by string concatenation	Precompiled with ? placeholders
SQL injection	Vulnerable	Safe (parameters are escaped)
Performance	Compiled every time	Compiled once, reused
Readability	String concatenation mess	Clean <code>setString</code> , <code>setInt</code>
Binary data	Hard	<code>setBlob</code> , <code>setBytes</code>

Interview answer: “PreparedStatement is preferred because it prevents SQL injection — user input is sent as parameters, not as part of the SQL string, so it can’t change the query’s structure. It’s also precompiled, so running the same query multiple times is faster, and the code is cleaner.”

19.7 SQL Injection MUST-KNOW

```
// VULNERABLE
String sql = "SELECT * FROM users WHERE username = '" + user + "' AND password = '" + pass + "'";
// If user enters: ' OR '1'='1' --
// Query becomes: SELECT * FROM users WHERE username = '' OR '1'='1' --' AND ... → logs in!

// SAFE
PreparedStatement ps = con.prepareStatement(
    "SELECT * FROM users WHERE username = ? AND password = ?");
ps.setString(1, user);
ps.setString(2, pass);
```

Interview answer: “SQL injection is an attack where a user enters malicious SQL through an input field, and because the application builds the query by concatenating strings, that input changes the meaning of the query — for example, bypassing a login or deleting data. We prevent it by using PreparedStatement with parameters, validating input, and giving the database user minimum privileges.”

19.8 Basic JDBC CRUD Example

```

import java.sql.*;

public class StudentDao {
    private static final String URL = "jdbc:mysql://localhost:3306/college";
    private static final String USER = "root";
    private static final String PASS = "password";

    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }

    // CREATE
    public int insert(int id, String name, String email) throws SQLException {
        String sql = "INSERT INTO students (id, name, email) VALUES (?, ?, ?)";
        try (Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setString(3, email);
            return ps.executeUpdate(); // rows inserted
        }
    }

    // READ
    public void findAll() throws SQLException {
        String sql = "SELECT id, name, email FROM students";
        try (Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {
            while (rs.next()) { // move cursor row by row
                System.out.println(rs.getInt("id") + " " + rs.getString("name")
                    + " " + rs.getString("email"));
            }
        }
    }

    // UPDATE
    public int updateEmail(int id, String email) throws SQLException {
        String sql = "UPDATE students SET email = ? WHERE id = ?";
        try (Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, email);
            ps.setInt(2, id);
            return ps.executeUpdate();
        }
    }

    // DELETE
    public int delete(int id) throws SQLException {
        String sql = "DELETE FROM students WHERE id = ?";
        try (Connection con = getConnection();
            PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, id);
            return ps.executeUpdate();
        }
    }
}

```

19.9 Transactions: commit and rollback MUST-KNOW

Interview answer: “A transaction is a group of SQL operations that must all succeed or all fail together. By default, JDBC auto-commits every statement. For a transaction, we set autoCommit to false, run our statements, and call commit() if everything succeeds or rollback() if anything fails. A classic example is a bank transfer — debit and credit must both happen, or neither.”

```

try (Connection con = getConnection()) {
    con.setAutoCommit(false);
    try (PreparedStatement debit = con.prepareStatement(
        "UPDATE accounts SET balance = balance - ? WHERE id = ?");
        PreparedStatement credit = con.prepareStatement(
            "UPDATE accounts SET balance = balance + ? WHERE id = ?")) {
        debit.setDouble(1, 500); debit.setInt(2, 1); debit.executeUpdate();
        credit.setDouble(1, 500); credit.setInt(2, 2); credit.executeUpdate();
        con.commit(); // both succeed
    } catch (SQLException e) {
        con.rollback(); // undo both
        throw e;
    }
}

```

ACID (commonly paired): **A**tomicity (all or nothing), **C**onsistency (valid state to valid state), **I**solation (concurrent transactions don't interfere), **D**urability (committed data survives crashes).

19.10 CallableStatement OPTIONAL

```
CallableStatement cs = con.prepareCall("{call get_student_count(?)}");
cs.registerOutParameter(1, Types.INTEGER);
cs.execute();
int count = cs.getInt(1);
```

Follow-ups: Connection pooling (HikariCP) — reuses DB connections instead of opening a new one per request. In Spring Boot we typically use JPA/Hibernate or `JdbcTemplate`, but they're built on JDBC underneath.

20 Common Design & Concept Questions

20.1 Singleton Pattern MUST-KNOW

Interview answer: “Singleton is a design pattern that ensures a class has only one instance throughout the application and provides a global access point to it. We implement it with a private constructor, a private static instance variable, and a public static method that returns that instance. It’s used for things like configuration managers, logging, or connection pools.”

```
// 1. Eager – simple, thread-safe, created at class load
public class AppConfig {
    private static final AppConfig INSTANCE = new AppConfig();
    private AppConfig() { }
    public static AppConfig getInstance() { return INSTANCE; }
}

// 2. Lazy, thread-safe with double-checked locking
public class Logger {
    private static volatile Logger instance;    // volatile is required
    private Logger() { }
    public static Logger getInstance() {
        if (instance == null) {                // 1st check (no lock)
            synchronized (Logger.class) {
                if (instance == null) {        // 2nd check (with lock)
                    instance = new Logger();
                }
            }
        }
        return instance;
    }
}

// 3. Bill Pugh – lazy + thread-safe without synchronization
public class Cache {
    private Cache() { }
    private static class Holder { static final Cache INSTANCE = new Cache(); }
    public static Cache getInstance() { return Holder.INSTANCE; }
}

// 4. Enum – safest (handles serialization & reflection)
public enum Registry { INSTANCE; public void doWork() { } }
```

Follow-ups: Why volatile in double-checked locking? — “Without it, another thread might see a partially constructed object because of instruction reordering.” How can Singleton be broken? — Reflection, serialization, cloning — enum singleton prevents all three. In Spring? — Beans are singleton-scoped by default (one per container).

20.2 Immutable Class MUST-KNOW

Interview answer: “An immutable class is one whose objects can’t be changed after creation — like String or Integer. To make one: declare the class final, make all fields private and final, don’t provide setters, initialise everything in the constructor, and for mutable fields like lists or dates, make defensive copies both in the constructor and in getters.”

```
public final class Employee {
    private final int id;
    private final String name;
    private final List<String> skills;

    public Employee(int id, String name, List<String> skills) {
        this.id = id;
        this.name = name;
        this.skills = new ArrayList<>(skills);    // defensive copy in
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public List<String> getSkills() {
        return Collections.unmodifiableList(skills); // or List.copyOf(skills)
    }
}
```

Benefits: thread-safe by default, safe HashMap keys, can be cached and shared, simpler reasoning. **Java 16+ shortcut:** `record` gives you an immutable data class (but defensive copies of mutable fields are still your job).

20.3 POJO, JavaBean and DTO IMPORTANT

Term	Meaning
POJO (Plain Old Java Object)	A simple Java class not tied to any framework — just fields, constructors, getters/setters
JavaBean	A POJO following conventions: public no-arg constructor, private fields, getters/setters, Serializable
DTO (Data Transfer Object)	An object used to carry data between layers or over the network (e.g., API request/response), without business logic
Entity	A class mapped to a database table (JPA <code>@Entity</code>)

Interview answer (DTO): “A DTO is a simple object used to transfer data between layers — for example, from the service layer to the API response. We use DTOs so we don’t expose our database entities directly, we can send only the fields the client needs, and we keep the API stable even if the database schema changes.”

20.4 SOLID Principles IMPORTANT

Principle	Meaning in one line	Quick example
S — Single Responsibility	A class should have only one reason to change	<code>InvoiceService</code> computes; <code>InvoicePrinter</code> prints
O — Open/Closed	Open for extension, closed for modification	Add <code>UpiPayment</code> implements <code>Payment</code> instead of editing an if-else chain
L — Liskov Substitution	Subclass objects should be usable wherever the parent is expected without breaking behaviour	A <code>Square</code> that breaks <code>Rectangle.setWidth</code> logic violates LSP
I — Interface Segregation	Prefer many small interfaces over one fat interface	<code>Printable</code> , <code>Scannable</code> instead of one <code>Machine</code> with everything
D — Dependency Inversion	Depend on abstractions, not concrete classes	<code>OrderService</code> depends on <code>PaymentGateway</code> interface, not <code>RazorpayGateway</code>

Interview answer: “SOLID is a set of five design principles for writing maintainable object-oriented code: Single Responsibility — a class should do one thing; Open-Closed — we should extend behaviour without modifying existing code; Liskov Substitution — a child class should be usable in place of its parent; Interface Segregation — don’t force classes to implement methods they don’t need; and Dependency Inversion — depend on interfaces, not concrete implementations.”

20.5 Dependency Injection IMPORTANT

Interview answer: “Dependency Injection is a design principle where an object receives its dependencies from outside instead of creating them itself. This makes the code loosely coupled and easier to test, because we can swap in a different implementation or a mock. In Spring, the IoC container creates objects and injects them — usually through constructor injection.”

```
// Tight coupling – creates its own dependency
class OrderService {
    private final EmailSender sender = new SmtEmailSender();
}

// Dependency Injection – dependency passed in
class OrderService {
    private final EmailSender sender;
    OrderService(EmailSender sender) { this.sender = sender; } // constructor injection
}

// In tests: new OrderService(new FakeEmailSender());
```

20.6 Composition over Inheritance IMPORTANT

Interview answer: “Composition over inheritance means preferring to build classes by combining objects — HAS-A — rather than extending classes — IS-A. Inheritance creates tight coupling: a change in the parent can break every child, and Java only allows one parent. With composition, we can swap behaviours at runtime and keep classes independent.”

```
// Inheritance: every Duck subclass inherits fly(), even RubberDuck
// Composition:
interface FlyBehavior { void fly(); }
class Duck {
    private FlyBehavior flyBehavior; // HAS-A
    Duck(FlyBehavior f) { this.flyBehavior = f; }
    void performFly() { flyBehavior.fly(); }
}
```

20.7 Coupling and Cohesion IMPORTANT

	Coupling	Cohesion
Meaning	How much one class depends on another	How closely the responsibilities inside one class are related
Goal	Low (loose) coupling	High cohesion
Achieved by	Interfaces, DI, small public APIs	Single Responsibility, focused classes

Interview answer: “Coupling is the degree of dependency between classes — we want it low so changing one class doesn’t break others. Cohesion is how focused a class is on a single purpose — we want it high. Good design is low coupling and high cohesion.”

21 Java Interview Trick Questions

Short, confident answers. Say the **Yes/No first**, then one or two lines of reasoning.

21.1 Main Method & Basics

- 1. Can we overload the main() method?** Yes. We can have multiple `main` methods with different parameters, but the JVM only calls `public static void main(String[] args)` as the entry point. Others run only if we call them.
- 2. Can main() be private, protected or non-static?** It compiles, but with the classic launcher the JVM can't start the program — it reports that the main method isn't found. That's the answer interviewers expect. (Bonus: Java 25 finalised "instance main methods", which relaxes the static/public rule for simple programs — but a private main still isn't launched.)
- 3. Can we write `static public void main(String... args)`?** Yes. Modifier order doesn't matter, and varargs `String...` is equivalent to `String[]`.
- 4. Can a Java program run without main()?** Not since Java 7. Earlier, a static block could execute before the JVM complained; now the JVM checks for main first.
- 5. Can a .java file have multiple classes?** Yes, but only one can be `public`, and the file name must match that public class.
- 6. Is `null` a keyword?** No. `null`, `true` and `false` are literals, not keywords, but you still can't use them as identifiers.
- 7. What's the default value of a local variable?** There is none. Local variables must be initialised before use, otherwise compilation fails. Only instance and static variables get default values.
- 8. What does `System.out.println(1 + 2 + "3" + 4 + 5)` print?** `3345`. Evaluation is left to right: `1+2 = 3`, then `"3"+"3"` becomes a String, and everything after is concatenation.
- 9. What is `(char)('a' + 1)` and what is `'a' + 1`?** `'b'` and `98`. Arithmetic on char promotes it to int.
- 10. Can `0.1 + 0.2 == 0.3` be true?** No — it's false, because floating-point can't represent 0.1 exactly. Use `BigDecimal` or compare with a tolerance.

21.2 OOP & Constructors

- 11. Can a constructor be final, static or abstract?** No. Constructors aren't inherited, so `final` and `abstract` are meaningless; `static` is invalid because a constructor initialises an instance.
- 12. Can a constructor be private?** Yes. It prevents object creation from outside — used in Singleton, utility classes and factory methods.
- 13. Can an abstract class have a constructor?** Yes. It runs when a subclass object is created, via `super()`, to initialise common fields.
- 14. Can an interface have a constructor?** No. Interfaces have no instance state to initialise.
- 15. Can we create an object of an abstract class?** No, not directly. But we can create an anonymous subclass: `new Shape() { double area() { return 0; } }` — that's an object of the subclass.
- 16. Can an abstract class have no abstract methods?** Yes. It still can't be instantiated. Example: `HttpServlet` style base classes.
- 17. Can a class be both abstract and final?** No. Abstract needs a subclass; final forbids one. Compile error.
- 18. Can an interface extend another interface?** Yes — and it can extend **multiple** interfaces: `interface C extends A, B {}`.
- 19. Can a class implement multiple interfaces?** Yes. That's how Java supports multiple inheritance of type.
- 20. Can an interface implement another interface?** No. Interfaces `extend` interfaces; only classes `implement`.
- 21. Can we declare a class as private or protected?** Not a top-level class — only `public` or default. Nested classes can be private or protected.

22. Why doesn't Java support multiple inheritance with classes? To avoid the diamond problem — ambiguity when two parents have the same method. Java allows multiple inheritance through interfaces, and if two default methods clash, the class must override and resolve it.

23. Can we call a parent's constructor and `this()` both in one constructor? No. Both `this()` and `super()` must be the first statement, so only one can be used.

24. What is a copy constructor? Does Java have one by default? A constructor that takes an object of the same class and copies its fields. Java doesn't provide one automatically — we write it ourselves.

21.3 Overloading & Overriding

25. Can we override static methods? No. If a subclass defines the same static method, it's **method hiding** — the version called depends on the reference type, not the object.

26. Can we overload static methods? Yes. Overloading only needs a different parameter list.

27. Can we override private methods? No. Private methods aren't visible to the subclass. A method with the same name in the child is a brand-new method.

28. Can we override final methods? No — compile error. That's the purpose of `final` on a method.

29. Can we overload a method by changing only the return type? No. The compiler can't decide which one to call from the arguments alone.

30. Can we reduce visibility when overriding (`public` → `protected`)? No. We can keep it the same or make it wider. Reducing would break code that uses the parent reference.

31. Can an overriding method throw a broader checked exception? No. It can throw the same, a narrower subclass, none, or any unchecked exception.

32. What is a covariant return type? When an overriding method returns a subclass of the parent method's return type — e.g., parent returns `Animal`, child returns `Dog`. Allowed since Java 5.

33. Are fields (variables) overridden? No. Fields are hidden, not overridden. `Parent p = new Child(); p.x` gives the parent's field.

34. Can we override a method and make it static? No. Instance → static (or static → instance) is a compile error.

21.4 Keywords

35. Difference between `final`, `finally` and `finalize`? `final` restricts — constant variable, non-overridable method, non-inheritable class. `finally` is a block that always runs after try/catch for cleanup. `finalize()` was a GC callback on `Object` — deprecated since Java 9.

36. Can a final variable be initialised later? Yes, if it's a blank final — once, in the constructor (instance) or static block (static). After that it can't change.

37. If a list is final, can we add elements? Yes. `final` locks the reference, not the object's contents. We can't reassign it to a new list.

38. Can we use `this` or `super` in a static method? No. Static methods don't belong to any object.

39. Can a static method access instance variables? Not directly — only through an object reference.

40. Difference between `throw` and `throws`? `throw` actually throws one exception object inside a method. `throws` declares in the signature which checked exceptions the method might throw.

41. Does `volatile` make `count++` thread-safe? No. `volatile` ensures visibility, not atomicity. `count++` is read-modify-write, so use `AtomicInteger` or `synchronized`.

21.5 Strings

42. Why is `String` immutable? For String pool safety (shared literals), security (URLs, credentials can't be changed after validation), thread safety, and cached `hashCode` — which makes it a reliable `HashMap` key.

- 43. How many objects are created by `String s = new String("abc");`?** Up to two: one `"abc"` in the pool (if not already there) and one new object in the heap.
- 44. What does `"abc" == "abc"` return? And `new String("abc") == "abc"`?** `true` (same pooled literal) and `false` (heap object vs pool object).
- 45. Is `String` thread-safe?** Yes, because it's immutable.
- 46. Why use `char[]` instead of `String` for passwords?** A `String` stays in memory (and possibly the pool) until GC; a `char[]` can be wiped immediately after use, and it won't be accidentally printed in logs.
- 47. Can we extend the `String` class?** No, `String` is `final`.

21.6 Collections & HashMap

- 48. Why does `HashMap` allow one null key but `Hashtable` doesn't?** `HashMap` special-cases null — it treats its hash as 0 and puts it in bucket 0. `Hashtable` calls `key.hashCode()` directly, which would throw `NPE`. Keys are unique, so `HashMap` has at most one null key.
- 49. Why doesn't `ConcurrentHashMap` allow null?** In a concurrent map, `get(key)` returning null would be ambiguous — you couldn't tell "absent" from "mapped to null", and you can't safely check `containsKey` separately because another thread may change it in between.
- 50. What happens if we put the same key twice in a `HashMap`?** The old value is replaced, and `put` returns the old value.
- 51. What if two keys have the same `hashCode`?** They go to the same bucket (collision). `HashMap` chains them and uses `equals()` to tell them apart. Both entries are stored.
- 52. What if we override `equals()` but not `hashCode()`?** Equal objects may get different hash codes → different buckets → `HashMap/HashSet` won't find them and will allow duplicates. This breaks the `equals-hashCode` contract.
- 53. Can we use a custom object as a `HashMap` key?** Yes — override `equals()` and `hashCode()` using the same fields, and ideally make it immutable.
- 54. Can we add elements to a list while iterating over it?** Not with a for-each loop — `ConcurrentModificationException`. Use `ListIterator.add()`, iterate over a copy, or use `CopyOnWriteArrayList`.
- 55. Does `Arrays.asList()` return an `ArrayList`?** It returns a fixed-size list (an internal `Arrays$ArrayList`), not `java.util.ArrayList`. `add/remove` throw `UnsupportedOperationException`; `set` works.
- 56. Can a `TreeSet` store null?** No — it needs to compare elements, and comparing with null throws `NPE`.
- 57. Can we store duplicates in a `Set` if the objects don't override `equals/hashCode`?** Yes — two logically-equal objects will be treated as different because default `equals` compares references.

21.7 Exceptions

- 58. Will `finally` execute if there's a return in `try`?** Yes. `finally` runs before the method actually returns. It won't run only on `System.exit()` or a JVM crash.
- 59. Can we have `try` without `catch`?** Yes, if it has `finally`, or if it's `try-with-resources`.
- 60. Can we catch an `Error`?** Technically yes (it's a `Throwable`), but we shouldn't — errors like `OutOfMemoryError` usually mean the JVM can't recover.
- 61. Is it OK to catch `Exception` before `IOException`?** No — compile error, because `IOException` is already caught by the broader handler above it. Specific catches come first.
- 62. What happens to an uncaught exception in a thread?** That thread terminates and the stack trace is printed; other threads keep running.

21.8 Threads

- 63. Can we call `start()` twice on the same thread?** No — the second call throws `IllegalThreadStateException`.

64. What if we call `run()` instead of `start()`? No new thread is created; `run()` executes on the current thread like a normal method.

65. Why are `wait()`, `notify()` in `Object` and not `Thread`? Because they operate on an object's monitor lock, and any object can be used as a lock.

66. Can we call `wait()` outside a synchronized block? It compiles, but throws `IllegalMonitorStateException` at runtime, since the thread doesn't own the lock.

21.9 Memory & Misc

67. Can we force garbage collection? No. `System.gc()` is only a request; the JVM decides.

68. Is Java pass-by-value or pass-by-reference? Always pass-by-value. For objects, the value passed is a copy of the reference.

69. What is the size of an empty `ArrayList`'s internal array? With the no-arg constructor it starts as an empty array, and becomes 10 on the first add.

70. Is `int[]` a primitive or an object? An object. All arrays are objects stored in the heap, even arrays of primitives.

71. What happens with `Integer a = 128, b = 128; a == b`? `false`. Integer caching covers -128 to 127 only; outside that, different objects are created. Use `equals()`.

72. What is the output of `Math.round(-2.5)`? `-2`. `Math.round` adds 0.5 and takes the floor: `floor(-2.0) = -2`.

73. Can an enum extend a class? No — every enum implicitly extends `java.lang.Enum`. It can implement interfaces.

74. What is a marker interface? Give examples. An interface with no methods that tags a class with special behaviour: `Serializable`, `Cloneable`, `RandomAccess`. Annotations are the modern alternative.

75. Can we declare a variable inside an interface that isn't a constant? No. All interface fields are implicitly `public static final`.

22 50 Most Repeated Java Fresher Coding Questions

These are the problems that keep appearing in fresher online tests and technical rounds at service and product companies. Each one follows the same format so you can revise quickly.

How to talk while coding (works for every problem):

1. Repeat the problem and confirm edge cases ("Can the input be empty? Negative?").
2. State the brute-force idea in one line, then the better approach.
3. Code while narrating *why*, not *what*.
4. Dry-run with the example, then state time and space complexity.

22.1 Part A — Basic Programming

1. Reverse a Number **MUST-KNOW**

Problem: Reverse the digits of an integer. **Input:** 12345 **Output:** 54321

Approach: Repeatedly take the last digit with `% 10`, append it to the result, and drop it with `/ 10`.

Steps: 1) `rev = 0`. 2) While `n != 0`: `digit = n % 10`, `rev = rev * 10 + digit`, `n /= 10`. 3) Return `rev`.

```
static int reverseNumber(int n) {
    int rev = 0;
    while (n != 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    return rev;
    // works for negatives too: -123 → -321
}
```

Time: $O(d)$, d = number of digits · **Space:** $O(1)$

Follow-up: Handle overflow (e.g., reversing 1534236469) — use `long` for `rev` and check against `Integer.MAX_VALUE`.

Say while coding: "n mod 10 gives me the last digit, I shift the result left by multiplying by 10 and add the digit, then drop the last digit by dividing by 10."

2. Palindrome Number **MUST-KNOW**

Problem: Check if a number reads the same backward. **Input:** 121 **Output:** true; 123 → false

Approach: Reverse the number and compare with the original.

Steps: 1) Negative → false. 2) Reverse a copy. 3) Compare.

```
static boolean isPalindrome(int n) {
    if (n < 0) return false;
    int original = n, rev = 0;
    while (n > 0) {
        rev = rev * 10 + n % 10;
        n /= 10;
    }
    return original == rev;
}
```

Time: $O(d)$ · **Space:** $O(1)$

Follow-up: Do it without reversing fully — reverse only half the digits and compare halves.

Say while coding: "I keep a copy of the original because the loop destroys n, then compare the reversed value with it."

3. Armstrong Number **MUST-KNOW**

Problem: A number is Armstrong if the sum of its digits each raised to the power of the number of digits equals the number.

Input: 153 **Output:** true ($1^3 + 5^3 + 3^3 = 153$)

Approach: Count digits, then sum each digit raised to that power.

Steps: 1) Count digits k . 2) For each digit, add `digitk`. 3) Compare with original.

```

static boolean isArmstrong(int n) {
    int k = String.valueOf(n).length();
    int sum = 0, temp = n;
    while (temp > 0) {
        int d = temp % 10;
        sum += (int) Math.pow(d, k);
        temp /= 10;
    }
    return sum == n;
}

```

Time: $O(d)$ · **Space:** $O(1)$

Follow-up: Print all Armstrong numbers between 1 and 1000 (1-9, 153, 370, 371, 407).

Say while coding: “The power depends on the number of digits, so I compute that first, then add each digit to that power.”

4. Prime Number MUST-KNOW

Problem: Check if `n` is prime. **Input:** 29 **Output:** true

Approach: A number is prime if no number from 2 to $\sqrt{n}$ divides it. Checking up to $\sqrt{n}$ is enough because factors come in pairs.

Steps: 1) `n < 2` → false. 2) Loop `i` from 2 while `i * i <= n`. 3) If `n % i == 0` → false. 4) Return true.

```

static boolean isPrime(int n) {
    if (n < 2) return false;
    for (int i = 2; (long) i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}

// Follow-up: all primes up to N – Sieve of Eratosthenes,  $O(N \log \log N)$ 
static List<Integer> primesUpTo(int n) {
    boolean[] composite = new boolean[n + 1];
    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++) {
        if (!composite[i]) {
            primes.add(i);
            for (long j = (long) i * i; j <= n; j += i) composite[(int) j] = true;
        }
    }
    return primes;
}

```

Time: $O(\sqrt{n})$ · **Space:** $O(1)$

Follow-up: Print primes in a range → Sieve (above). Why $\sqrt{n}$? “If $n = a \times b$ and both were greater than $\sqrt{n}$, the product would exceed n .”

Say while coding: “I only need to check divisors up to the square root, because any factor above it pairs with one below it.”

5. Fibonacci Series (iterative, recursive, memoized) MUST-KNOW

Problem: Print the first `n` Fibonacci numbers, and find the `n`th term. **Input:** `n = 7` **Output:** 0 1 1 2 3 5 8

Approach: Each term is the sum of the previous two. Iteration keeps just two variables.

Steps: 1) `a = 0`, `b = 1`. 2) Print `a`, compute `next = a + b`, shift `a = b`, `b = next`. 3) Repeat `n` times.

```

static void printFibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        System.out.print(a + " ");
        int next = a + b;
        a = b;
        b = next;
    }
}

// Recursive - simple but O(2^n)
static int fibRec(int n) {
    if (n <= 1) return n;
    return fibRec(n - 1) + fibRec(n - 2);
}

// Memoized - O(n)
static long fibMemo(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

```

Time: Iterative $O(n)$; plain recursion $O(2^n)$; memoized $O(n)$ · **Space:** Iterative $O(1)$; recursion $O(n)$ stack

Follow-up: “Why is recursive Fibonacci slow?” — It recomputes the same subproblems repeatedly; memoization stores results so each is computed once.

Say while coding: “I only need the last two values at any time, so I keep two variables and slide them forward.”

6. Factorial (iterative and recursive) MUST-KNOW

Problem: Compute $n!$. **Input:** 5 **Output:** 120

Approach: Multiply $1 \times 2 \times \dots \times n$. Recursively: $n! = n \times (n-1)!$ with base case $0! = 1$.

Steps: 1) `result = 1`. 2) Multiply by each `i` from 2 to `n`.

```

static long factorial(int n) {
    long result = 1;
    for (int i = 2; i <= n; i++) result *= i;
    return result;
}

static long factorialRec(int n) {
    if (n <= 1) return 1; // base case
    return n * factorialRec(n - 1); // recursive case
}

```

Time: $O(n)$ · **Space:** Iterative $O(1)$; recursive $O(n)$ stack

Follow-up: `long` overflows after 20! — use `BigInteger` for larger values. What happens without a base case? `StackOverflowError`.

Say while coding: “I use `long` because factorials grow very fast — `int` overflows at 13 factorial.”

7. GCD and LCM MUST-KNOW

Problem: Find the Greatest Common Divisor and Least Common Multiple of two numbers. **Input:** 12, 18 **Output:** GCD = 6, LCM = 36

Approach: Euclidean algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ until $b = 0$. Then $\text{lcm} = a / \text{gcd} \times b$.

Steps: 1) While `b != 0`: `temp = b`; `b = a % b`; `a = temp`. 2) `a` is the GCD. 3) `LCM = a / gcd * b` (divide first to avoid overflow).

```

static int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return Math.abs(a);
}

static long lcm(int a, int b) {
    return (long) a / gcd(a, b) * b;
}

```

Time: $O(\log(\min(a, b)))$ · **Space:** $O(1)$

Follow-up: GCD of an array — fold: `g = gcd(g, arr[i])`. Recursive version: `return b == 0 ? a : gcd(b, a % b)`;

Say while coding: “Euclid’s algorithm works because any common divisor of `a` and `b` also divides `a mod b`, so the problem

keeps shrinking.”

8. Sum of Digits **MUST-KNOW**

Problem: Add all digits of a number. **Input:** 4567 **Output:** 22

Approach: Extract each digit with `% 10` and add it.

Steps: 1) `sum = 0`. 2) While `n > 0`: `sum += n % 10`; `n /= 10`.

```
static int sumOfDigits(int n) {
    n = Math.abs(n);
    int sum = 0;
    while (n > 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
```

Time: $O(d)$ · **Space:** $O(1)$

Follow-up: Repeated digit sum until single digit (digital root) — `n == 0 ? 0 : 1 + (n - 1) % 9`.

Say while coding: “Same digit-extraction pattern as reversing — mod 10 for the digit, divide by 10 to move on.”

9. Count Digits **IMPORTANT**

Problem: Count the digits in an integer. **Input:** 98765 **Output:** 5

Approach: Divide by 10 until 0, counting steps. Handle 0 separately.

Steps: 1) If `n == 0` return 1. 2) Loop dividing by 10, increment count.

```
static int countDigits(int n) {
    if (n == 0) return 1;
    n = Math.abs(n);
    int count = 0;
    while (n > 0) {
        count++;
        n /= 10;
    }
    return count;
}
```

Time: $O(d)$ · **Space:** $O(1)$

Follow-up: Other ways — `String.valueOf(Math.abs(n)).length()` or `(int) Math.log10(n) + 1` for positive `n`.

Say while coding: “Each division by 10 removes one digit, so the number of divisions is the digit count. Zero is a special case — it has one digit.”

22.2 Part B — Strings

10. Reverse a String **MUST-KNOW**

Problem: Reverse a string without using `StringBuilder.reverse()`. **Input:** "hello" **Output:** "olleh"

Approach: Convert to char array and swap characters from both ends moving inward (two pointers).

Steps: 1) `left = 0`, `right = n-1`. 2) Swap, move pointers inward. 3) Stop when they meet.

```
static String reverseString(String s) {
    char[] ch = s.toCharArray();
    int left = 0, right = ch.length - 1;
    while (left < right) {
        char temp = ch[left];
        ch[left++] = ch[right];
        ch[right--] = temp;
    }
    return new String(ch);
}
// Built-in: new StringBuilder(s).reverse().toString();
```

Time: $O(n)$ · **Space:** $O(n)$ for the char array (strings are immutable)

Follow-up: Reverse using recursion: `s.isEmpty() ? s : reverse(s.substring(1)) + s.charAt(0)` — $O(n^2)$ due to string creation.

Say while coding: “Strings are immutable, so I work on a char array and swap from both ends — each swap fixes two

positions.”

11. Palindrome String **MUST-KNOW**

Problem: Check if a string is a palindrome. **Input:** "madam" **Output:** true

Approach: Two pointers comparing characters from both ends.

Steps: 1) $i = 0$, $j = n - 1$. 2) If $s[i] \neq s[j]$ return false. 3) Move inward.

```
static boolean isPalindrome(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        if (s.charAt(i++) != s.charAt(j--)) return false;
    }
    return true;
}

// Follow-up: ignore case and non-alphanumeric ("A man, a plan, a canal: Panama")
static boolean isPalindromeClean(String s) {
    int i = 0, j = s.length() - 1;
    while (i < j) {
        while (i < j && !Character.isLetterOrDigit(s.charAt(i))) i++;
        while (i < j && !Character.isLetterOrDigit(s.charAt(j))) j--;
        if (Character.toLowerCase(s.charAt(i++)) != Character.toLowerCase(s.charAt(j--)))
            return false;
    }
    return true;
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Ignore spaces/case (above). Longest palindromic substring (expand around center — harder).

Say while coding: “I don’t need to build the reversed string — comparing from both ends uses constant space and can exit early.”

12. Count Vowels and Consonants **MUST-KNOW**

Problem: Count vowels and consonants in a string. **Input:** "Hello World" **Output:** Vowels = 3, Consonants = 7

Approach: For each letter, check if it’s in "aeiou"; otherwise it’s a consonant. Skip non-letters.

Steps: 1) Lowercase the string. 2) Loop; if letter → check vowel set.

```
static int[] countVowelsConsonants(String s) {
    int vowels = 0, consonants = 0;
    for (char c : s.toLowerCase().toCharArray()) {
        if (c >= 'a' && c <= 'z') {
            if ("aeiou".indexOf(c) != -1) vowels++;
            else consonants++;
        }
    }
    return new int[]{vowels, consonants};
}
```

Time: $O(n)$ · **Space:** $O(1)$ (plus the lowercase copy)

Follow-up: Count digits, spaces and special characters too — use `Character.isDigit`, `Character.isWhitespace`.

Say while coding: “I only count letters — spaces and punctuation are neither vowels nor consonants.”

13. Character Frequency **MUST-KNOW**

Problem: Count occurrences of each character. **Input:** "banana" **Output:** b=1, a=3, n=2

Approach: Use a `LinkedHashMap` (keeps first-appearance order) and `merge/getOrDefault`.

Steps: 1) Loop over chars. 2) `map.put(c, map.getOrDefault(c, 0) + 1)`.

```

static Map<Character, Integer> charFrequency(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getDefault(c, 0) + 1); // or freq.merge(c, 1, Integer::sum)
    }
    return freq;
}

// Only lowercase a-z: faster with an array
static int[] charFrequencyArray(String s) {
    int[] count = new int[26];
    for (char c : s.toCharArray()) count[c - 'a']++;
    return count;
}

```

Time: $O(n)$ · **Space:** $O(k)$, k = distinct characters

Follow-up: Stream version: `s.chars().mapToObj(c -> (char) c).collect(groupingBy(c -> c, LinkedHashMap::new, counting()))`. Find the most frequent character.

Say while coding: "I use LinkedHashMap so the output follows the order characters first appear. For only lowercase letters, an int array of 26 is faster."

14. First Non-Repeating Character MUST-KNOW

Problem: Find the first character that appears exactly once. **Input:** "swiss" **Output:** 'w'

Approach: Two passes: count frequencies, then scan the string in order and return the first with count 1.

Steps: 1) Build frequency map. 2) Loop the string again; first char with count 1 wins. 3) None → return a sentinel.

```

static char firstNonRepeating(String s) {
    Map<Character, Integer> freq = new HashMap<>();
    for (char c : s.toCharArray()) freq.merge(c, 1, Integer::sum);
    for (char c : s.toCharArray()) {
        if (freq.get(c) == 1) return c;
    }
    return '_'; // none found
}

```

Time: $O(n)$ · **Space:** $O(k)$

Follow-up: First repeating character (use a HashSet — first add that returns false). Return the index instead of the character.

Say while coding: "The second pass goes over the original string, not the map, so I preserve the original order."

15. Remove Duplicate Characters MUST-KNOW

Problem: Remove duplicate characters, keeping the first occurrence. **Input:** "programming" **Output:** "progamin"

Approach: Track seen characters in a set; append a character only the first time.

Steps: 1) `seen = new HashSet`. 2) For each char, if `seen.add(c)` is true → append.

```

static String removeDuplicateChars(String s) {
    Set<Character> seen = new HashSet<>();
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (seen.add(c)) sb.append(c); // add returns false if already present
    }
    return sb.toString();
}

```

Time: $O(n)$ · **Space:** $O(k)$

Follow-up: Without extra collections — `boolean[256]` for ASCII. Using streams: `s.chars().distinct()`.

Say while coding: "HashSet.add returns false if the element already exists, so I can check and insert in one step."

16. Anagram Check MUST-KNOW

Problem: Check if two strings are anagrams (same characters, same counts). **Input:** "listen", "silent" **Output:** true

Approach: Count characters: increment for the first string, decrement for the second; all counts must end at zero.

Steps: 1) Lengths differ → false. 2) `count[26] : ++` for `s1`, `--` for `s2`. 3) Any non-zero → false.

```

static boolean isAnagram(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        count[a.charAt(i) - 'a']++;
        count[b.charAt(i) - 'a']--;
    }
    for (int c : count) if (c != 0) return false;
    return true;
}

// Alternative: sort both - O(n log n)
static boolean isAnagramSort(String a, String b) {
    char[] x = a.toCharArray(), y = b.toCharArray();
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}

```

Time: $O(n)$ with counting; $O(n \log n)$ with sorting · **Space:** $O(1)$ (fixed 26)

Follow-up: Handle uppercase and spaces — normalise with `toLowerCase().replaceAll("\\s", "")`. Group anagrams from a list (sort each word as a map key).

Say while coding: “One array, increment for the first word and decrement for the second — if every count returns to zero, they have the same letters.”

17. String Compression IMPORTANT

Problem: Compress consecutive repeated characters. **Input:** "aaabbc" **Output:** "a3b2c1"

Approach: Walk the string counting runs of the same character; when the run ends, append char + count.

Steps: 1) `count = 1`. 2) If next char is same → `count++`; else append and reset. 3) Handle last run.

```

static String compress(String s) {
    if (s.isEmpty()) return s;
    StringBuilder sb = new StringBuilder();
    int count = 1;
    for (int i = 1; i <= s.length(); i++) {
        if (i < s.length() && s.charAt(i) == s.charAt(i - 1)) {
            count++;
        } else {
            sb.append(s.charAt(i - 1)).append(count);
            count = 1;
        }
    }
    return sb.toString();
}

```

Time: $O(n)$ · **Space:** $O(n)$ for output

Follow-up: Return the original if compression isn't shorter. Omit the count when it's 1 ("a3b2c").

Say while coding: “Looping to `i` equals length lets me flush the last run inside the same loop instead of duplicating code after it.”

18. Count Words MUST-KNOW

Problem: Count the words in a sentence (handle multiple spaces). **Input:** " Java is fun " **Output:** 3

Approach: Trim, then split on one-or-more whitespace `\\s+`.

Steps: 1) Trim. 2) Empty → 0. 3) `split("\\s+").length`.

```

static int countWords(String s) {
    s = s.trim();
    if (s.isEmpty()) return 0;
    return s.split("\\s+").length;
}

// Without split: count transitions from space to non-space
static int countWordsManual(String s) {
    int count = 0;
    boolean inWord = false;
    for (char c : s.toCharArray()) {
        if (Character.isWhitespace(c)) inWord = false;
        else if (!inWord) { inWord = true; count++; }
    }
    return count;
}

```

Time: $O(n)$ · **Space:** $O(n)$ with split; $O(1)$ manual

Follow-up: Frequency of each word — split, then `HashMap<String, Integer>`.

Say while coding: “I trim first because a leading space would create an empty first token, and I split on `\s+` so multiple spaces count as one separator.”

19. Reverse Words in a Sentence MUST-KNOW

Problem: Reverse the order of words. **Input:** “Java is fun” **Output:** “fun is Java”

Approach: Split into words, then append from last to first.

Steps: 1) `trim().split("\\s+")`. 2) Loop backwards appending words with a space. 3) Return.

```
static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (int i = words.length - 1; i >= 0; i--) {
        sb.append(words[i]);
        if (i > 0) sb.append(' ');
    }
    return sb.toString();
}

// Variant: reverse each word in place → "avaJ si nuf"
static String reverseEachWord(String s) {
    StringBuilder sb = new StringBuilder();
    for (String w : s.trim().split("\\s+")) {
        sb.append(new StringBuilder(w).reverse()).append(' ');
    }
    return sb.toString().trim();
}
```

Time: $O(n)$ · **Space:** $O(n)$

Follow-up: Do it in place on a char array — reverse the whole array, then reverse each word.

Say while coding: “Splitting on `\s+` handles extra spaces, and I only add a space between words, not after the last one.”

22.3 Part C — Arrays

20. Find Maximum and Minimum MUST-KNOW

Problem: Find the largest and smallest elements. **Input:** [3, 9, 1, 7] **Output:** max = 9, min = 1

Approach: One pass, starting with the first element as both max and min.

Steps: 1) `max = min = arr[0]`. 2) Update in a single loop.

```
static int[] findMaxMin(int[] arr) {
    int max = arr[0], min = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) max = arr[i];
        if (arr[i] < min) min = arr[i];
    }
    return new int[]{max, min};
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Why start with `arr[0]` and not 0? — Because all elements might be negative. Empty array? Throw or return a sentinel.

Say while coding: “I initialise with the first element instead of zero so it works for negative numbers.”

21. Second Largest Element MUST-KNOW

Problem: Find the second largest distinct element. **Input:** [12, 35, 1, 10, 34, 35] **Output:** 34

Approach: Track `first` and `second` in one pass; skip values equal to `first`.

Steps: 1) `first = second = MIN`. 2) If `x > first`: `second = first`; `first = x`. 3) Else if `x > second && x != first`: `second = x`.

```

static int secondLargest(int[] arr) {
    int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int x : arr) {
        if (x > first) {
            second = first;
            first = x;
        } else if (x > second && x != first) {
            second = x;
        }
    }
    return second; // Integer.MIN_VALUE means no second largest exists
}

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Why not sort? — Sorting is $O(n \log n)$; this is $O(n)$. Kth largest → min-heap of size k.

Say while coding: “When I find a new maximum, the old maximum becomes the second largest. The x not equal to first check handles duplicates of the max.”

22. Reverse an Array MUST-KNOW

Problem: Reverse an array in place. **Input:** [1, 2, 3, 4, 5] **Output:** [5, 4, 3, 2, 1]

Approach: Two pointers swapping from both ends.

Steps: 1) $i = 0$, $j = n-1$. 2) Swap and move inward until $i \geq j$.

```

static void reverseArray(int[] arr) {
    int i = 0, j = arr.length - 1;
    while (i < j) {
        int temp = arr[i];
        arr[i++] = arr[j];
        arr[j--] = temp;
    }
}

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Reverse only a subarray from index l to r (same logic — used in array rotation).

Say while coding: “Only $n/2$ swaps are needed because each swap places two elements.”

23. Sort an Array of 0s, 1s and 2s IMPORTANT

Problem: Sort an array containing only 0, 1 and 2 in one pass (Dutch National Flag). **Input:** [2, 0, 2, 1, 1, 0] **Output:** [0, 0, 1, 1, 2, 2]

Approach: Three pointers: `low` (next position for 0), `mid` (current), `high` (next position for 2).

Steps: 1) If `arr[mid] == 0` → swap with `low`, `low++`, `mid++`. 2) If `1` → `mid++`. 3) If `2` → swap with `high`, `high--` (don't move `mid`).

```

static void sort012(int[] arr) {
    int low = 0, mid = 0, high = arr.length - 1;
    while (mid <= high) {
        if (arr[mid] == 0) swap(arr, low++, mid++);
        else if (arr[mid] == 1) mid++;
        else swap(arr, mid, high--);
    }
}

static void swap(int[] a, int i, int j) {
    int t = a[i]; a[i] = a[j]; a[j] = t;
}

// General sorting: Arrays.sort(arr) - dual-pivot quicksort for primitives,  $O(n \log n)$ 

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Simpler approach — count 0s, 1s, 2s and overwrite (two passes). Why don't we increment `mid` after swapping with `high`? — The swapped-in element hasn't been checked yet.

Say while coding: “Everything before `low` is 0, everything after `high` is 2, and `mid` scans the unknown region.”

24. Remove Duplicates from Sorted Array MUST-KNOW

Problem: Remove duplicates in place and return the new length. **Input:** [1, 1, 2, 2, 3] **Output:** 3 → [1, 2, 3]

Approach: Two pointers — `k` marks where the next unique element goes.

Steps: 1) $k = 1$. 2) For i from 1: if `arr[i] != arr[k-1]` → `arr[k++] = arr[i]`.

```

static int removeDuplicatesSorted(int[] arr) {
    if (arr.length == 0) return 0;
    int k = 1;
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] != arr[k - 1]) arr[k++] = arr[i];
    }
    return k; // first k elements are unique
}

// Unsorted array: use LinkedHashSet to keep order
static int[] removeDuplicatesUnsorted(int[] arr) {
    return Arrays.stream(arr).distinct().toArray();
}

```

Time: $O(n)$ · **Space:** $O(1)$ (sorted); $O(n)$ (unsorted)

Follow-up: Allow at most two duplicates — compare with `arr[k-2]` instead.

Say while coding: “Since the array is sorted, duplicates are adjacent, so I only compare with the last unique element I’ve written.”

25. Find Duplicate Elements MUST-KNOW

Problem: Print elements that appear more than once. **Input:** [4, 3, 2, 7, 8, 2, 3, 1] **Output:** [2, 3]

Approach: Use a HashSet — if `add` fails, it’s a duplicate. Collect into another set to avoid reporting twice.

Steps: 1) `seen`, `dups` sets. 2) For each `x`: if `!seen.add(x) → dups.add(x)`.

```

static Set<Integer> findDuplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dups = new LinkedHashSet<>();
    for (int x : arr) {
        if (!seen.add(x)) dups.add(x);
    }
    return dups;
}

```

Time: $O(n)$ · **Space:** $O(n)$

Follow-up: Values in range $1..n$ and $O(1)$ space — mark visited by negating `arr[abs(x) - 1]`. Brute force is $O(n^2)$ with nested loops.

Say while coding: “The brute force compares every pair, $O(n^2)$. A HashSet gives $O(1)$ lookups, so the whole thing is $O(n)$.”

26. Find the Missing Number MUST-KNOW

Problem: Array has numbers from 1 to n with one missing. Find it. **Input:** [1, 2, 4, 5, 6] ($n = 6$) **Output:** 3

Approach: Expected sum $n(n+1)/2$ minus actual sum.

Steps: 1) `n = arr.length + 1`. 2) `expected = n*(n+1)/2`. 3) Subtract each element.

```

static int missingNumber(int[] arr) {
    int n = arr.length + 1;
    long expected = (long) n * (n + 1) / 2;
    long actual = 0;
    for (int x : arr) actual += x;
    return (int) (expected - actual);
}

// XOR version — no overflow risk
static int missingNumberXor(int[] arr) {
    int n = arr.length + 1, xor = 0;
    for (int i = 1; i <= n; i++) xor ^= i;
    for (int x : arr) xor ^= x;
    return xor;
}

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Overflow for huge $n →$ use `long` or XOR. Two missing numbers $→$ use sum and sum of squares, or a boolean array.

Say while coding: “The sum of 1 to n is known by formula, so whatever is missing from the actual sum is the missing number.”

27. Move Zeros to the End MUST-KNOW

Problem: Move all zeros to the end while keeping the order of non-zero elements. **Input:** [0, 1, 0, 3, 12] **Output:** [1, 3, 12, 0, 0]

Approach: Two pointers: `pos` = next position for a non-zero. Swap non-zeros forward.

Steps: 1) `pos = 0`. 2) For each `i`: if `arr[i] != 0` → swap `arr[i]` and `arr[pos]`, `pos++`.

```
static void moveZeros(int[] arr) {
    int pos = 0;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] != 0) {
            int temp = arr[pos];
            arr[pos++] = arr[i];
            arr[i] = temp;
        }
    }
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Move zeros to the front instead (iterate from the end). Minimise writes.

Say while coding: “`pos` always points to where the next non-zero belongs. Swapping keeps the relative order of non-zeros and pushes zeros back.”

28. Two Sum MUST-KNOW

Problem: Return indices of two numbers that add up to target. **Input:** `nums = [2, 7, 11, 15]`, `target = 9` **Output:** `[0, 1]`

Approach: HashMap of value → index. For each number, check if `target - num` was seen.

Steps: 1) For each `i`: `need = target - nums[i]`. 2) If map has `need` → return. 3) Else store `nums[i] → i`.

```
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1};
}

// Sorted array → two pointers, O(1) space
static int[] twoSumSorted(int[] nums, int target) {
    int l = 0, r = nums.length - 1;
    while (l < r) {
        int sum = nums[l] + nums[r];
        if (sum == target) return new int[]{l, r};
        if (sum < target) l++; else r--;
    }
    return new int[]{-1, -1};
}
```

Time: $O(n)$ · **Space:** $O(n)$

Follow-up: Sorted input → two pointers (above). Return all pairs. Three Sum (sort + two pointers, $O(n^2)$).

Say while coding: “Brute force checks every pair in $O(n^2)$. Instead, for each number I ask ‘have I already seen its complement?’ — a HashMap answers that in $O(1)$.”

29. Merge Two Sorted Arrays MUST-KNOW

Problem: Merge two sorted arrays into one sorted array. **Input:** `[1, 3, 5]`, `[2, 4, 6]` **Output:** `[1, 2, 3, 4, 5, 6]`

Approach: Two pointers — pick the smaller front element each time, then copy leftovers.

Steps: 1) `i, j, k = 0`. 2) While both have elements, take smaller. 3) Copy remaining.

```
static int[] mergeSorted(int[] a, int[] b) {
    int[] result = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) {
        result[k++] = (a[i] <= b[j]) ? a[i++] : b[j++];
    }
    while (i < a.length) result[k++] = a[i++];
    while (j < b.length) result[k++] = b[j++];
    return result;
}
```

Time: $O(n + m)$ · **Space:** $O(n + m)$

Follow-up: Merge in place when the first array has extra space at the end — fill from the back. This is the merge step of merge sort.

Say while coding: “Both arrays are already sorted, so the smallest remaining element is always at one of the two fronts.”

30. Intersection of Two Arrays **IMPORTANT**

Problem: Find common elements (unique). **Input:** [1, 2, 2, 1], [2, 2] **Output:** [2]

Approach: Put the first array in a set; for each element of the second, if it's in the set, add to the result set.

Steps: 1) set1 from a. 2) For x in b: if set1.contains(x) → add to result.

```
static Set<Integer> intersection(int[] a, int[] b) {
    Set<Integer> set1 = new HashSet<>();
    for (int x : a) set1.add(x);
    Set<Integer> result = new LinkedHashSet<>();
    for (int x : b) {
        if (set1.contains(x)) result.add(x);
    }
    return result;
}
```

Time: $O(n + m)$ · **Space:** $O(n)$

Follow-up: Keep duplicates ([2, 2]) → use a frequency HashMap. Both sorted → two pointers with $O(1)$ extra space. Collections way: set1.retainAll(set2).

Say while coding: “Lookups in a HashSet are $O(1)$, so this is linear instead of the $O(n \times m)$ nested loop.”

31. Union of Two Arrays **IMPORTANT**

Problem: Find all distinct elements present in either array. **Input:** [1, 2, 3], [2, 3, 4] **Output:** [1, 2, 3, 4]

Approach: Add everything to a set.

Steps: 1) Create set. 2) Add all from both arrays.

```
static Set<Integer> union(int[] a, int[] b) {
    Set<Integer> result = new TreeSet<>(); // TreeSet → sorted output
    for (int x : a) result.add(x);
    for (int x : b) result.add(x);
    return result;
}
```

Time: $O((n + m) \log(n + m))$ with TreeSet; $O(n + m)$ with HashSet · **Space:** $O(n + m)$

Follow-up: Both arrays sorted → two-pointer merge skipping duplicates, $O(n + m)$ and sorted output without a tree.

Say while coding: “A set removes duplicates automatically. I used TreeSet so the output is sorted; if order doesn't matter, HashSet is faster.”

32. Rotate Array by K **MUST-KNOW**

Problem: Rotate the array to the right by k steps. **Input:** [1, 2, 3, 4, 5, 6, 7], k = 3 **Output:** [5, 6, 7, 1, 2, 3, 4]

Approach: Reversal algorithm: reverse whole array, reverse first k, reverse the rest.

Steps: 1) $k \% n$. 2) Reverse [0, n-1]. 3) Reverse [0, k-1]. 4) Reverse [k, n-1].

```
static void rotateRight(int[] arr, int k) {
    int n = arr.length;
    if (n == 0) return;
    k %= n;
    reverse(arr, 0, n - 1);
    reverse(arr, 0, k - 1);
    reverse(arr, k, n - 1);
}

static void reverse(int[] arr, int l, int r) {
    while (l < r) {
        int t = arr[l]; arr[l++] = arr[r]; arr[r--] = t;
    }
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Rotate left by k → reverse first k, reverse rest, reverse whole (or rotate right by $n - k$). Simple approach with extra array: result[(i + k) % n] = arr[i].

Say while coding: “ $k \bmod n$ handles k larger than the array. Reversing the whole array brings the last k elements to the front, then two more reversals fix their internal order.”

33. Frequency of Array Elements MUST-KNOW

Problem: Count how many times each element occurs. **Input:** [1, 2, 2, 3, 3, 3] **Output:** 1→1, 2→2, 3→3

Approach: HashMap counting.

Steps: 1) For each x: `map.merge(x, 1, Integer::sum)`.

```
static Map<Integer, Integer> frequency(int[] arr) {
    Map<Integer, Integer> freq = new LinkedHashMap<>();
    for (int x : arr) freq.merge(x, 1, Integer::sum);
    return freq;
}
```

Time: $O(n)$ · **Space:** $O(k)$

Follow-up: Most frequent element; elements appearing more than $n/2$ times (Boyer-Moore majority vote, $O(1)$ space).

Say while coding: “merge inserts 1 if the key is new, otherwise adds 1 to the existing count.”

34. Maximum Subarray Sum (Kadane’s Algorithm) MUST-KNOW

Problem: Find the contiguous subarray with the largest sum. **Input:** [-2, 1, -3, 4, -1, 2, 1, -5, 4] **Output:** 6 (subarray [4, -1, 2, 1])

Approach: At each index, either extend the previous subarray or start fresh — whichever is larger. Track the best seen.

Steps: 1) `current = best = arr[0]`. 2) For each next x: `current = max(x, current + x)`, `best = max(best, current)`.

```
static int maxSubarraySum(int[] arr) {
    int current = arr[0], best = arr[0];
    for (int i = 1; i < arr.length; i++) {
        current = Math.max(arr[i], current + arr[i]);
        best = Math.max(best, current);
    }
    return best;
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Return the start and end indices (track where `current` restarts). All negative? — Works; returns the largest single element because we start with `arr[0]`, not 0.

Say while coding: “If the running sum becomes worse than the current element alone, carrying it forward only hurts, so I start a new subarray.”

22.4 Part D — Searching & Sorting

35. Linear Search MUST-KNOW

Problem: Find the index of a target in an unsorted array. **Input:** [5, 3, 8, 1], target = 8 **Output:** 2

Approach: Check each element one by one.

Steps: 1) Loop through. 2) Return index on match. 3) Return -1 if not found.

```
static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: When is linear better than binary? — When data is unsorted and searched only once (sorting costs $O(n \log n)$).

Say while coding: “No assumptions about order, so I have to check every element in the worst case.”

36. Binary Search MUST-KNOW

Problem: Find the index of a target in a **sorted** array. **Input:** [1, 3, 5, 7, 9, 11], target = 7 **Output:** 3

Approach: Compare with the middle; discard the half that can’t contain the target.

Steps: 1) `low = 0, high = n-1`. 2) `mid = low + (high - low) / 2`. 3) Equal → return; target bigger → `low = mid + 1`; smaller → `high = mid - 1`.

```
static int binarySearch(int[] arr, int target) {
    int low = 0, high = arr.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2; // avoids int overflow
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

static int binarySearchRec(int[] arr, int target, int low, int high) {
    if (low > high) return -1;
    int mid = low + (high - low) / 2;
    if (arr[mid] == target) return mid;
    return arr[mid] < target
        ? binarySearchRec(arr, target, mid + 1, high)
        : binarySearchRec(arr, target, low, mid - 1);
}
```

Time: $O(\log n)$ · **Space:** $O(1)$ iterative; $O(\log n)$ recursive

Follow-up: Why `low + (high - low) / 2` — `(low + high)` can overflow for large indices. First/last occurrence of a duplicate; search in a rotated sorted array; square root using binary search.

Say while coding: “Each comparison halves the search space, so it’s $\log n$. I compute mid this way to avoid integer overflow.”

37. Bubble Sort **MUST-KNOW**

Problem: Sort an array using bubble sort. **Input:** `[5, 1, 4, 2, 8]` **Output:** `[1, 2, 4, 5, 8]`

Approach: Repeatedly swap adjacent elements that are out of order; after each pass, the largest remaining element “bubbles” to the end.

Steps: 1) Outer loop for passes. 2) Inner loop compares `arr[j]` and `arr[j+1]`. 3) Stop early if no swaps happened.

```
static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break; // already sorted
    }
}
```

Time: $O(n^2)$ worst/average, $O(n)$ best (already sorted, with the flag) · **Space:** $O(1)$ · **Stable:** Yes

Follow-up: Why `n - 1 - i`? — The last `i` elements are already in place.

Say while coding: “After each pass the largest unsorted element is at the end, so the inner loop gets shorter. The swapped flag lets me stop early on sorted input.”

38. Selection Sort **MUST-KNOW**

Problem: Sort using selection sort. **Input:** `[64, 25, 12, 22, 11]` **Output:** `[11, 12, 22, 25, 64]`

Approach: For each position, find the minimum in the remaining unsorted part and swap it into place.

Steps: 1) For `i` from 0: `minIdx = i`. 2) Scan `j > i` for smaller. 3) Swap `arr[i]` and `arr[minIdx]`.

```
static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        int t = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = t;
    }
}
```

Time: $O(n^2)$ always · **Space:** $O(1)$ · **Stable:** No

Follow-up: Advantage — at most $n - 1$ swaps (useful when writes are expensive).

Say while coding: “The left part is always sorted. I pick the smallest from the right part and put it at the boundary.”

39. Insertion Sort **MUST-KNOW**

Problem: Sort using insertion sort. **Input:** [12, 11, 13, 5, 6] **Output:** [5, 6, 11, 12, 13]

Approach: Like sorting playing cards — take each element and insert it into the correct place in the sorted left part, shifting larger elements right.

Steps: 1) `key = arr[i]`. 2) Shift elements greater than key one step right. 3) Place key.

```
static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

Time: $O(n^2)$ worst, $O(n)$ best (nearly sorted) · **Space:** $O(1)$ · **Stable:** Yes

Follow-up: Why is it good for small or nearly-sorted data? — Few shifts needed; Java’s TimSort uses insertion sort for small runs.

Say while coding: “Everything left of `i` is sorted. I hold the current value as key and shift bigger elements right until I find its slot.”

40. Merge Sort **MUST-KNOW**

Problem: Sort using merge sort. **Input:** [38, 27, 43, 3, 9, 82, 10] **Output:** [3, 9, 10, 27, 38, 43, 82]

Approach: Divide and conquer — split the array in halves, sort each half recursively, then merge the two sorted halves.

Steps: 1) Base case: size ≤ 1 . 2) Sort left and right halves. 3) Merge using two pointers into a temp array.

```
static void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

static void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];
    int i = left, j = mid + 1, k = 0;
    while (i <= mid && j <= right) {
        temp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];
    System.arraycopy(temp, 0, arr, left, temp.length);
}

// Call: mergeSort(arr, 0, arr.length - 1);
```

Time: $O(n \log n)$ in all cases · **Space:** $O(n)$ · **Stable:** Yes

Follow-up: Why $O(n \log n)$? — $\log n$ levels of splitting, $O(n)$ merge work per level. Merge sort on linked lists needs no extra array. Count inversions using merge sort.

Say while coding: “I keep splitting until single elements, which are trivially sorted, then merge back up. The `<=` in merge keeps it stable.”

41. Quick Sort **MUST-KNOW**

Problem: Sort using quick sort. **Input:** [10, 7, 8, 9, 1, 5] **Output:** [1, 5, 7, 8, 9, 10]

Approach: Pick a pivot, partition so smaller elements go left and larger go right, then recursively sort both sides.

Steps (Lomuto partition): 1) Pivot = last element. 2) `i` tracks the end of the “smaller” region. 3) Swap each smaller element into it. 4) Put pivot at `i + 1`.

```

static void quickSort(int[] arr, int low, int high) {
    if (low < high) {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

static int partition(int[] arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
    }
    int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
    return i + 1;
}
// Call: quickSort(arr, 0, arr.length - 1);

```

Time: $O(n \log n)$ average, $O(n^2)$ worst (already sorted with last-element pivot) · **Space:** $O(\log n)$ recursion · **Stable:** No
Follow-up: How to avoid the worst case? — Random pivot or median-of-three. Quick sort vs merge sort — quick sort is in-place and cache-friendly; merge sort is stable and guaranteed $O(n \log n)$.
Say while coding: “After partitioning, the pivot is in its final sorted position — everything left is smaller, everything right is larger — so I recurse on both sides.”

Sorting Algorithms Summary

Algorithm	Best	Average	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Java Arrays.sort (primitives)	—	$O(n \log n)$	$O(n \log n)$	$O(\log n)$	No (dual-pivot quicksort)
Java Arrays.sort / Collections.sort (objects)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes (TimSort)

22.5 Part E — Linked List

All linked list problems use this node class:

```

static class ListNode {
    int val;
    ListNode next;
    ListNode(int val) { this.val = val; }
}

```

42. Reverse a Linked List MUST-KNOW

Problem: Reverse a singly linked list. **Input:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \text{null}$ **Output:** $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow \text{null}$

Approach: Iterate with three pointers — `prev`, `curr`, `next` — and flip each link.

Steps: 1) `prev = null`, `curr = head`. 2) Save `next`, point `curr.next` to `prev`, move both forward. 3) `prev` is the new head.

```

static ListNode reverseList(ListNode head) {
    ListNode prev = null, curr = head;
    while (curr != null) {
        ListNode next = curr.next; // save
        curr.next = prev;         // reverse link
        prev = curr;              // move prev
        curr = next;              // move curr
    }
    return prev;
}

static ListNode reverseListRec(ListNode head) {
    if (head == null || head.next == null) return head;
    ListNode newHead = reverseListRec(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}

```

Time: $O(n)$ · **Space:** $O(1)$ iterative; $O(n)$ recursive

Follow-up: Reverse in groups of k ; check if a linked list is a palindrome (find middle, reverse second half, compare).

Say while coding: “I must save next before changing curr.next, otherwise I lose the rest of the list.”

43. Find the Middle Node MUST-KNOW

Problem: Return the middle node (second middle for even length). **Input:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ **Output:** 3

Approach: Slow and fast pointers — slow moves 1 step, fast moves 2. When fast reaches the end, slow is at the middle.

Steps: 1) Both start at head. 2) While `fast != null && fast.next != null`: advance. 3) Return slow.

```

static ListNode middleNode(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }
    return slow;
}

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Return the first middle for even length (start fast at `head.next`). Find the k th node from the end (move fast k steps ahead first).

Say while coding: “Fast covers twice the distance, so when it finishes, slow has covered half — one pass, no need to count the length first.”

44. Detect a Cycle in a Linked List MUST-KNOW

Problem: Return true if the list has a cycle. **Input:** $3 \rightarrow 2 \rightarrow 0 \rightarrow -4 \rightarrow (\text{back to } 2)$ **Output:** true

Approach: Floyd’s cycle detection (tortoise and hare). If there’s a loop, the fast pointer eventually meets the slow one.

Steps: 1) slow/fast from head. 2) Move 1 and 2 steps. 3) If they meet $\rightarrow$ cycle. 4) If fast hits null $\rightarrow$ no cycle.

```

static boolean hasCycle(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) return true;
    }
    return false;
}

// Follow-up: find where the cycle starts
static ListNode cycleStart(ListNode head) {
    ListNode slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
        if (slow == fast) {
            slow = head;
            while (slow != fast) {
                slow = slow.next;
                fast = fast.next;
            }
            return slow;
        }
    }
    return null;
}

```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Alternative with a HashSet of visited nodes — $O(n)$ space. Find the cycle start (above) or cycle length.

Say while coding: “Inside a loop, fast gains one step on slow each move, so it must catch up — they can’t jump over each other.”

45. Remove Duplicates from a Sorted Linked List **IMPORTANT**

Problem: Delete duplicate nodes so each value appears once. **Input:** `1 → 1 → 2 → 3 → 3` **Output:** `1 → 2 → 3`

Approach: Since it’s sorted, duplicates are adjacent — skip the next node while it has the same value.

Steps: 1) `curr = head`. 2) If `curr.val == curr.next.val` → `curr.next = curr.next.next`; else move on.

```
static ListNode deleteDuplicates(ListNode head) {
    ListNode curr = head;
    while (curr != null && curr.next != null) {
        if (curr.val == curr.next.val) curr.next = curr.next.next;
        else curr = curr.next;
    }
    return head;
}
```

Time: $O(n)$ · **Space:** $O(1)$

Follow-up: Unsorted list → use a HashSet of seen values and a `prev` pointer.

Say while coding: “I don’t move curr when I delete, because the new next node might also be a duplicate.”

22.6 Part F — Stack & Queue

46. Balanced Parentheses **MUST-KNOW**

Problem: Check whether brackets are balanced. **Input:** `"{[()]}"` → `true`; `"([)]"` → `false`

Approach: Stack — push opening brackets; on a closing bracket, the top must be the matching opener.

Steps: 1) Opening → push. 2) Closing → stack empty or mismatch → `false`; else pop. 3) End: stack must be empty.

```
static boolean isBalanced(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '{' || c == '[') {
            stack.push(c);
        } else if (c == ')' || c == '}' || c == ']') {
            if (stack.isEmpty()) return false;
            char open = stack.pop();
            if ((c == ')' && open != '(') ||
                (c == '}' && open != '{') ||
                (c == ']' && open != '[')) return false;
        }
    }
    return stack.isEmpty();
}
```

Time: $O(n)$ · **Space:** $O(n)$

Follow-up: Only (and) → a counter is enough (never let it go negative). Minimum additions to balance. Why ArrayDeque instead of Stack? — Stack is legacy and synchronized.

Say while coding: “The most recent unmatched opener must be closed first — that’s last-in-first-out, so a stack fits exactly.”

47. Implement a Stack Using an Array **MUST-KNOW**

Problem: Implement push, pop, peek, isEmpty. **Input:** `push(10), push(20), pop(), peek()` **Output:** `pop` → 20, `peek` → 10

Approach: Array plus a `top` index. Push increments `top`; pop decrements.

Steps: 1) `top = -1`. 2) push: check overflow, `arr[++top] = x`. 3) pop: check underflow, return `arr[top--]`.

```

static class ArrayStack {
    private final int[] data;
    private int top = -1;

    ArrayStack(int capacity) { data = new int[capacity]; }

    void push(int x) {
        if (top == data.length - 1) throw new RuntimeException("Stack Overflow");
        data[++top] = x;
    }

    int pop() {
        if (isEmpty()) throw new RuntimeException("Stack Underflow");
        return data[top--];
    }

    int peek() {
        if (isEmpty()) throw new RuntimeException("Stack is empty");
        return data[top];
    }

    boolean isEmpty() { return top == -1; }
    int size() { return top + 1; }
}

```

Time: O(1) for all operations · **Space:** O(n)

Follow-up: Make it dynamic (double the array when full); implement with a linked list; min-stack that returns the minimum in O(1) (keep a second stack of minimums).

Say while coding: “top points to the last inserted element. I check for overflow and underflow first so the class fails clearly instead of throwing an index error.”

48. Implement a Queue (Using Two Stacks) IMPORTANT

Problem: Implement a FIFO queue with `enqueue`, `dequeue`, `peek` using two stacks. **Input:** `enqueue(1)`, `enqueue(2)`, `dequeue()`, `enqueue(3)`, `dequeue()` **Output:** 1, 2

Approach: `in` stack receives new elements; `out` stack serves removals. When `out` is empty, pour everything from `in` into `out` — this reverses the order, making the oldest element come out first.

Steps: 1) `enqueue` → push to `in`. 2) `dequeue` → if `out` empty, move all from `in`. 3) Pop from `out`.

```

static class QueueUsingStacks {
    private final Deque<Integer> in = new ArrayDeque<>();
    private final Deque<Integer> out = new ArrayDeque<>();

    void enqueue(int x) { in.push(x); }

    int dequeue() {
        shift();
        if (out.isEmpty()) throw new RuntimeException("Queue is empty");
        return out.pop();
    }

    int peek() {
        shift();
        if (out.isEmpty()) throw new RuntimeException("Queue is empty");
        return out.peek();
    }

    boolean isEmpty() { return in.isEmpty() && out.isEmpty(); }

    private void shift() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
    }
}

// Queue basics with the built-in API
// Queue<Integer> q = new ArrayDeque<>(); q.offer(1); q.poll(); q.peek();

```

Time: Amortised O(1) per operation (each element moves at most once) · **Space:** O(n)

Follow-up: Circular array queue (use `front`, `rear` with `% capacity`). Stack using queues. `offer/poll/peek` vs `add/remove/element` (return special values vs throw).

Say while coding: “I only transfer when the out stack is empty, so each element is moved once overall — that’s what makes it amortised O(1).”

22.7 Part G — Recursion & Dynamic Programming

(Fibonacci and factorial with recursion are covered in Problems 5 and 6.)

49. Climbing Stairs MUST-KNOW

Problem: You can climb 1 or 2 steps at a time. How many distinct ways to reach step n? **Input:** n = 4 **Output:** 5 (1111, 112, 121, 211, 22)

Approach: To reach step n, the last move came from n-1 or n-2, so $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$ — the Fibonacci pattern.

Steps: 1) Base: $\text{ways}(1) = 1$, $\text{ways}(2) = 2$. 2) Build up with two variables.

```
static int climbStairs(int n) {
    if (n <= 2) return n;
    int oneBack = 2, twoBack = 1;    // ways(2), ways(1)
    for (int i = 3; i <= n; i++) {
        int current = oneBack + twoBack;
        twoBack = oneBack;
        oneBack = current;
    }
    return oneBack;
}

// Recursive + memo (top-down DP)
static int climbMemo(int n, int[] memo) {
    if (n <= 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = climbMemo(n - 1, memo) + climbMemo(n - 2, memo);
}
```

Time: $O(n)$ · **Space:** $O(1)$ bottom-up; $O(n)$ memo

Follow-up: Allow 1, 2 or 3 steps → add $\text{ways}(n-3)$. Minimum cost climbing stairs. What is DP? — “Solving a problem by breaking it into overlapping subproblems and storing their results so each is solved only once.”

Say while coding: “The last step is either a 1-step or a 2-step, so the answer is the sum of the ways to reach the previous two steps.”

50. 0/1 Knapsack **IMPORTANT**

Problem: Given item weights and values and a bag capacity W, pick items (each at most once) to maximise total value.

Input: weights = [1, 3, 4, 5], values = [1, 4, 5, 7], W = 7 **Output:** 9 (items with weight 3 + 4)

Approach: DP table $\text{dp}[i][w]$ = best value using the first i items with capacity w. For each item: skip it, or take it if it fits.

Steps: 1) $\text{dp}[0][*] = 0$. 2) For each item i and capacity w: $\text{dp}[i][w] = \text{dp}[i-1][w]$ (skip); if $\text{wt}[i-1] \leq w$, also consider $\text{val}[i-1] + \text{dp}[i-1][w - \text{wt}[i-1]]$ (take). 3) Answer $\text{dp}[n][W]$.

```
static int knapsack(int[] wt, int[] val, int W) {
    int n = wt.length;
    int[][] dp = new int[n + 1][W + 1];
    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
            dp[i][w] = dp[i - 1][w];    // skip item
            if (wt[i - 1] <= w) {
                dp[i][w] = Math.max(dp[i][w],
                                    val[i - 1] + dp[i - 1][w - wt[i - 1]]); // take item
            }
        }
    }
    return dp[n][W];
}

// Space-optimised: 1D array, iterate capacity backwards
static int knapsack1D(int[] wt, int[] val, int W) {
    int[] dp = new int[W + 1];
    for (int i = 0; i < wt.length; i++) {
        for (int w = W; w >= wt[i]; w--) {
            dp[w] = Math.max(dp[w], val[i] + dp[w - wt[i]]);
        }
    }
    return dp[W];
}
```

Time: $O(n \times W)$ · **Space:** $O(n \times W)$, or $O(W)$ optimised

Follow-up: Why iterate backwards in 1D? — So each item is used at most once (forward iteration would reuse it → unbounded knapsack). Why not greedy by value/weight? — Greedy fails for 0/1; it only works for fractional knapsack.

Say while coding: “For every item I have two choices — take it or leave it. The table stores the best answer for each smaller capacity, so I never recompute.”

23 Top 100 Java Interview Questions for Freshers

Priority markers: **VERY HIGH** · **HIGH** · **MEDIUM**

Priorities reflect how often each topic shows up as a standard fresher interview pattern — not measured statistics. The section where each is answered is shown in brackets.

23.1 Java Basics

#	Question	Priority
1	What is Java? What are its main features? [1]	VERY HIGH
2	Why is Java platform independent? Is the JVM platform independent? [1]	VERY HIGH
3	Difference between JDK, JRE and JVM [1]	VERY HIGH
4	Explain <code>public static void main(String[] args)</code> [1]	VERY HIGH
5	What is bytecode? How is a Java program compiled and executed? [1]	HIGH
6	Is Java 100% object-oriented? [1]	HIGH
7	Is Java pass-by-value or pass-by-reference? [1]	VERY HIGH
8	Primitive vs non-primitive data types [2]	HIGH
9	What are wrapper classes? Autoboxing and unboxing? [2]	HIGH
10	Difference between <code>==</code> and <code>equals()</code> [2]	VERY HIGH
11	What is type casting? Widening vs narrowing [2]	MEDIUM
12	Difference between Java and C++ [1]	HIGH
13	What is JIT compiler? [1]	MEDIUM
14	Local vs instance vs static variables [2]	HIGH

23.2 OOP

#	Question	Priority
15	What are the four pillars of OOP? Explain each with an example. [4]	VERY HIGH
16	What is a class and an object? [4]	VERY HIGH
17	What is a constructor? Types of constructors? [4]	VERY HIGH
18	Constructor vs method [4]	HIGH
19	What is encapsulation? How do you achieve it? [4]	VERY HIGH
20	What is inheritance? Types supported in Java? [4]	VERY HIGH
21	Why doesn't Java support multiple inheritance through classes? [4]	VERY HIGH
22	What is polymorphism? Compile-time vs runtime [4]	VERY HIGH
23	Method overloading vs method overriding [4]	VERY HIGH
24	What is abstraction? How is it achieved? [4]	VERY HIGH
25	Abstract class vs interface [4]	VERY HIGH
26	Can we override static / private / final methods? [21]	VERY HIGH
27	<code>this</code> vs <code>super</code> keyword [4]	HIGH
28	What is dynamic method dispatch? [4]	HIGH
29	Access modifiers and their scope [4]	VERY HIGH
30	IS-A vs HAS-A; aggregation vs composition [4]	HIGH
31	<code>static</code> keyword — variable, method, block [5]	VERY HIGH
32	<code>final</code> vs <code>finally</code> vs <code>finalize</code> [5]	VERY HIGH
33	Can an abstract class have a constructor? [21]	HIGH
34	What are default methods in interfaces and why were they added? [12]	HIGH

23.3 Strings

#	Question	Priority
35	Why is String immutable? [6]	VERY HIGH
36	What is the String pool? [6]	VERY HIGH
37	String vs StringBuilder vs StringBuffer [6]	VERY HIGH
38	String literal vs new String(); how many objects are created? [6]	VERY HIGH
39	== vs equals() for Strings [6]	VERY HIGH
40	Common String methods (substring, split, charAt...) [6]	HIGH
41	What does intern() do? [6]	MEDIUM

23.4 Collections

#	Question	Priority
42	Explain the Collection framework hierarchy [8]	VERY HIGH
43	ArrayList vs LinkedList [8]	VERY HIGH
44	Array vs ArrayList [7]	VERY HIGH
45	List vs Set vs Map [8]	VERY HIGH
46	HashSet vs LinkedHashSet vs TreeSet [8]	HIGH
47	How does HashSet prevent duplicates? [8]	HIGH
48	Comparable vs Comparator [8]	VERY HIGH
49	Collection vs Collections [8]	HIGH
50	Fail-fast vs fail-safe iterators; ConcurrentModificationException [8]	HIGH
51	ArrayList vs Vector [8]	MEDIUM
52	Iterator vs ListIterator [8]	MEDIUM
53	How does ArrayList grow internally? [8]	HIGH

23.5 HashMap

#	Question	Priority
54	How does HashMap work internally? [9]	VERY HIGH
55	What is a collision and how is it handled? [9]	VERY HIGH
56	equals() and hashCode() contract — what if you override only one? [15]	VERY HIGH
57	HashMap vs Hashtable [8]	VERY HIGH
58	HashMap vs LinkedHashMap vs TreeMap [8]	HIGH
59	What is load factor and rehashing? [9]	HIGH
60	What changed in HashMap in Java 8 (treeification)? [9]	HIGH
61	Why does HashMap allow one null key? [9]	HIGH
62	HashMap vs ConcurrentHashMap [8]	HIGH

23.6 Exception Handling

#	Question	Priority
63	What is an exception? Explain the exception hierarchy [10]	VERY HIGH
64	Checked vs unchecked exceptions [10]	VERY HIGH
65	throw vs throws [10]	VERY HIGH
66	Exception vs Error [10]	HIGH
67	Does finally always execute? When doesn't it? [10]	HIGH
68	How do you create a custom exception? [10]	HIGH
69	What is try-with-resources? [10]	HIGH

23.7 Multithreading

#	Question	Priority
70	Process vs thread [11]	HIGH
71	Ways to create a thread; Thread vs Runnable [11]	VERY HIGH
72	start() vs run() [11]	VERY HIGH
73	Thread lifecycle [11]	HIGH
74	What is synchronization? synchronized method vs block [11]	VERY HIGH
75	sleep() vs wait() [11]	HIGH
76	What is a deadlock and how do you avoid it? [11]	HIGH
77	What is volatile? [11]	MEDIUM
78	What is ExecutorService / thread pool? [11]	MEDIUM
79	Runnable vs Callable [11]	MEDIUM

23.8 Java 8

#	Question	Priority
80	What are the main features of Java 8? [12]	VERY HIGH
81	What is a lambda expression? [12]	VERY HIGH
82	What is a functional interface? Examples? [12]	VERY HIGH
83	What is the Stream API? Intermediate vs terminal operations [12]	VERY HIGH
84	map() vs flatMap() [12]	HIGH
85	What is Optional? [12]	HIGH
86	Predicate, Function, Consumer, Supplier [12]	HIGH
87	Write a stream to group/filter/find the second highest [12]	VERY HIGH

23.9 JVM & Memory

#	Question	Priority
88	Stack vs heap memory [13]	VERY HIGH
89	What is garbage collection? When is an object eligible? [14]	VERY HIGH
90	JVM memory areas [13]	HIGH
91	StackOverflowError vs OutOfMemoryError [13]	MEDIUM
92	Can we force garbage collection? System.gc() [14]	MEDIUM

23.10 Coding (must be able to write on paper)

#	Question	Priority
93	Reverse a string / check palindrome [22: #10, #11]	VERY HIGH
94	Find duplicates / character frequency / first non-repeating char [22: #13-15, #25]	VERY HIGH
95	Fibonacci, prime, factorial, Armstrong [22: #3-6]	VERY HIGH
96	Second largest, missing number, Two Sum, reverse linked list [22]	VERY HIGH

23.11 JDBC

#	Question	Priority
97	Steps to connect Java to a database using JDBC [19]	HIGH
98	Statement vs PreparedStatement; what is SQL injection? [19]	VERY HIGH

23.12 Miscellaneous

#	Question	Priority
99	What is a Singleton class? How do you make it thread-safe? [20]	VERY HIGH
100	How do you create an immutable class? [20]	HIGH

24 Rapid Revision

24.1 1-Day Java Revision Sheet

Work through these blocks in order the day before an interview. Each block $\approx$ 1 hour. Read the interview answers **out loud**.

Block 1 — Foundations (1 hr)

- JDK = JRE + tools; JRE = JVM + libraries; bytecode + JVM = platform independence.
- `main` signature — why public, static, void.
- 8 primitives with sizes; wrapper classes; Integer cache (-128 to 127).
- `==` vs `equals()`; pass-by-value.

Block 2 — OOP (1.5 hr) — highest-weight block

- Four pillars with one example each (use `BankAccount` / `Payment`).
- Overloading vs overriding table + overriding rules (access, exceptions, return type).
- Abstract class vs interface table; Java 8 default methods; diamond problem.
- `static`, `final`, `this`, `super`; constructor chaining; execution order (static block $\rightarrow$ main $\rightarrow$ instance block $\rightarrow$ constructor).
- Access modifier table.

Block 3 — Strings (45 min)

- Immutability — 4 reasons (pool, security, thread safety, hashCode caching).
- String pool, literal vs `new`, how many objects.
- String vs StringBuilder vs StringBuffer.

Block 4 — Collections + HashMap (1.5 hr) — second-highest weight

- Hierarchy diagram (draw it from memory).
- ArrayList vs LinkedList; HashSet internals; Comparable vs Comparator.
- HashMap 2-minute answer — say it three times.
- equals/hashCode contract.
- HashMap vs Hashtable vs ConcurrentHashMap.

Block 5 — Exceptions + Threads (1 hr)

- Hierarchy; checked vs unchecked; throw vs throws; finally rules; custom exception; try-with-resources.
- Thread creation, start vs run, synchronized, sleep vs wait, deadlock.

Block 6 — Java 8 (45 min)

- Lambda, functional interface, 4 core interfaces.
- Stream pipeline; write 5 stream snippets by hand (filter-map-collect, groupingBy, second highest, frequency, sum of evens).
- Optional.

Block 7 — JVM/GC + JDBC + Design (45 min)

- Stack vs heap; GC eligibility; generational heap.
- JDBC steps; PreparedStatement + SQL injection; commit/rollback.
- Singleton (double-checked locking), immutable class, SOLID one-liners.

Block 8 — Coding (2 hrs)

- Write without an IDE: reverse string, palindrome, anagram, first non-repeating char, second largest, missing number, Two Sum, move zeros, Kadane, binary search, bubble sort, reverse linked list, balanced parentheses, Fibonacci.
- For each, say the time and space complexity out loud.

24.2 30-Minute Revision

1. **JDK/JRE/JVM** — JDK = JRE + dev tools; JRE = JVM + libraries; JVM executes bytecode.
2. **Platform independence** — compiled to bytecode; each OS has its own JVM.

3. **OOP pillars** — Encapsulation (private fields + getters/setters), Inheritance (`extends` , IS-A), Polymorphism (overloading = compile-time, overriding = runtime), Abstraction (abstract class/interface).
4. **Overloading vs overriding** — same class different params vs child same signature; compile vs runtime.
5. **Abstract class vs interface** — state + constructors + single inheritance vs contract + multiple implementation.
6. **No multiple inheritance with classes** — diamond problem.
7. **static** — class-level; one copy; no `this`. **final** — constant / no override / no inheritance.
8. **String immutable** — pool, security, thread-safe, cached hash.
9. **String vs StringBuilder vs StringBuffer** — immutable vs mutable-fast vs mutable-synchronized.
10. **== vs equals** — reference vs content.
11. **ArrayList vs LinkedList** — array $O(1)$ get vs nodes $O(1)$ ends.
12. **HashMap** — buckets; `hashCode` → index; `equals` → exact key; collision → list → tree after 8; capacity 16, load factor 0.75, `resize` $\times 2$; one null key; not synchronized.
13. **equals/hashCode contract** — equal objects must have equal hash codes.
14. **Comparable vs Comparator** — natural order inside class vs custom order outside.
15. **Checked vs unchecked** — compile-time must-handle vs `RuntimeException`.
16. **throw vs throws** — throws an object vs declares in signature.
17. **final/finally/finalize** — keyword / cleanup block / deprecated GC method.
18. **start vs run** — new thread vs normal call.
19. **synchronized** — one thread at a time on a lock; **deadlock** — circular waiting on locks.
20. **Lambda / functional interface / Stream** — anonymous function / one abstract method / declarative pipeline (lazy, source untouched).
21. **Stack vs heap** — method frames & locals vs objects; per-thread vs shared.
22. **GC** — frees unreachable objects; `System.gc()` is only a request.
23. **PreparedStatement** — precompiled, parameterised, prevents SQL injection.
24. **Singleton** — private constructor + static instance + static getter; volatile + double-checked locking.

24.3 10-Minute Revision

- **JVM** runs bytecode → platform independence. **JDK** $\supset$ **JRE** $\supset$ **JVM**.
- **OOP**: Encapsulation, Inheritance, Polymorphism, Abstraction.
- **Overload** = same name, different params, compile-time. **Override** = same signature in child, runtime.
- **Interface** = contract, multiple; **abstract class** = partial implementation, single.
- **String** is immutable & pooled; use **StringBuilder** for changes.
- **HashMap**: `hashCode` → bucket, `equals` → key; 16 / 0.75; treeify at 8; $O(1)$.
- Override `equals` ⇒ override `hashCode`.
- **Checked** = compile-time; **unchecked** = `RuntimeException`.
- **start()** creates a thread; **run()** doesn't.
- **Streams**: filter → map → collect; lazy; single use.
- **Stack** = methods & locals; **Heap** = objects.
- **PreparedStatement** stops SQL injection.

24.4 Last-Minute Interview Cheat Sheet

Question	1-3 line answer
What is Java?	High-level, object-oriented, platform-independent language; compiled to bytecode that runs on any JVM — “write once, run anywhere”.
JDK vs JRE vs JVM	JVM runs bytecode; JRE = JVM + libraries to run programs; JDK = JRE + tools like <code>javac</code> to develop them.
Why main is static?	So the JVM can call it without creating an object.
Encapsulation	Bundling data and methods in a class and hiding data with private fields, exposing it through controlled getters/setters.
Inheritance	A class acquiring another class's properties with <code>extends</code> — IS-A relationship, for code reuse.
Polymorphism	One name, many forms — overloading (compile-time) and overriding (runtime).
Abstraction	Showing what an object does, hiding how — via abstract classes and interfaces.
Abstract class vs interface	Abstract class: state, constructors, single inheritance. Interface: contract, constants only, multiple implementation.

Overloading vs overriding	Overloading: same name, different params, same class. Overriding: same signature in child, resolved at runtime.
Why no multiple inheritance?	Diamond problem — ambiguity when two parents have the same method. Interfaces solve it.
static	Belongs to the class; one shared copy; accessible without an object.
final	Variable → constant; method → can't override; class → can't extend.
final / finally / finalize	Keyword / always-run cleanup block / deprecated GC method.
Why String immutable?	String pool sharing, security, thread safety, cached hashCode.
String vs StringBuilder vs StringBuffer	Immutable / mutable, not synchronized, fastest / mutable, synchronized.
== vs equals	== compares references; equals compares content (when overridden).
ArrayList vs LinkedList	Dynamic array with O(1) access vs doubly linked list with O(1) insert/remove at ends.
HashMap internals	Array of buckets; hashCode picks bucket, equals finds key; collisions chained, treeified after 8; resizes ×2 past 0.75 load.
equals/hashCode contract	If two objects are equal, their hash codes must be equal. Override both together.
HashMap vs Hashtable	HashMap: not synchronized, one null key. Hashtable: synchronized, no nulls, legacy.
Comparable vs Comparator	Comparable: natural order via <code>compareTo</code> in the class. Comparator: custom orders via <code>compare</code> , outside the class.
Checked vs unchecked	Checked: verified at compile time, must handle. Unchecked: RuntimeException, not enforced.
throw vs throws	<code>throw</code> throws an exception object; <code>throws</code> declares exceptions in the method signature.
try-with-resources	Auto-closes AutoCloseable resources after the try block.
start() vs run()	start() creates a new thread which calls run(); calling run() directly runs on the current thread.
sleep vs wait	sleep keeps the lock; wait releases it and needs synchronized + notify.
Deadlock	Threads waiting forever for each other's locks; avoid with consistent lock ordering.
Lambda	Anonymous function implementing a functional interface: <code>(a, b) -> a + b</code> .
Functional interface	Interface with exactly one abstract method — Runnable, Comparator, Predicate.
Stream API	Declarative, lazy pipeline: source → intermediate ops → terminal op; doesn't modify the source.
Optional	Container that may hold a value — avoids returning null.
Stack vs heap	Stack: per-thread method frames and locals. Heap: shared objects managed by GC.
Garbage collection	JVM automatically frees memory of unreachable objects.
PreparedStatement	Precompiled, parameterised SQL — faster and prevents SQL injection.
Singleton	One instance: private constructor, private static instance, public static getInstance().
Immutable class	final class, private final fields, no setters, defensive copies.

25 If the Interviewer Asks Me Anything About Java

This is the survival sheet — the patterns that let you handle questions you haven't seen before.

25.1 The Universal Answer Structure

For any **“What is X?”**: Definition (1 line) → Why it exists (1 line) → Example (1 line).

“X is ... It's used because ... For example, ...”

For any **“X vs Y?”**: Name the one core difference first, then 2-3 supporting points, then when you'd use each.

“The main difference is ... Also, ... and ... I'd use X when ... and Y when ...”

For any **“Can we ...?”**: Yes/No → the rule that decides it → what happens instead.

“No, because ... If we try, the compiler/JVM ...”

For **something you don't know**: be honest, then reason from what you do know.

“I haven't used that directly, but based on how [related concept] works, I'd expect ...”

25.2 20 Definitions You Must Say Without Thinking

1. **Class** — blueprint that defines properties and behaviours of objects.
2. **Object** — instance of a class with state, behaviour and identity, stored in the heap.
3. **Constructor** — special method with the class name and no return type that initialises an object.
4. **Encapsulation** — bundling data and methods, hiding data with private access and exposing it through methods.
5. **Inheritance** — acquiring properties of a parent class using `extends` (IS-A).
6. **Polymorphism** — one name, many forms: overloading (compile-time) and overriding (runtime).
7. **Abstraction** — hiding implementation, showing only essential features.
8. **Interface** — a contract of methods a class must implement; supports multiple inheritance.
9. **Abstract class** — a partially implemented class that can't be instantiated.
10. **JVM** — engine that loads, verifies and executes bytecode and manages memory.
11. **Bytecode** — platform-independent instructions in `.class` files, executed by the JVM.
12. **String pool** — heap area where string literals are stored once and shared.
13. **Collection framework** — interfaces and classes in `java.util` for storing and processing groups of objects.
14. **HashMap** — key-value store using hashing; O(1) average get/put.
15. **Exception** — an event that disrupts normal flow; handled with try-catch-finally.
16. **Thread** — smallest unit of execution within a process.
17. **Synchronization** — allowing only one thread at a time into a critical section.
18. **Lambda** — an anonymous function implementing a functional interface.
19. **Stream** — a lazy pipeline for processing collection data declaratively.
20. **Garbage collection** — automatic freeing of memory used by unreachable objects.

25.3 15 Differences That Cover Most Comparison Questions

Pair	The one-line difference
JDK / JRE / JVM	Develop / run / execute
== / equals()	Reference / content
Overloading / overriding	Compile-time, different params / runtime, same signature
Abstract class / interface	Partial implementation + state / pure contract, multiple
String / StringBuilder / StringBuffer	Immutable / mutable fast / mutable thread-safe
ArrayList / LinkedList	Fast access / fast insert-delete at ends
Array / ArrayList	Fixed, primitives / dynamic, objects
List / Set	Duplicates + order / unique
HashMap / Hashtable	Unsynchronized, nulls OK / synchronized, no nulls
HashMap / TreeMap	O(1) unordered / O(log n) sorted
Comparable / Comparator	Natural order inside / custom order outside
Checked / unchecked	Compile-time enforced / runtime
throw / throws	Throws an object / declares in signature
final / finally / finalize	Restrict / cleanup block / deprecated GC hook
Stack / heap	Methods & locals per thread / shared objects, GC-managed

25.4 Reasoning Rules for Unexpected “Can we / What happens” Questions

Most trick questions can be answered from these rules:

1. **Static belongs to the class** → no `this`, no overriding (only hiding), no access to instance members directly.
2. **Private isn't visible outside the class** → can't be overridden or accessed by children.
3. **Final means “no change”** → no reassignment, no overriding, no subclassing.
4. **Abstract means “incomplete”** → can't instantiate, can't be final/private/static.
5. **Constructors aren't inherited** → can't be overridden, final, static or abstract.
6. **Methods are resolved by object type; fields and static methods by reference type.**
7. **Overriding can't make things more restrictive** — access must be same or wider; checked exceptions same or narrower.
8. **Hash-based collections use hashCode() then equals();** sorted collections use compareTo()/compare().
9. **Only one of this() / super(),** and it must be the first line.
10. **The compiler checks checked exceptions; the JVM throws unchecked ones.**
11. **Immutable objects are thread-safe; shared mutable state needs synchronization.**
12. **Objects live in the heap; references and locals live in the stack.**

25.5 Coding Patterns That Solve Most Fresher Problems

Pattern	Use it for	Problems
Digit extraction (% 10 , / 10)	Number manipulation	Reverse, palindrome, Armstrong, sum/count digits
Two pointers (ends)	Reversing, palindromes, sorted pair search	Reverse string/array, palindrome, Two Sum (sorted)
Two pointers (read/write)	In-place filtering	Remove duplicates, move zeros
HashMap counting	Frequencies, duplicates, lookup	Char frequency, first non-repeating, anagram, Two Sum
HashSet	Uniqueness, “seen before?”	Duplicates, intersection, remove duplicate chars
Fixed array int[26]	Counting lowercase letters	Anagram, frequency
Running max / Kadane	Best contiguous result	Max subarray, max/min, second largest
Binary search	Sorted data, “find position”	Search, first/last occurrence
Slow/fast pointers	Linked lists	Middle node, cycle detection
Stack	Matching, “most recent first”	Balanced brackets, undo, next greater element
Recursion + memo / DP	Overlapping subproblems	Fibonacci, climbing stairs, knapsack
Math formula	Avoiding loops	Missing number (n(n+1)/2), GCD (Euclid)

25.6 The 8 Complexities to Know

Operation	Complexity
Array/ArrayList access by index	O(1)
HashMap / HashSet get, put, contains	O(1) average
TreeMap / TreeSet / PriorityQueue operations	O(log n)
Binary search	O(log n)
Single loop over data	O(n)
Good sorting (merge, quick, Arrays.sort)	O(n log n)
Nested loops / bubble / selection / insertion sort	O(n ²)
Naive recursive Fibonacci	O(2 ⁿ)

25.7 Final Confidence Tips

- **Pause for two seconds before answering.** It looks thoughtful, not unsure.
- **Always offer an example** — “For example, in a banking app...” turns a definition into understanding.
- **Link to your projects.** “In my Spring Boot project, I used X to solve Y” is worth more than a textbook line — prepare one such link for OOP, Collections, exceptions and JDBC.
- **If you make a mistake, correct it calmly.** “Actually, let me correct that — ...” is a positive signal.
- **Ask clarifying questions before coding.** Input size, empty input, negatives, duplicates.
- **Say the complexity without being asked.**